Q1. **STREETCARE AI - POTHOLE AND STREET DEFECT REPORT REVIEW PLATFORM**

PROBLEM STATEMENT

Wrenford Borough Council looks after forty-eight named streets in four inspection districts: the **Town Centre**, **North**, **South** and the **Villages**. Residents, businesses, councillors, bus operators and schools report street defects through three channels, the council **Website**, the **StreetCare app** and the **Contact centre**, and every report joins one central report queue in the council's **Street Care Office**. Each of the five defect types, **Carriageway**, **Footway**, **Drainage**, **Vegetation** and **Street lighting**, has its own defect standard of exact requirements, and a defect that is two defect types at once is reported once, with the second as its **linked defect type**, and must meet both standards. Nothing is repaired until a **Highways Inspector** appointed for the report's district decides it.

The office has three problems. Every channel writes the report its own way: report references in three notations, depths in millimetres, centimetres or inches, heights and distances in metres or feet, and crew hours in hours or minutes, and a reading that is never converted meets or misses the wrong figure. The five defect standards share one template, and the carriageway and footway standards word their depth requirement alike but set different figures, so a requirement read from the wrong document passes and fails the wrong reports. And the documents attached to a report sometimes give a measurement other than the report's, such as a resident saying there is only 2.1 metres clear under the branches against a report that says 2.4, which a coordinator working through the queue can miss. Behind all three sits the street itself: a repair booked where street works are live in the same part of the highway, a carriageway or drainage repair on a traffic-sensitive street, and a repair to a lighting column too tall to reach from the footway must be coordinated first.

Customer services has a problem of its own. Its advisers answer residents' calls, and may be told a report's defect type, its street and its status and, once a repair is ordered, where on the street the defect is and what the resident has been told. They may never be told whether a defect is likely to come back or how long its repair will take, and nobody may be given a figure about a street that fewer than five reports make up, because such a figure points at the household that made the report.

Wrenford requires a system that screens every open report against its own defect type's standard and its linked type's, counts the street works live on its street, works out whether coordination must come first, reads every sentence of the documents attached to it, estimates how likely the defect is to be reported again and how many crew hours its repair will really take, prepares a review with a recommended decision, and then stops. No repair is ordered, written or sent on the word of a model.

OBJECTIVE

Build **StreetCare AI**, a working street defect report review platform that a street care office can use end to end. The finished system must:

- Clean five raw extracts that arrive with real data-quality defects, and produce the evidence that each one needed cleaning.
- Read every defect type's requirements, with their own figures, out of that defect type's own defect standard. No requirement is written into the code or the configuration.
- Read every answer on the report as the **Report Rules** read it: every depth and width in whole millimetres, every height and distance in metres, every lean in degrees and every crew time in hours.
- Screen each report under its own standard and its linked type's: every requirement **Met**, **Not met** or **Not documented**, the street works live on its street, the **coordination triggers**, the assessment, and the decisions it leaves open.
- Train two machine learning models on tabular data, one estimating the **repeat likelihood** of a defect and one estimating the **expected crew hours** its repair will really take, and one NLP model that names the type of each sentence of a report's documents.
- Build a searchable knowledge base from sixteen documents, so that every statement the system makes can be traced to the document it came from.
- Read every sentence of the documents attached to a report, record the requirements each one bears on, and flag every measurement that disagrees with the report or describes an answer the report left blank.
- Run a multi-agent workflow in which each agent is a single language model call that interprets facts the tools established, and calculates nothing of its own.

- Stop that workflow before anything is decided, wait for a Highways Inspector appointed for the report's district who did not open the review, and survive the machine being switched off while it waits.
- After the decision, write exactly what that decision calls for: a **works order** and a **resident update**, an **inspection request**, or a **coordination brief**.
- Record every run, every review, every decision and every language model call whose reply a review accepts, with the tokens it used and the time it took, and every question the chat replies to, in a database that can be audited afterwards.
- Serve all of it through a REST service and a dashboard, with every figure the dashboard shows behind that service.
- Require everyone to sign in, and show each of the three roles only the pages their role covers, with the service refusing the matching request on its own account.
- Answer questions from Wrenford's documents and from nothing else, answer every question about a report from the review records by the asker's role, give a figure about a street only where at least five reports make it up, and never give customer services a prediction.

PROJECT STRUCTURE

```
Project/
|-- .env                              <- pre-loaded, your key goes in it
|-- .streamlit/                       <- pre-loaded
|   '-- config.toml
|-- access/
|   |-- password_security.py
|   '-- permissions.py
|-- agentic/
|   |-- agents.py
|   |-- graph.py
|   |-- language_model.py
|   |-- nodes.py
|   '-- state.py
|-- api/
|   |-- routes.py
|   '-- schemas.py
|-- chat/
|   |-- conversation_memory.py
|   |-- record_answer.py
|   '-- street_care_assistant.py
|-- config/
|   '-- config.py                     <- pre-loaded, complete, do not edit
|-- dashboard/
|   |-- api_client.py
|   |-- audit_trail.py
|   |-- decision_queue.py
|   |-- report_detail.py
|   |-- report_queue.py
|   |-- sign_in.py
|   '-- street_care_chat.py
|-- data/                             <- pre-loaded
|   |-- docs/                         <- the sixteen PDF documents
|   |-- input/
|   |   |-- open_defect_reports.csv
|   |   |-- report_documents.csv
|   |   '-- street_works_register.csv
|   |-- repair_logs_eval.csv
|   |-- repair_logs_train.csv
|   |-- report_history_eval.csv
|   |-- report_history_train.csv
|   '-- statement_sentences.csv
|-- data_pipeline/
|   |-- common_cleaning.py
|   |-- dataset_loader.py
|   |-- document_cleaning.py
|   |-- feature_builder.py
|   |-- history_cleaning.py
|   |-- quality_profiler.py
|   |-- register_cleaning.py
|   |-- repair_log_cleaning.py
|   |-- report_cleaning.py
|   '-- sentence_cleaning.py
|-- database/                         <- empty
|   |-- streetcare.db                 <- created by run_database_setup.py
|   |-- checkpoint.db                 <- created by run_api_service.py
|   '-- chroma_store/                 <- created by run_knowledge_indexing.py
|-- knowledge/
|   |-- chunk_splitting.py
|   |-- pdf_extraction.py
|   |-- standard_reading.py
|   '-- vector_index.py
|-- modeling/
|   |-- crew_hours_model.py
|   |-- model_evaluation.py
|   |-- model_registry.py
|   |-- repeat_model.py
|   '-- sentence_model.py
|-- output/                           <- empty
|   |-- models/                       <- empty; filled by run_model_training.py
|   |-- data_quality_report.json      <- created by run_data_pipeline.py
|   '-- model_metrics.json            <- created by run_model_training.py
|-- persistence/
|   |-- database.py
|   |-- models.py
|   '-- repository.py
|-- processed/                        <- empty; filled by run_data_pipeline.py
|-- tools/
|   |-- decision_tools.py
|   |-- evidence_tools.py
|   |-- prediction_tools.py
|   |-- report_data_tools.py
|   |-- retrieval_tools.py
|   |-- review_document_tools.py
|   '-- screening_tools.py
|-- requirements.txt                  <- pre-loaded
|-- run_api_service.py
|-- run_dashboard.py
|-- run_data_pipeline.py
|-- run_database_setup.py
|-- run_knowledge_indexing.py
'-- run_model_training.py
```

DATASET DESCRIPTION

report_history_train.csv —
One row per revision of a decided defect report. This is what the repeat model learns from.

Column	Type once cleaned	What it holds
report_reference	**str**	The report, written **SC** and seven digits by the website, **SC-** and the number by the StreetCare app and the bare number by the contact centre; read as **SC** and seven digits. **config.REPORT_REFERENCE_COLUMN** names it
report_channel	**str**	**Website**, **StreetCare app** or **Contact centre**
revision_number	**int**	The revision of the report, from **1**. The highest revision is the report. **config.REVISION_NUMBER_COLUMN** names it
reporter_type	**str**	Resident, Business, Councillor, Bus operator or School
street_name	**str**	One of the council's forty-eight named streets. **config.STREET_NAME_COLUMN** names it
district	**str**	The street's inspection district: Town Centre, North, South or Villages
street_category	**str**	**Traffic sensitive** or **Local road**, written in each channel's words. **config.STREET_CATEGORY_COLUMN** names it
location_description	**str**	Where on the street, in the reporter's own words. **config.LOCATION_DESCRIPTION_COLUMN** names it
location_fixed	**str**	Whether the location is fixed to a house number, a lamp column number or a road junction
defect_type	**str**	One of the five defect types, written in full, as a description or as a code. **config.DEFECT_TYPE_COLUMN** names it
linked_defect_type	**str**	A second defect type, or **No linked defect**. **config.LINKED_DEFECT_TYPE_COLUMN** names it
photograph_attached	**str**	Whether a photograph is attached
defect_depth_mm	**int**	The depth or vertical step, in whole millimetres once read
defect_width_mm	**int**	The width across, in whole millimetres once read
water_depth_mm	**int**	The depth of standing water, in whole millimetres once read
water_location	**str**	Where the water stands: Carriageway or Footway. **config.WATER_LOCATION_COLUMN** names it
water_reach_m	**float**	How far the water reaches out from the kerb, in metres to one decimal once read. **config.WATER_REACH_COLUMN** names it
overhang_location	**str**	Where the vegetation overhangs: Carriageway or Footway. **config.OVERHANG_LOCATION_COLUMN** names it
clear_height_m	**float**	The clear height under the overhang, in metres to one decimal once read. **config.CLEAR_HEIGHT_COLUMN** names it
on_highway_land	**str**	Whether the tree or hedge grows on highway land
lighting_fault	**str**	**Lamp out**, **Day burning** or **Leaning column**. **config.LIGHTING_FAULT_COLUMN** names it
column_number_given	**str**	Whether the lamp column number is given
column_lean_degrees	**float**	The lean of the column from upright, in degrees to one decimal. **config.COLUMN_LEAN_COLUMN** names it
column_height_m	**float**	The column height from the lighting inventory, in metres to one decimal. **config.COLUMN_HEIGHT_COLUMN** names it
previous_reports_at_location	**int**	Earlier reports at the same location
planned_crew_hours	**float**	The crew hours planned from the council's standard times, in hours to one decimal. **config.PLANNED_CREW_HOURS_COLUMN** names it
live_street_works_on_street	**int**	Street works permits live on the street when the report was screened. **config.LIVE_STREET_WORKS_COLUMN** names it
conflicting_street_works	**int**	Of those, the permits in the same part of the highway as the report's defect types. **config.CONFLICTING_STREET_WORKS_COLUMN** names it
review_outcome	**str**	How the report was dealt with: **Repaired as reported**, **Repaired after inspection**, **Repaired after coordination** or **Closed after inspection**
reported_again	**str**	**The answer.** Whether the same defect was reported again after it was dealt with. **config.HISTORY_TARGET_COLUMN** names it
repeat_cause	**str**	Why it came back, where it did: Repair failed, Defect returned nearby, Regrowth, Blocked again, Lamp failed again or Worsened after inspection

report_history_train.csv: SAMPLE RECORDS

Column	Record 1	Record 2	Record 3
report_reference	SC0021454	SC-22705	25908
report_channel	Website	StreetCare app	Contact centre
revision_number	2	1	1
reporter_type	Councillor	Resident	Resident
street_name	Castle Street	windmill lane	Oakfield Drive
district	Town Centre	North	North
street_category	Local road	Residential street	LR
location_description	outside no. 78	by lamp column 20	outside no. 169
location_fixed	Yes	true	Y
defect_type	Vegetation	Pothole or road surface	CW
linked_defect_type	No linked defect	None	DR
photograph_attached	N/A	true	Y
defect_depth_mm	N/A	2.8 cm	1.5 in
defect_width_mm	N/A	null	caller unsure
water_depth_mm	N/A		2.5 in
water_location	N/A		ROAD
water_reach_m	N/A		7 ft
overhang_location	Footway		-
clear_height_m	not known		-
on_highway_land	Yes		-
lighting_fault	N/A		-
column_number_given	N/A		-
column_lean_degrees	N/A		-
column_height_m	N/A		-
previous_reports_at_location	1	0	0
planned_crew_hours	1.5	150 min	3 hrs
live_street_works_on_street	0	0	0
conflicting_street_works	0	0	0
review_outcome	Repaired after inspection	Closed - inspected	REP-INS
reported_again	N/A	false	N
repeat_cause	Regrowth		-

Three rows as they arrive, one from each channel, with the defects visible: a reference in each channel's notation, a street in lower case, the defect type as a description and as a code, a street category and an outcome in a channel's own words, yes and no written **true** and **Y**, readings in centimetres, inches, feet, minutes and hours, and "missing" written **N/A**, **not known**, **null**, **caller unsure**, **-** or left empty. The first row's answer is missing, so it cannot be learned from.

report_history_eval.csv —
The same **31** columns as **report_history_train.csv**, in the same order, held out. The repeat model is scored on this file and never fitted on it.

repair_logs_train.csv —
One row per repaired defect, recording the crew hours its repair took. This is what the crew hours model learns from.

Column	Type once cleaned	What it holds
report_reference	**str**	The report the repair belongs to, in the same three notations. Every row gives it, and no report appears twice
report_channel	**str**	The channel the report came through
street_name	**str**	The street
district	**str**	The street's inspection district
street_category	**str**	Traffic sensitive or Local road
defect_type	**str**	The defect type
linked_defect_type	**str**	The linked defect type, or No linked defect
defect_depth_mm	**int**	The depth or vertical step, in whole millimetres
defect_width_mm	**int**	The width across, in whole millimetres
water_depth_mm	**int**	The depth of standing water, in whole millimetres
water_location	**str**	Where the water stands
water_reach_m	**float**	How far the water reaches from the kerb, in metres
overhang_location	**str**	Where the vegetation overhangs
clear_height_m	**float**	The clear height under the overhang, in metres
lighting_fault	**str**	The lighting fault
column_height_m	**float**	The column height, in metres
column_lean_degrees	**float**	The lean of the column, in degrees
previous_reports_at_location	**int**	Earlier reports at the same location
live_street_works_on_street	**int**	Street works permits live on the street
conflicting_street_works	**int**	Of those, the permits that conflict
repair_method	**str**	Patch repair, Permanent reinstatement, Slab relay, Gully cleanse, Drain repair, Cut back, Lamp replacement, Photocell replacement or Column replacement
traffic_management	**str**	None, Signs only, Lane closure or Road closure
crew_size	**int**	The people in the crew
planned_crew_hours	**float**	The crew hours planned, in hours to one decimal
reported_again	**str**	Whether the defect was reported again
completion_code	**str**	**Completed**, **Completed with overrun** or **Completed after return visit**
actual_crew_hours	**float**	**The answer.** The crew hours the repair really took, from the crew's arrival to the site being cleared, in hours to one decimal. **config.LOG_TARGET_COLUMN** names it

repair_logs_train.csv: SAMPLE RECORDS

Column	Record 1	Record 2	Record 3
report_reference	SC0024239	SC-20085	27679
report_channel	Website	StreetCare app	Contact centre
street_name	Park Avenue	Oakfield Drive	southfield road
district	North	North	South
street_category	Traffic sensitive	Residential street	LR
defect_type	Drainage	Flooding or blocked drain	VG
linked_defect_type	No linked defect	None	FW
defect_depth_mm	N/A		1.25 in
defect_width_mm	N/A		11 in
water_depth_mm	not known	14.5 cm	-
water_location	Footway	Road	-
water_reach_m	N/A	null	-
overhang_location	N/A		PATH
clear_height_m	N/A		8 ft
lighting_fault	N/A		-
column_height_m	N/A		-
column_lean_degrees	N/A		-
previous_reports_at_location	0	0	caller unsure
live_street_works_on_street	0	0	0
conflicting_street_works	0	0	0
repair_method	Gully cleanse	Cleanse gully	CUT
traffic_management	Lane closure	Signs	NONE
crew_size	2	2	3
planned_crew_hours	2	90 min	3 hrs
reported_again	No	false	Y
completion_code	Completed with overrun	done	comp
actual_crew_hours	2.7	72 min	2.2 hrs

The same defects as the history: a repair method, a traffic management and a completion code each in its channel's own words, and crew hours in hours, in minutes and in **hrs**.

repair_logs_eval.csv —
The same **27** columns as **repair_logs_train.csv**, in the same order, held out. The crew hours model is scored on this file and never fitted on it.

statement_sentences.csv —
One row per sentence of a document attached to an older report, labelled by the street care office with its type. This is what the sentence model learns from.

Column	Type once cleaned	What it holds
sentence_id	**str**	The sentence's own identifier, **SEN-** and seven digits. Every row gives it, and no sentence appears twice. **config.SENTENCE_ID_COLUMN** names it
report_reference	**str**	The report the document belongs to, in the three notations. Every row gives it
report_channel	**str**	The channel the report came through
document_type	**str**	**Resident description**, **Call note**, **Photo note** or **Inspection note**
sentence_number	**int**	The sentence's place in its document, from **1**
sentence_text	**str**	The sentence. **config.SENTENCE_TEXT_COLUMN** names it
sentence_length_characters	**int**	The sentence's length in characters. **config.SENTENCE_LENGTH_COLUMN** names it, and in the raw file it was measured on the raw text
sentence_type	**str**	**The answer.** One of the nine sentence types, written several ways. **config.SENTENCE_TARGET_COLUMN** names it

statement_sentences.csv: SAMPLE RECORDS

Column	Record 1	Record 2	Record 3
sentence_id	SEN-6201357	SEN-6213400	SEN-6217536
report_reference	1196	SC0003241	SC-3926
report_channel	Contact centre	Website	StreetCare app
document_type	Call note	Resident description	Resident description
sentence_number	1	2	4
sentence_text	Caller reports that a pedestrian was hurt on Station Road.	It is halfway between London Road and the school.	Wet leaves from the tree make the path on Wool Lane slippery.
sentence_length_characters	58	49	61
sentence_type	INCIDENT	Location	Weather and water

Elsewhere in the file a sentence carries markup, escaped characters, no-break spaces or padding, is written in capitals, is too short to learn from, or has no type at all.

open_defect_reports.csv — 45 rows, 27 columns
The live queue, under **data/input/**, written exactly as each channel sent it. Once read, it holds the **40** open reports at their latest revision. **config.OPEN_DEFECT_REPORTS_FILE** names it.

Column	Type once cleaned	What it holds
report_reference	**str**	The report, in any of the three channels' notations
report_channel	**str**	**Website**, **StreetCare app** or **Contact centre**
revision_number	**int**	The revision of the report, from **1**; the highest revision is the report
reporter_type	**str**	Who reported the defect
street_name	**str**	One of the council's forty-eight named streets
district	**str**	The street's inspection district
street_category	**str**	**Traffic sensitive** or **Local road**, in each channel's words
location_description	**str**	Where on the street, in the reporter's own words
location_fixed	**str**	Whether the location is fixed to a house number, a lamp column number or a road junction
defect_type	**str**	One of the five defect types
linked_defect_type	**str**	A second defect type, or **No linked defect**
photograph_attached	**str**	Whether a photograph is attached
defect_depth_mm	**int**	The depth or vertical step, in whole millimetres once read
defect_width_mm	**int**	The width across, in whole millimetres once read
water_depth_mm	**int**	The depth of standing water, in whole millimetres once read
water_location	**str**	Where the water stands: Carriageway or Footway
water_reach_m	**float**	How far the water reaches out from the kerb, in metres once read
overhang_location	**str**	Where the vegetation overhangs: Carriageway or Footway
clear_height_m	**float**	The clear height under the overhang, in metres once read
on_highway_land	**str**	Whether the tree or hedge grows on highway land
lighting_fault	**str**	**Lamp out**, **Day burning** or **Leaning column**
column_number_given	**str**	Whether the lamp column number is given
column_lean_degrees	**float**	The lean of the column from upright, in degrees
column_height_m	**float**	The column height from the lighting inventory, in metres
previous_reports_at_location	**int**	Earlier reports at the same location
planned_crew_hours	**float**	The crew hours planned, in hours once read
supporting_document_count	**int**	How many documents are attached to the report. **config.SUPPORTING_DOCUMENT_COUNT_COLUMN** names it

The street works beside a report are not a column here: they are counted from the street works register when the report is screened. The file carries no answer column.

open_defect_reports.csv: SAMPLE RECORDS

Column	Record 1	Record 2	Record 3
report_reference	SC0052141	SC-52133	52183
report_channel	Website	StreetCare app	Contact centre
revision_number	1	1	1
reporter_type	Resident	Resident	Resident
street_name	Albert Street	Foundry Lane	Oakfield Drive
district	South	South	North
street_category	Local road	Residential street	LR
location_description	near the park entrance	near the shops	opposite no. 34
location_fixed	No	null	Y
defect_type	Footway	Street light fault	FW
linked_defect_type	No linked defect	None	VG
photograph_attached	Yes		Y
defect_depth_mm	35		1.25 in
defect_width_mm	not known		16 in
water_depth_mm	N/A		-
water_location	N/A		-
water_reach_m	N/A		-
overhang_location	N/A		PATH
clear_height_m	N/A		8 ft
on_highway_land	N/A		Y
lighting_fault	N/A	Column leaning	-
column_number_given	N/A	true	-
column_lean_degrees	N/A	3.5 deg	-
column_height_m	N/A	6 m	-
previous_reports_at_location	1	1	0
planned_crew_hours	2	270 min	2.5 hrs
supporting_document_count	1	1	2

report_documents.csv —

Column	Type once cleaned	What it holds
document_id	**str**	The document's own identifier, **DOC-** and six digits. Every row gives it, and no document appears twice. **config.DOCUMENT_ID_COLUMN** names it
report_reference	**str**	The report it is attached to, in the three notations. Every row gives it
report_channel	**str**	The channel the report came through
document_type	**str**	Resident description, Call note, Photo note or Inspection note. **config.DOCUMENT_TYPE_COLUMN** names it
document_title	**str**	The document's title. **config.DOCUMENT_TITLE_COLUMN** names it
document_text	**str**	The document, as plain words once read. **config.DOCUMENT_TEXT_COLUMN** names it

report_documents.csv: SAMPLE RECORDS

Column	Record 1	Record 2	Record 3
document_id	DOC-841268	DOC-841264	DOC-841220
report_reference	SC-52128	SC-52197	SC0052176
report_channel	StreetCare app	StreetCare app	Website
document_type	Resident description	Resident description	Resident description
document_title	Resident description - Vegetation - Long Lane	Resident description - Drainage - Park Avenue	Resident description - Carriageway - Back Lane
document_text	It is on the south side of Long Lane. There is only 5.1 m clear above the road. Children walk along Long Lane to the primary school. The hedge on Long Lane was cut back but it has grown out again. Just to say, buses use Long Lane from early on. A previous repair here has worn away. It is outside number 133.	I am reporting this for my father. Hope this helps. Rain water runs down the bank and collects here. I fell over here and cut my hand.	This is the second time this has been reported by us. It is roughly 1400 mm wide. Someone has already tripped here. Just to say, it could easily cause an accident on Back Lane. There is already a report about this on Back Lane, but nothing has happened.

street_works_register.csv — 29 rows, 6 columns

The street works permits on the council's streets that are not yet closed, under **data/input/**. Once read, it holds **28** permits, **23** of them live. **config.STREET_WORKS_REGISTER_FILE** names it.

Column	Type once cleaned	What it holds
permit_reference	**str**	The permit, **SWP** and seven digits. Every row gives it, and no permit appears twice. **config.PERMIT_REFERENCE_COLUMN** names it
street_name	**str**	The street the works are on. Every row gives it
works_location	**str**	The part of the highway the works are in: Carriageway or Footway. Every row gives it. **config.WORKS_LOCATION_COLUMN** names it
works_promoter	**str**	Water company, Gas network, Electricity network, Telecoms or Council contractor
works_description	**str**	What the works are
permit_status	**str**	**Live** or **Cancelled**. Every row gives it. **config.PERMIT_STATUS_COLUMN** names it

street_works_register.csv: SAMPLE RECORDS

Column	Record 1	Record 2	Record 3
permit_reference	SWP0071240	SWP0071240	SWP0071246
street_name	Mill Lane	Mill Lane	Mill Lane
works_location	Carriageway	Carriageway	Carriageway
works_promoter	Water company	Water company	Telecoms
works_description	Water main repair	Water main repair	Duct repair
permit_status	Live	Live	Cancelled

The first two rows are the same permit written twice; elsewhere a value arrives in capitals or padded with spaces.

THE DEFECTS IN THE RAW EXTRACTS

Every defect below is present in the files. Where **config/config.py** already holds what is needed to repair one, the constants are named; a dash means the repair needs no lookup. Nothing downstream may assume a clean table.

Defect	How it looks	Where the repair values live
Padding and case	**" Church Street "**, **"windmill lane"**, **"CARRIAGEWAY"**	**config.FORM_VALUE_MAPPINGS**, **config.HISTORY_VALUE_MAPPINGS**, **config.LOG_VALUE_MAPPINGS**, **config.SENTENCE_VALUE_MAPPINGS**, **config.DOCUMENT_VALUE_MAPPINGS**, **config.REGISTER_VALUE_MAPPINGS**
Answers in a channel's own words	A defect type as a description or a code (**"Pothole or road surface"**, **"CW"**), a street category (**"Main route"**, **"LR"**), where water stands (**"Road"**, **"PATH"**), a fault (**"Light not working"**, **"LEAN"**), an outcome (**"Closed - inspected"**, **"REP-INS"**), a repair method, a traffic management, a completion code	**config.DEFECT_TYPE_MAPPING**, **config.LINKED_DEFECT_TYPE_MAPPING**, **config.STREET_CATEGORY_MAPPING**, **config.HIGHWAY_PART_MAPPING**, **config.LIGHTING_FAULT_MAPPING**, **config.REVIEW_OUTCOME_MAPPING**, **config.REPEAT_CAUSE_MAPPING**, **config.REPAIR_METHOD_MAPPING**, **config.TRAFFIC_MANAGEMENT_MAPPING**, **config.COMPLETION_CODE_MAPPING**
Yes and no written many ways	**"Yes"**, **"true"**, **"Y"**, **"1"**, **"No"**, **"false"**, **"N"**, **"0"**	**config.BOOLEAN_MAPPING**, **config.BOOLEAN_YES**, **config.BOOLEAN_NO**
Missing written many ways	**"N/A"**, **"not known"**, **"null"**, **"-"**, **"caller unsure"**, blank	**config.NULL_TOKENS**
References in three notations	**"SC0021454"**, **"SC-22705"**, **"25908"**	**config.REPORT_REFERENCE_PATTERN**, **config.REPORT_REFERENCE_PREFIX**, **config.REPORT_REFERENCE_DIGITS**
Readings in a channel's units	**"2.8 cm"**, **"1.5 in"**, **"7 ft"**, **"6 m"**, **"3.5 deg"**, **"150 min"**, **"3 hrs"**	**config.FORM_READINGS**, **config.LOG_READINGS**, with **config.MILLIMETRE_READING**, **config.METRE_READING**, **config.DEGREE_READING** and **config.CREW_HOURS_READING**
Whole numbers written as text	A revision, a count of previous reports, permits or documents, a crew size, a sentence's place	**config.FORM_WHOLE_NUMBER_PATTERNS**, **config.HISTORY_WHOLE_NUMBER_PATTERNS**, **config.LOG_WHOLE_NUMBER_PATTERNS**, **config.SENTENCE_WHOLE_NUMBER_PATTERNS**, **config.LIVE_WHOLE_NUMBER_PATTERNS**
Impossible readings	A negative depth, a width of nought, negative standing water, a clear height or a column height of nought, a lean of more than **config.LEAN_MAXIMUM** degrees, planned crew hours of nought, a negative count of previous reports, a crew of nobody	**config.FORM_MINIMUM_VALUES**, **config.FORM_MAXIMUM_VALUES**, **config.LOG_MINIMUM_VALUES**, **config.LOG_MAXIMUM_VALUES**
Earlier revisions of a report	The same reference again with a lower **revision_number**	**config.REPORT_REFERENCE_COLUMN**, **config.REVISION_NUMBER_COLUMN**
Duplicated rows	The same record twice, identical in every column	—
Missing answer column	A history row with no **reported_again**, a repair log with no **actual_crew_hours**, a sentence with no **sentence_type**	**config.HISTORY_TARGET_COLUMN**, **config.LOG_TARGET_COLUMN**, **config.SENTENCE_TARGET_COLUMN**
Impossible answer	Negative actual crew hours	**config.LOG_MINIMUM_VALUES**
Sentence types written several ways	**"INCIDENT"**, **"Traffic & use"**, **"Where"**, **"Context"**	**config.SENTENCE_TYPE_MAPPING**
Identifiers of a fixed form	**SEN-**, **DOC-** and **SWP** references that must keep their form	**config.SENTENCE_IDENTIFIER_PATTERNS**, **config.DOCUMENT_IDENTIFIER_PATTERNS**, **config.REGISTER_IDENTIFIER_PATTERNS**
Markup in free text	**** and ** ** around and inside a sentence or a document	**config.MARKUP_PATTERN**
Escaped characters and odd spacing	** **, **-**, no-break spaces, padding	**config.WHITESPACE_RUN_PATTERN**; each entity is read as its character, and **config.ENCODING_ARTIFACT_PATTERN** finds them for the quality report
Sentences written in capitals	A whole sentence shouted, which can be restored only in lower case	—
Sentences too short to learn from	A sentence of fewer than **config.MINIMUM_SENTENCE_WORDS** words	**config.MINIMUM_SENTENCE_WORDS**
A stale derived column	**sentence_length_characters** measured on the raw text	**config.SENTENCE_LENGTH_COLUMN**

Some gaps are facts rather than defects: a water depth on a footway report, a column height on a carriageway report, a repeat cause on a defect that never came back. Those stay empty in the cleaned tables, and the preparation saved with each model fills them when it is fitted and when it scores.

THE SIXTEEN DOCUMENTS

The documents sit under **data/docs/** as PDFs. They are Wrenford's own, and they are the only source of any requirement, rule or check the system states.

Document	What it settles
Defect-Standard-Carriageway.pdf	The carriageway defect type, its requirements and their figures, what an inspection records, the works order and the resident update
Defect-Standard-Footway.pdf	The same for the footway, whose standard shares the carriageway's template and differs in its depth figure
Defect-Standard-Drainage.pdf	The same for drainage: standing water and how far it reaches from the kerb
Defect-Standard-Vegetation.pdf	The same for vegetation: highway land and the clear height over a footway and over a carriageway
Defect-Standard-Street-Lighting.pdf	The same for street lighting: the column number and the lean of a leaning column
Highways-Policy.pdf	Who opens and who decides a review in each inspection district, the street works register and the coordination triggers, the three decisions, the documents each one writes, the report queue, the two figures, what customer services may be told, and the records kept
Report-Rules.pdf	How each channel writes each answer and what it is read as, references, readings and their units, revisions, how each requirement is judged, and how statements are read against the report
Past-Report-Decisions-Casebook.pdf	Closed reports and how each was decided. It quotes figures of its own, and it is the one document the chat may not read
Guide-Measurement.pdf	What a measurement sentence covers, how it shows up, what the reviewer checks, the requirements it bears on, when it disagrees with the report and how to tell it apart
Guide-Location.pdf	The same, for a location sentence
Guide-Photograph.pdf	The same, for a photograph sentence
Guide-Hazard.pdf	The same, for a hazard sentence
Guide-Incident.pdf	The same, for an incident sentence
Guide-Traffic-And-Use.pdf	The same, for a traffic and use sentence
Guide-Previous-Reports.pdf	The same, for a previous reports sentence
Guide-Weather-And-Water.pdf	The same, for a weather and water sentence

Every document opens with **WRENFORD BOROUGH COUNCIL** and its own title, then one header line of a fixed form, matched by **config.DOCUMENT_HEADER_PATTERN**, and is divided into sections numbered from **1**, matched by **config.SECTION_HEADING_PATTERN**.
The five defect standards are written from one template: the same headings, and in places the same sentences. Section **config.DEFECT_TYPE_SECTION_NUMBER** of each states its defect type in one sentence that **config.DEFECT_TYPE_PATTERN** reads; section **config.REQUIREMENTS_SECTION_NUMBER** lists its requirements in order, each written in one of the sentence forms of **config.REQUIREMENT_SENTENCE_PATTERNS**; and sections **config.INSPECTION_RECORDS_SECTION_NUMBER**, **config.WORKS_ORDER_SECTION_NUMBER** and **config.RESIDENT_UPDATE_SECTION_NUMBER** say what an inspection records, what a works order says and what a resident update says. **config.DEFECT_STANDARD_DOCUMENTS** names each defect type's standard, **config.SENTENCE_TYPE_GUIDE_DOCUMENTS** the guide for each sentence type that has one, **config.HIGHWAYS_POLICY_DOCUMENT** the policy, and **config.CHAT_EXCLUDED_DOCUMENTS** the casebook. The configuration holds the sentence forms. It holds no requirement and no figure: every one of them is read from its own standard at run time.

DOMAIN KNOWLEDGE CONTEXT

Nobody is expected to arrive knowing highway maintenance. This section gives only what is needed to read the requirements that follow. Every value named here already exists in **config/config.py**, except the requirements and their figures, which exist only in the defect standards.

WHAT A STREET DEFECT REPORT IS

A street defect report is someone telling the council that something on a street needs putting right: a pothole, a broken or rocking paving slab, standing water from a blocked gully, a tree or hedge hanging too low, or a street light that is out, burning by day or leaning. The report gives the street, where on it the defect is, the defect type and the answers that defect type's standard asks for. The council must decide whether the repair can be ordered straight from the report, whether an inspector must visit first, or whether the repair must be coordinated with other works on the street.

FIVE DEFECT TYPES, FIVE STANDARDS

The five defect types are **config.CARRIAGEWAY**, **config.FOOTWAY**, **config.DRAINAGE**, **config.VEGETATION** and **config.STREET_LIGHTING**, and each has its own defect standard, named by **config.DEFECT_STANDARD_DOCUMENTS**. A defect that is two defect types at once is reported once, under the defect type the reporter chooses, with the other as its linked defect type, and must meet both standards; a report with no second type gives **config.NO_LINKED_DEFECT**. No defect type is linked to itself.

THE REQUIREMENTS ARE NOT IN THE CODE
Each standard lists its requirements in its section **config.REQUIREMENTS_SECTION_NUMBER**, numbered from **1**, each a name and one sentence, in one of the sentence forms **config.REQUIREMENT_SENTENCE_PATTERNS** gives for the ten requirement kinds, from **config.LOCATION_FIXED_REQUIREMENT** to **config.COLUMN_LEAN_REQUIREMENT**. A sentence form holds the words; the figures inside it, a minimum, a maximum, a footway limit or a carriageway limit, are read from the standard and kept as exact decimals under **config.MINIMUM_RULE**, **config.MAXIMUM_RULE**, **config.FOOTWAY_LIMIT_RULE** and **config.CARRIAGEWAY_LIMIT_RULE**. The carriageway and footway standards word their depth requirement alike and set different figures, which is why a requirement is only ever read from the report's own standard.

THREE CHANNELS, ONE READING
Every channel writes the same report its own way, and the Report Rules read each answer the same whichever channel wrote it.

- A report reference is written **SC0050118** by the website, **SC-50118** by the StreetCare app and **50118** by the contact centre, and is read as **config.REPORT_REFERENCE_PREFIX** and **config.REPORT_REFERENCE_DIGITS** digits.
- A depth, a width and standing water are read as whole millimetres: centimetres are multiplied by **config.MILLIMETRES_PER_CENTIMETRE** and inches by **config.MILLIMETRES_PER_INCH**.
- A reach, a clear height and a column height are read as metres to one decimal: feet are multiplied by **config.METRES_PER_FOOT**.
- A lean is read as degrees to one decimal, and crew hours as hours to one decimal: minutes are divided by **config.MINUTES_PER_HOUR**.
- Every conversion is worked as an exact decimal and rounded half up once, at the reading's own precision: **4.5 cm** is **45** mm, **1.75 in** is **44** mm, **4 ft** is **1.2** m and **45 min** is **0.8** hours.
- A spelling the Report Rules do not list is refused, naming it, never guessed.

THE LATEST REVISION
A report sent again with an answer added or put right comes with a higher **revision_number**. The highest revision is the report, wherever its rows sit in the file; an earlier revision is never screened and never learned from. A row repeated exactly is one row.

MET, NOT MET AND NOT DOCUMENTED
Each requirement of the report's own standard, and of its linked type's, is **config.REQUIREMENT_MET**, **config.REQUIREMENT_NOT_MET** or **config.REQUIREMENT_NOT_DOCUMENTED**, judged on the report's answers named by **config.REQUIREMENT_ANSWER_COLUMNS**.

- A yes-or-no requirement is Met on Yes, Not met on No and Not documented when the answer is blank.
- A requirement written **at least** is met at its figure; one written **at least and at most** is met at either figure; one written **less than** is not met at its figure.
- The water reach, **config.WATER_REACH_REQUIREMENT**, is decided by where the water stands: Met on **config.FOOTWAY_PART**; on **config.CARRIAGEWAY_PART**, Met at or beyond the figure and Not met short of it.
- The clear height, **config.CLEAR_HEIGHT_REQUIREMENT**, is held to the footway limit over the footway and to the carriageway limit over the carriageway.
- The column lean, **config.COLUMN_LEAN_REQUIREMENT**, is Met for a lamp that is out or burns by day, and for **config.LEANING_COLUMN** is held to the figure.
- A requirement Not documented names the blank answers it needs, and only those: a blank water location, overhang location or lighting fault is named alone, whatever else is blank; otherwise the blank reach, clear height, lean or answer is named. A requirement Met or Not met names none.
- A report with a linked type is screened under its own standard first and then under its linked type's, and each requirement keeps the standard it came from.

THE STREET WORKS REGISTER AND THE COORDINATION TRIGGERS
The register lists each permit's street, the part of the highway its works are in and whether it is **config.PERMIT_STATUS_LIVE** or **config.PERMIT_STATUS_CANCELLED**. Only a live permit on the report's street counts, and it conflicts when its works are in the part of the highway **config.DEFECT_TYPE_HIGHWAY_PARTS** gives one of the report's defect types; vegetation is in neither part. Coordination must come first when any of the three triggers applies:

Trigger	When it applies
config.LIVE_STREET_WORKS_TRIGGER	At least one conflicting permit is live on the street
config.TRAFFIC_SENSITIVE_TRIGGER	The report, or its linked type, is one of **config.TRAFFIC_SENSITIVE_DEFECT_TYPES**, and the street is not **config.LOCAL_ROAD**
config.HIGH_COLUMN_TRIGGER	The report, or its linked type, is street lighting, on a column taller than **config.HIGH_COLUMN_TALLER_THAN_METRES** metres

A blank answer never lets a report slip past coordination: a blank street category counts as traffic sensitive, and a blank column height counts as taller than **config.HIGH_COLUMN_TALLER_THAN_METRES** metres.

THE ASSESSMENT AND THE THREE DECISIONS
Any requirement Not met or Not documented makes the assessment **config.DECISION_INSPECT**; otherwise any coordination trigger makes it **config.DECISION_REFER**; otherwise it is **config.DECISION_ORDER**. **config.DECISIONS_OPEN** gives the decisions each assessment leaves open:

Decision	What it means	Open when
ORDER REPAIR	The repair goes ahead from the report; a works order and then a resident update are written	Only when the assessment is **ORDER REPAIR**
INSPECT ON SITE	An inspector visits and measures before anything is repaired; an inspection request is written	Always, because a report is never closed from the desk
REFER FOR COORDINATION	The report goes to the network coordination team with its review and a coordination brief	Always

THE WORKED EXAMPLE

Section 13 of the Report Rules works one report through a made-up standard that belongs to no defect type: a depth of at least **30** mm, a width of at least **100** and at most **1000** mm, standing water at least **40** mm deep, and a reach of at least **1.5** m where the water stands on the carriageway. A contact centre report written **CW** linked **DR**, on a street written **TS**, gives a depth of **1.25 in** (31.75, so **32** mm: Met), a width of **3.9 in** (99.06, so **99** mm: Not met), standing water of **1.5 in** (38.1, so **38** mm: Not met), water on the **ROAD** and a reach written **caller unsure** (Not documented, asking for the reach alone). That is Met **1**, Not met **2**, Not documented **1**, so the assessment is **INSPECT ON SITE**; the traffic-sensitive street trigger applies too, but the requirements come first.

THE REPEAT LIKELIHOOD AND ITS BAND

The repeat likelihood is the repeat model's estimate, learned from the report history with each report's requirement counts, street works and trigger flags, of the chance that this defect is reported again after its repair. It is the model's probability rounded half up to **config.LIKELIHOOD_STEP**, and its band is **config.HIGH_LIKELIHOOD_BAND** from **config.HIGH_LIKELIHOOD_MINIMUM**, **config.MEDIUM_LIKELIHOOD_BAND** from **config.MEDIUM_LIKELIHOOD_MINIMUM**, and **config.LOW_LIKELIHOOD_BAND** below. It informs the Highways Inspector and decides nothing.

THE EXPECTED CREW HOURS AND THE OVERRUN LINE

The expected crew hours are the crew hours model's estimate, learned from the repair logs, of the time this repair will take on site, rounded half up to **config.TENTH_STEP**. An overrun is expected when the expected crew hours are more than **config.OVERRUN_RATIO** times the planned crew hours; when the report gives no planned crew hours, no overrun is worked out. **config.OVERRUN_TEMPLATE**, **config.NO_OVERRUN_TEMPLATE** and **config.EXPECTED_HOURS_WITHOUT_PLAN_TEMPLATE** word the figure. Like the likelihood, it decides nothing.

READING A STATEMENT

Every document attached to a report is read as plain words and split into sentences where **config.STATEMENT_SENTENCE_PATTERN** marks the end of one. The sentence model gives each sentence one of the nine types of **config.SENTENCE_TYPE_REQUIREMENT_KINDS**, which also names the requirement kinds each type bears on; hazard, incident, traffic and use, previous reports and background bear on none.

Only a sentence typed **config.MEASUREMENT** is searched for figures, in the four forms of **config.STATEMENT_MEASUREMENT_PATTERNS**: a depth, a width, standing water and a clear height. Each figure is compared, unrounded and as an exact decimal, with the report's own reading of the same answer. A sentence whose answer the report left blank is flagged **config.DESCRIBES_BLANK_ANSWER**; otherwise one with a figure that differs is flagged **config.DISAGREES_WITH_FORM**. A flag never changes an answer.

THE DOCUMENTS EACH DECISION WRITES

Decision	What is written	What it carries
ORDER REPAIR	A **works order** to the council's repair contractor, then a **resident update** for the reporter	The report reference, the defect type and any linked type, the street, the location, the defect's own measurements and the repair the standard's section **config.WORKS_ORDER_SECTION_NUMBER** names; the update follows its section **config.RESIDENT_UPDATE_SECTION_NUMBER**
INSPECT ON SITE	An **inspection request** to the inspector who will visit	Every requirement Not met or Not documented with the report's answers and the answers it needs, what each standard's section **config.INSPECTION_RECORDS_SECTION_NUMBER** says an inspection records, and every flagged sentence
REFER FOR COORDINATION	A **coordination brief**, built by the code from the report and the tools' findings	The report's details, the requirements and their results, the triggers, the live and conflicting permits, the flags and the two figures

No works order, inspection request or resident update carries a figure from a model, the resident update names no member of the council's staff, and every document a model writes is signed **config.DOCUMENT_SIGN_OFF**.

THE REPORT QUEUE

The queue lists every open report, whatever its review has reached. Reports whose defect type or linked type is **config.QUEUE_FIRST_DEFECT_TYPE** come first, then every other report, each group in order of its report reference, so that the oldest report is reviewed first. The queue is never ordered by a model's figure. A report is

config.REPORT_STATUS_AWAITING_REVIEW until its review is stored at the gate, **config.REVIEW_STATUS_AWAITING** from then, and then the status **config.REVIEW_STATUS_BY_DECISION** gives its decision.

THE THREE ROLES

Role	Permissions	Pages
config.ROLE_HIGHWAYS_INSPECTOR	Queue, review, decision, audit and chat; opens and decides reviews only for reports on the streets of its own district, and never decides a review it opened	All five pages
config.ROLE_STREET_CARE_COORDINATOR	Queue, review, audit and chat; opens reviews for any report and never decides one	Every page but the decision queue
config.ROLE_CUSTOMER_SERVICES_ADVISER	Chat only	The street care chat

config.ROLE_PERMISSIONS gives each role its permissions, **config.ROLE_PAGE_ACCESS** its pages, and **config.PERMISSION_BY_ROUTE** the permission of every route except signing in and signing out. **config.INSPECTION_DISTRICT_STREETS** gives the streets of each of the four districts, and each Highways Inspector's district is given in **config.SEED_USER_ACCOUNTS**; two Highways Inspectors are appointed for each district, so that one can decide what the other opened.

THE FIGURE LINE

Customer services speaks to residents, so what it hears is limited by the Highways Policy, and the code, never the model, writes every answer about a report or a street.

- A Customer Services Adviser asking about a named report hears its defect types, its street and its status, worded by **config.CHAT_ADVISER_STATUS_TEMPLATE** with the phrase **config.REPORT_STATUS_PHRASES** gives, and, once a repair is ordered, **config.CHAT_ORDERED_REPAIR_TEMPLATE**: where on the street the defect is and what the resident has been told.
- Asked whether a defect will come back or how long its repair will take, a Customer Services Adviser hears **config.OUTCOME_NOT_PREDICTED_LINE** and nothing more.
- A Highways Inspector or a Street Care Coordinator asking about a report is given the review's record, figures included.
- A reference that matches no open report is answered **config.CHAT_NO_REPORT_TEMPLATE**.
- Anyone asking how many defects of one type have been reported on a street is given the figure only when at least **config.GROUP_SIZE_MINIMUM** reports make it up, counted by the defect type alone from the report history, never from the queue; below that everyone hears **config.GROUP_SIZE_LINE**.
- A question the documents do not answer is answered **config.CHAT_REFUSAL_MESSAGE**.

A SHORT GLOSSARY

Term	Meaning
Defect report	A report of a defect on one of the council's streets
Report reference	The report's identifier, **SC** and seven digits once read
Channel	The Website, the StreetCare app or the Contact centre
Defect type	Carriageway, Footway, Drainage, Vegetation or Street lighting
Linked defect type	A second defect type the same defect is, screened under its own standard too
Defect standard	The document that sets a defect type's requirements and their figures
Requirement	One condition of a standard, Met, Not met or Not documented on a report
Revision	A report sent again; the highest revision is the report
Street works register	The utility and contractor permits on the council's streets, Live or Cancelled
Conflicting permit	A live permit on the report's street in the same part of the highway as its defect type or its linked defect type
Coordination trigger	A condition that makes coordination come first: live street works, a traffic-sensitive street or a high column
Assessment	The decision the screening's results point to
Decisions open	The decisions a Highways Inspector may record for an assessment
Statement	A sentence of a document attached to a report
Flag	A measurement sentence that disagrees with the report or describes an answer it left blank
Repeat likelihood	How likely a defect is to be reported again, with its band
Expected crew hours	How long the repair is expected to take on site
Review	The agents' write-up and recommended decision, waiting at the gate
Works order	The document that orders the repair from the contractor
Resident update	What the reporter is told once a repair is ordered
Inspection request	The document that sends an inspector to settle what the report left open
Coordination brief	The record sent to the network coordination team

TASKS

For each module the responsibility is stated, and then every function within it is specified by its signature, its parameters, its purpose, the requirements its result must satisfy, and what it returns.
Names matter. Every function name, parameter name, module path and dictionary key given below is read by other parts of the system and by the marking suite, and must be reproduced exactly as written, including case.
Every path, spelling correction, validation limit, threshold, vocabulary, pattern and business rule already exists in **config/config.py**, which is supplied complete.

========== DATA PREPARATION ==========

MODULE: data_pipeline/dataset_loader.py
The only data preparation module that reads or writes CSV files. Raw files are read without guessing any column's type; cleaned files are read with each column's type taken from its contents.

FUNCTION SIGNATURE
`load_raw_dataset(file_path)`
PARAMETERS
file_path — **str**. The path of one raw extract under **data/**, or of a live file under **data/input/**, passed from config by **run_data_pipeline.py** or by the live-file readers of **tools/report_data_tools.py**, for example **config.RAW_REPORT_HISTORY_TRAIN_FILE** or **config.STREET_WORKS_REGISTER_FILE**.
PURPOSE
Read one raw file as text, guessing nothing.
REQUIREMENTS
- Every column holds text, and no value has leading or trailing whitespace.
- A value matching one of **config.NULL_TOKENS**, ignoring case and surrounding spaces, is missing (NaN); no other value is.
- The rows and columns are exactly those of the file on disk.

RETURNS
pandas.DataFrame. Every row of the named file, every column held as text and every missing value as NaN.

FUNCTION SIGNATURE

`save_processed_dataset(dataframe, file_path)`

PARAMETERS
dataframe — **pandas.DataFrame**. A finished cleaned table, passed by **run_data_pipeline.py**.
file_path — **str**. The destination under **processed/**, passed by **run_data_pipeline.py** from config, for example **config.CLEAN_REPAIR_LOGS_TRAIN_FILE**.
PURPOSE
Save a cleaned table where the modelling stage will look for it.
REQUIREMENTS

- The file's folder exists after the call, whether or not it existed before.
- The file holds the table's own columns and no added index column.

RETURNS
None.

FUNCTION SIGNATURE
`load_processed_dataset(file_path)`

PARAMETERS
file_path — **str**. The path of a cleaned table under **processed/**, passed from config by **run_model_training.py** or by **load_report_history**, for example **config.CLEAN_STATEMENT_SENTENCES_FILE**.
PURPOSE
Read a table the pipeline has already cleaned.
REQUIREMENTS

- Column types follow the file's contents, and only an empty cell is missing, so a canonical value such as **None** stays text.
- Every row and column of the file is returned, unfiltered and unsorted.

RETURNS
pandas.DataFrame. The file as it stands on disk.

MODULE: data_pipeline/common_cleaning.py
The repairs the cleaning runs share. Every function here returns a repaired table or column and leaves the one it was given untouched; none reads or writes a file. A value in a notation a function does not recognise is refused with a **ValueError** naming the column and the values.

FUNCTION SIGNATURE
`remove_duplicate_records(dataframe)`

PARAMETERS
dataframe — **pandas.DataFrame**. A raw table as **load_raw_dataset** returned it, passed by a cleaning function of this package.
PURPOSE
Drop records written twice.
REQUIREMENTS

- Rows identical in every column appear once, the first kept; rows that differ anywhere are all kept.
- The result is numbered from nought upwards with no gaps.

RETURNS
pandas.DataFrame. The table without exact duplicates.

FUNCTION SIGNATURE
`refuse_missing_values(dataframe, required_columns)`

PARAMETERS
dataframe — **pandas.DataFrame**. The table a cleaning run has reached, passed by that cleaning function.
required_columns — **list[str]**. The columns every row must give, written by that cleaning function from config, for example **config.REGISTER_REQUIRED_COLUMNS**.
PURPOSE
Stop a cleaning run when a column that identifies a row is blank.
REQUIREMENTS

- A blank in any of **required_columns** is refused, naming the column and how many rows leave it blank.
- A table with no such blank is returned unchanged.

RETURNS
pandas.DataFrame. The table as given.

FUNCTION SIGNATURE
`refuse_repeated_references(dataframe, reference_column)`

PARAMETERS
dataframe — **pandas.DataFrame**. The table a cleaning run has reached, passed by that cleaning function after duplicates are removed.
reference_column — **str**. The column that must be unique, passed by that cleaning function from config, for example **config.PERMIT_REFERENCE_COLUMN**.
PURPOSE
Stop a cleaning run when two different rows claim the same reference.

REQUIREMENTS

- A value of **reference_column** on more than one row is refused, naming **reference_column** and every such value as the table holds it.
- A table whose references are all different is returned unchanged.

RETURNS

pandas.DataFrame. The table as given.

FUNCTION SIGNATURE

```
remove_rows_missing_target(dataframe, target_column)
```

PARAMETERS

dataframe — **pandas.DataFrame**. The table a cleaning run has reached, passed by that cleaning function.
target_column — **str**. The answer column, passed by that cleaning function from config, for example **config.LOG_TARGET_COLUMN**.

PURPOSE

Drop the rows a model cannot learn from.

REQUIREMENTS

- A row whose **target_column** is missing is removed, and every other row is kept in order.
- The result is numbered from nought upwards with no gaps.

RETURNS

pandas.DataFrame. The rows that give the answer.

FUNCTION SIGNATURE

```
normalize_category_values(dataframe, value_mappings)
```

PARAMETERS

dataframe — **pandas.DataFrame**. The table a cleaning run has reached, passed by that cleaning function or by **read_history_answers** or **read_log_answers**.
value_mappings — **dict[str, dict]**. Each category column paired with its mapping, passed from config, for example **config.FORM_VALUE_MAPPINGS**.

PURPOSE

Fold every spelling a channel writes onto its one canonical value.

REQUIREMENTS

- Every value of a mapped column is read whatever its case, and becomes the canonical value its mapping gives.
- A value the mapping does not list is refused, naming the column and the values; a missing value stays missing.
- Columns the mappings do not name are left as they are.

RETURNS

pandas.DataFrame. The table with every mapped column canonical.

FUNCTION SIGNATURE

```
convert_report_references(references)
```

PARAMETERS

references — **pandas.Series**. A table's **report_reference** column, passed by a cleaning function of this package.

PURPOSE

Read a report reference written in any channel's notation as one canonical reference.

REQUIREMENTS

- A reference in any notation **config.REPORT_REFERENCE_PATTERN** accepts becomes **config.REPORT_REFERENCE_PREFIX** followed by its number in **config.REPORT_REFERENCE_DIGITS** digits, so **SC0050118**, **SC-50118** and **50118** are all **SC0050118**.
- A reference in no such notation is refused, naming the column and the values; a missing reference stays missing.

RETURNS

pandas.Series. The references, canonical, on the same index.

FUNCTION SIGNATURE

```
refuse_unreadable_identifiers(dataframe, identifier_patterns)
```

PARAMETERS

dataframe — **pandas.DataFrame**. The table a cleaning run has reached, passed by that cleaning function.
identifier_patterns — **dict[str, str]**. Each identifier column paired with the form it must take, passed from config, for example **config.DOCUMENT_IDENTIFIER_PATTERNS**.

PURPOSE

Stop a cleaning run when an identifier is not in its one fixed form.

REQUIREMENTS

- A value that does not match its column's pattern in full is refused, naming the column and the values.
- A table whose identifiers all match is returned unchanged.

RETURNS

pandas.DataFrame. The table as given.

FUNCTION SIGNATURE

```
convert_whole_numbers(dataframe, number_patterns)
```

PARAMETERS
dataframe — **pandas.DataFrame**. The table a cleaning run has reached, passed by that cleaning function or by **read_history_answers** or **read_log_answers**.
number_patterns — **dict[str, str]**. Each whole-number column paired with the pattern its values must match, passed from config, for example **config.LOG_WHOLE_NUMBER_PATTERNS**.
PURPOSE
Read counts and revisions as whole numbers.
REQUIREMENTS
- Each named column holds whole numbers of **config.WHOLE_NUMBER_DTYPE**, a missing value staying missing.
- A value its pattern does not match is refused, naming the column and the values.

RETURNS
pandas.DataFrame. The table with every named column as whole numbers.

FUNCTION SIGNATURE
`convert_readings(dataframe, readings)`

PARAMETERS
dataframe — **pandas.DataFrame**. The table a cleaning run has reached, passed by that cleaning function or by **read_history_answers** or **read_log_answers**.
readings — **dict[str, dict]**. Each reading column paired with its reading, passed from config, for example **config.FORM_READINGS**. A reading gives its **notations**, the **multipliers** and **divisors** of each, its **step** and its **dtype**.
PURPOSE
Read every measurement in its standard's unit, whichever unit a channel wrote it in.
REQUIREMENTS
- A value in one of the reading's notations is held in the reading's unit, converted by that notation's multiplier or divisor as an exact decimal and rounded half up once to the reading's **step**.
- Each named column holds the reading's **dtype**, and a missing value stays missing.
- A value in none of its reading's notations is refused, naming the column and the values.

RETURNS
pandas.DataFrame. The table with every reading in its unit.

FUNCTION SIGNATURE
`treat_values_below_minimum_as_missing(dataframe, minimum_values)`

PARAMETERS
dataframe — **pandas.DataFrame**. The table a cleaning run has reached, its readings and counts already numbers, passed by that cleaning function.
minimum_values — **dict[str, int | float]**. Each column paired with the smallest value that can be real, passed from config, for example **config.FORM_MINIMUM_VALUES**.
PURPOSE
Blank a reading that cannot be real, never correcting it.
REQUIREMENTS
- A value below its column's minimum becomes missing; a value at the minimum or above is kept.

RETURNS
pandas.DataFrame. The table with impossible low values made missing.

FUNCTION SIGNATURE
`treat_values_above_maximum_as_missing(dataframe, maximum_values)`

PARAMETERS
dataframe — **pandas.DataFrame**. The table a cleaning run has reached, passed by that cleaning function.
maximum_values — **dict[str, int | float]**. Each column paired with the largest value that can be real, passed from config, for example **config.FORM_MAXIMUM_VALUES**.
PURPOSE
Blank a reading too large to be real.
REQUIREMENTS
- A value above its column's maximum becomes missing; a value at the maximum or below is kept.

RETURNS
pandas.DataFrame. The table with impossible high values made missing.

FUNCTION SIGNATURE
`keep_latest_revision_of_each_report(dataframe)`

PARAMETERS
dataframe — **pandas.DataFrame**. A table of reports whose references are canonical and whose revisions are whole numbers, passed by **clean_report_records**, **clean_history_dataset** or **profile_history_details**.
PURPOSE
Keep each report at its latest revision, the only revision that is the report.
REQUIREMENTS
- Each **report_reference** appears once, on the row with its highest **revision_number**, wherever its rows sat in the table.
- The rows kept stay in the order they had, numbered from nought upwards with no gaps.

- A row with no reference or no revision, and two rows that share both a reference and a revision, are refused, naming the reports and **config.REVISION_NUMBER_COLUMN**.

RETURNS
pandas.DataFrame. One row per report.

FUNCTION SIGNATURE
`convert_markup_to_plain_text(texts)`

PARAMETERS
texts — **pandas.Series**. A column of free text, a sentence or a document, passed by **clean_sentence_dataset**, **clean_document_records** or **profile_sentence_details**.
PURPOSE
Read free text as the plain words a person reads.
REQUIREMENTS
- Every tag **config.MARKUP_PATTERN** matches is removed, and every character entity is read as the character it stands for.
- Every run of whitespace, no-break spaces included, becomes one space, and no value carries leading or trailing space.
- The text is held as **config.TEXT_DTYPE**, and a missing value stays missing.

RETURNS
pandas.Series. The plain text, on the same index.

FUNCTION SIGNATURE
`lower_text_written_in_capitals(texts)`

PARAMETERS
texts — **pandas.Series**. Plain text as **convert_markup_to_plain_text** returned it, passed by **clean_sentence_dataset** or **profile_sentence_details**.
PURPOSE
Restore a sentence written wholly in capitals the only way it can be restored.
REQUIREMENTS
- A text that holds letters and no lower-case letter is written in lower case.
- Every other text, and every missing value, is unchanged.

RETURNS
pandas.Series. The text, on the same index.

MODULE: data_pipeline/report_cleaning.py
Reads the live queue of open reports as the Report Rules read them.

FUNCTION SIGNATURE
`clean_report_records(frame)`

PARAMETERS
frame — **pandas.DataFrame**. The open reports as **load_raw_dataset** read them from **config.OPEN_DEFECT_REPORTS_FILE**, passed by **load_open_reports**.
PURPOSE
Turn the live queue into one canonical row per open report.
REQUIREMENTS
- No row repeats another exactly, every category is canonical under **config.FORM_VALUE_MAPPINGS**, and every report reference is canonical.
- The whole numbers of **config.LIVE_WHOLE_NUMBER_PATTERNS** and the readings of **config.FORM_READINGS** are held in their own units; a reading outside **config.FORM_MINIMUM_VALUES** or **config.FORM_MAXIMUM_VALUES** is missing.
- Each report appears once, at its latest revision.

RETURNS
pandas.DataFrame. The open reports, one row each.

MODULE: data_pipeline/history_cleaning.py
The whole cleaning run for the report history.

FUNCTION SIGNATURE
`read_history_answers(frame)`

PARAMETERS
frame — **pandas.DataFrame**. Report history rows as **load_raw_dataset** read them, with exact duplicates removed, passed by **clean_history_dataset** or by **profile_history_details**.
PURPOSE
Read every answer of the history in its canonical form, without removing anything.
REQUIREMENTS
- Every category is canonical under **config.HISTORY_VALUE_MAPPINGS** and every report reference is canonical.
- The whole numbers of **config.HISTORY_WHOLE_NUMBER_PATTERNS** and the readings of **config.FORM_READINGS** are held in their own units.
- Every row is kept.

RETURNS
pandas.DataFrame. The rows with their answers read.

FUNCTION SIGNATURE
`clean_history_dataset(frame)`

PARAMETERS
frame — **pandas.DataFrame**. A raw report history extract as **load_raw_dataset** read it, passed by **run_data_pipeline.py** through **DATASET_JOBS**.
PURPOSE
Produce the history the repeat model learns from.
REQUIREMENTS

- No row repeats another exactly, and every answer is read as **read_history_answers** reads it.
- A reading outside **config.FORM_MINIMUM_VALUES** or **config.FORM_MAXIMUM_VALUES** is missing.
- Each report appears once, at its latest revision; a report whose latest revision gives no **reported_again** is left out, and an earlier revision never stands in for it.

RETURNS
pandas.DataFrame. The cleaned history.

MODULE: data_pipeline/repair_log_cleaning.py
The whole cleaning run for the repair logs.

FUNCTION SIGNATURE
`read_log_answers(frame)`

PARAMETERS
frame — **pandas.DataFrame**. Repair log rows as **load_raw_dataset** read them, with exact duplicates removed, passed by **clean_repair_log_dataset**, which has also refused a blank report reference by then, or by **profile_log_details**.
PURPOSE
Read every answer of the repair logs in its canonical form, without removing anything.
REQUIREMENTS

- Every category is canonical under **config.LOG_VALUE_MAPPINGS** and every report reference is canonical.
- The whole numbers of **config.LOG_WHOLE_NUMBER_PATTERNS** and the readings of **config.LOG_READINGS** are held in their own units.
- Every row is kept.

RETURNS
pandas.DataFrame. The rows with their answers read.

FUNCTION SIGNATURE
`clean_repair_log_dataset(frame)`

PARAMETERS
frame — **pandas.DataFrame**. A raw repair log extract as **load_raw_dataset** read it, passed by **run_data_pipeline.py** through **DATASET_JOBS**.
PURPOSE
Produce the repair logs the crew hours model learns from.
REQUIREMENTS

- No row repeats another exactly; a row with no report reference is refused, and every answer is read as **read_log_answers** reads it.
- No report appears on more than one row; one that does is refused as **refuse_repeated_references** refuses it.
- A value outside **config.LOG_MINIMUM_VALUES** or **config.LOG_MAXIMUM_VALUES** is missing, and a row left with no **actual_crew_hours** is removed.

RETURNS
pandas.DataFrame. The cleaned repair logs.

MODULE: data_pipeline/sentence_cleaning.py
The whole cleaning run for the labelled statement sentences.

FUNCTION SIGNATURE
`clean_sentence_dataset(frame)`

PARAMETERS
frame — **pandas.DataFrame**. The raw statement sentences as **load_raw_dataset** read them, passed by **run_data_pipeline.py** through **DATASET_JOBS**.
PURPOSE
Produce the sentences the sentence model learns from.
REQUIREMENTS

- No row repeats another exactly; a missing **sentence_id** or **report_reference**, an identifier outside **config.SENTENCE_IDENTIFIER_PATTERNS** and a repeated **sentence_id** are refused.

- Every category is canonical under **config.SENTENCE_VALUE_MAPPINGS**, every report reference is canonical, and **sentence_number** is a whole number as **config.SENTENCE_WHOLE_NUMBER_PATTERNS** reads it.
- **sentence_text** is plain text, a sentence written wholly in capitals is in lower case, and **sentence_length_characters** is the length of the cleaned text.
- A row with no **sentence_type**, and a sentence of fewer than **config.MINIMUM_SENTENCE_WORDS** words, are left out; the rows kept are numbered from nought with no gaps.

RETURNS
pandas.DataFrame. The cleaned sentences.

MODULE: data_pipeline/document_cleaning.py
Reads the documents attached to the live reports.

FUNCTION SIGNATURE
`clean_document_records(frame)`

PARAMETERS
frame — **pandas.DataFrame**. The report documents as **load_raw_dataset** read them from **config.REPORT_DOCUMENTS_FILE**, passed by **load_report_documents**.
PURPOSE
Turn the documents into one canonical row each, their text as plain words.
REQUIREMENTS

- No row repeats another exactly; a missing **document_id** or **report_reference**, an identifier outside **config.DOCUMENT_IDENTIFIER_PATTERNS** and a repeated **document_id** are refused.
- Every category is canonical under **config.DOCUMENT_VALUE_MAPPINGS**, and every report reference is canonical.
- **document_text** is plain text.

RETURNS
pandas.DataFrame. The documents, one row each.

MODULE: data_pipeline/register_cleaning.py
Reads the street works register.

FUNCTION SIGNATURE
`clean_register_records(frame)`

PARAMETERS
frame — **pandas.DataFrame**. The register as **load_raw_dataset** read it from **config.STREET_WORKS_REGISTER_FILE**, passed by **load_street_works_register**.
PURPOSE
Turn the register into one canonical row per permit.
REQUIREMENTS

- No row repeats another exactly; a blank in any of **config.REGISTER_REQUIRED_COLUMNS**, a permit reference outside **config.REGISTER_IDENTIFIER_PATTERNS** and a repeated permit reference are refused.
- Every category is canonical under **config.REGISTER_VALUE_MAPPINGS**.

RETURNS
pandas.DataFrame. The permits, one row each.

MODULE: data_pipeline/feature_builder.py
The columns the models read beyond the report's own answers, the table of a model's inputs, and the preparation saved with each tabular model.

FUNCTION SIGNATURE
`add_requirement_counts(dataframe, standards)`

PARAMETERS
dataframe — **pandas.DataFrame**. A cleaned report history table, passed by **run_model_training.py**.
standards — **dict[str, dict]**. Each defect type paired with its standard as **read_defect_standard** returned it, built by **run_model_training.py**.
PURPOSE
Give each past report the requirement counts its screening gives.
REQUIREMENTS

- Each row gains the three columns of **config.REQUIREMENT_COUNT_RESULTS**, **config.REQUIREMENTS_MET_COUNT_COLUMN**, **config.REQUIREMENTS_NOT_MET_COUNT_COLUMN** and **config.REQUIREMENTS_NOT_DOCUMENTED_COUNT_COLUMN**, each the number of that row's requirements with that result as **screen_report** gives them under **standards**.
- Every other column and every row is unchanged.

RETURNS
pandas.DataFrame. The table with the three counts.

FUNCTION SIGNATURE
`add_coordination_trigger_flags(dataframe)`

PARAMETERS
dataframe — **pandas.DataFrame**. A cleaned report history or repair log table, passed by **run_model_training.py**.
PURPOSE
Give each row a flag for each coordination trigger.
REQUIREMENTS
- Each row gains the three columns of **config.COORDINATION_TRIGGER_FLAG_COLUMNS**, each **1** when **find_coordination_triggers** finds that trigger for the row and **0** otherwise.
- Every other column and every row is unchanged.

RETURNS
pandas.DataFrame. The table with the three flags.

FUNCTION SIGNATURE
`build_feature_frame(dataframe, numeric_columns, categorical_columns)`

PARAMETERS
dataframe — **pandas.DataFrame**. A table holding every column named, passed by **run_model_training.py**, **estimate_repeat_likelihood** or **estimate_expected_crew_hours**.
numeric_columns — **list[str]**. The number inputs, passed from config, for example **config.REPEAT_NUMERIC_FEATURES**.
categorical_columns — **list[str]**. The category inputs, passed from config, for example **config.REPEAT_CATEGORICAL_FEATURES**.
PURPOSE
Lay out a model's inputs in the one order the model is fitted and scored on, whatever order the table holds them in.
REQUIREMENTS
- The columns are **numeric_columns** then **categorical_columns**, in the order given, and the rows and their index are the table's own.
- A number goes in as a float as cleaned, a gap as NaN; a category goes in as cleaned, a gap as NaN, never **None**, the text **nan** or an empty string.

RETURNS
pandas.DataFrame. The model's inputs.

FUNCTION SIGNATURE
`build_model_pipeline(estimator, numeric_columns, categorical_columns)`

PARAMETERS
estimator — **sklearn.base.BaseEstimator**. A scikit-learn estimator, unfitted, passed by **train_repeat_model** or **train_crew_hours_model**.
numeric_columns — **list[str]**. The number inputs, passed by that training function from config.
categorical_columns — **list[str]**. The category inputs, passed by that training function from config.
PURPOSE
Bundle the preparation with the estimator, so the saved model scores raw inputs with gaps exactly as it was fitted.
REQUIREMENTS
- The pipeline's first step is named **config.PREPROCESSING_STEP_NAME** and its last **config.MODEL_STEP_NAME**, the estimator.
- Numbers are filled with the training data's **config.NUMBER_FILL_STRATEGY** and scaled; categories are filled with the training data's **config.CATEGORY_FILL_STRATEGY** value and one-hot encoded.

RETURNS
sklearn.pipeline.Pipeline. The unfitted pipeline.

MODULE: data_pipeline/quality_profiler.py
Measures how dirty each raw extract was and what cleaning did, so a reviewer can see that every repair was needed. Counts of the extract as received, apart from **rows_received** and **duplicate_rows**, are taken with each exact repeat counted once; counts after cleaning are taken on the cleaned table. Across one extract's profile and its details, the reasons for removal, **duplicate_rows**, **rows_missing_target**, **answered_earlier_revisions_removed**, **rows_with_impossible_target** and **rows_with_too_few_words**, add up to **rows_removed**.

FUNCTION SIGNATURE
`profile_dataset(raw_frame, clean_frame, target_column)`

PARAMETERS
raw_frame — **pandas.DataFrame**. The extract as **load_raw_dataset** read it, passed by **run_data_pipeline.py**.
clean_frame — **pandas.DataFrame**. The same extract once cleaned, passed by **run_data_pipeline.py**.
target_column — **str**. The extract's answer column, passed by **run_data_pipeline.py** through **DATASET_JOBS**.
PURPOSE
Describe one extract before and after cleaning.
REQUIREMENTS
- The profile holds **rows_received** (every row of **raw_frame**), **rows_kept** (every row of **clean_frame**), **rows_removed** (the difference) and **duplicate_rows** (the rows repeating an earlier row exactly).

- It also holds **rows_missing_target**, **columns** (the cleaned table's column count), **missing_cells_as_received** and **missing_cells_after_cleaning**.
- The two missing-cell entries name only the columns with at least one gap, each with its count.

RETURNS
dict. The profile.

FUNCTION SIGNATURE
`profile_history_details(raw_frame, clean_frame)`

PARAMETERS
raw_frame — **pandas.DataFrame**. A raw report history extract, passed by **run_data_pipeline.py** through **DATASET_JOBS**.
clean_frame — **pandas.DataFrame**. The same extract once cleaned, passed by **run_data_pipeline.py**.
PURPOSE
Give the evidence peculiar to the report history.
REQUIREMENTS

- The details hold **answered_earlier_revisions_removed**, **report_references_not_written_canonically**, **readings_by_unit_as_written**, **values_below_minimum_as_received**, **values_above_maximum_as_received**, **category_values_not_written_canonically** and **target_shares**.
- **report_references_not_written_canonically** counts the references **config.CANONICAL_REPORT_REFERENCE_PATTERN** does not match; **readings_by_unit_as_written** counts each reading of **config.FORM_READINGS** by the notation it was written in; **target_shares** gives each answer's share of the cleaned rows to **config.TARGET_SHARE_DECIMALS** places.

RETURNS
dict. The details.

FUNCTION SIGNATURE
`profile_log_details(raw_frame, clean_frame)`

PARAMETERS
raw_frame — **pandas.DataFrame**. A raw repair log extract, passed by **run_data_pipeline.py** through **DATASET_JOBS**.
clean_frame — **pandas.DataFrame**. The same extract once cleaned, passed by **run_data_pipeline.py**.
PURPOSE
Give the evidence peculiar to the repair logs.
REQUIREMENTS

- The details hold **rows_with_impossible_target**, **report_references_not_written_canonically**, **readings_by_unit_as_written**, **values_below_minimum_as_received**, **values_above_maximum_as_received**, **category_values_not_written_canonically** and **median_actual_crew_hours**.

RETURNS
dict. The details.

FUNCTION SIGNATURE
`profile_sentence_details(raw_frame, clean_frame)`

PARAMETERS
raw_frame — **pandas.DataFrame**. The raw statement sentences, passed by **run_data_pipeline.py** through **DATASET_JOBS**.
clean_frame — **pandas.DataFrame**. The same extract once cleaned, passed by **run_data_pipeline.py**.
PURPOSE
Give the evidence peculiar to free-text sentences.
REQUIREMENTS

- The details hold **sentences_with_markup**, **sentences_with_encoding_artifacts**, **sentences_in_capitals**, **sentence_lengths_disagreeing_with_plain_text_as_received**, **rows_with_too_few_words**, **report_references_not_written_canonically**, **shortest_sentence_words**, **category_values_not_written_canonically** and **target_shares**.

RETURNS
dict. The details.

FUNCTION SIGNATURE
`write_quality_report(profiles)`

PARAMETERS
profiles — **dict[str, dict]**. One entry per raw extract, named by its file name without its extension, each holding **profile_dataset**'s keys together with its detail keys, built by **run_data_pipeline.py**.
PURPOSE
Leave a record a reviewer can read to see what the pipeline did.
REQUIREMENTS

- The report is written to **config.DATA_QUALITY_REPORT_FILE**, whose folder exists afterwards.
- It holds every profile it was given as JSON indented by **config.JSON_INDENT** with its keys sorted, so writing the same profiles twice produces the same file.

RETURNS
None.

========== MODELLING ==========

Three models are trained, scored on data they never learned from, and saved with everything needed to score one row. Each is trained with the settings the configuration holds for it: the repeat model and the sentence model are logistic regressions allowed **config.LOGISTIC_REGRESSION_MAX_ITERATIONS** iterations, and the crew hours model is a random forest of **config.CREW_HOURS_TREE_COUNT** trees with **config.CREW_HOURS_MIN_SAMPLES_LEAF** and **config.CREW_HOURS_MAX_FEATURES**, every random choice seeded with **config.RANDOM_SEED**. The interface, the scores reported and each model's minimum are fixed.

MODULE: modeling/model_registry.py
Saves and loads the models and their feature schemas, and saves their scores. Nothing else in the project writes to **output/models/**.

FUNCTION SIGNATURE
`save_model(model, file_path)`
PARAMETERS
model — **sklearn.pipeline.Pipeline**. A fitted pipeline, preparation and estimator together, as a training function returned it, passed by **run_model_training.py**.
file_path — **str**. Where to save it, passed by **run_model_training.py** from **config.REPEAT_MODEL_FILE**, **config.CREW_HOURS_MODEL_FILE** or **config.SENTENCE_MODEL_FILE**.
PURPOSE
Put a fitted model on disk, so that the service scores reports without training anything.
REQUIREMENTS
- The folder the file belongs in exists afterwards.
- The whole object is saved, compressed at **config.MODEL_FILE_COMPRESSION**, so that loading it gives something that scores raw inputs.

RETURNS
None.

FUNCTION SIGNATURE
`save_feature_schema(feature_names, file_path)`
PARAMETERS
feature_names — **list[str]**. The columns the model was fitted on, in the order it was fitted on them, taken by **run_model_training.py** from the feature frame it built.
file_path — **str**. Where to save them, passed by **run_model_training.py** from **config.REPEAT_SCHEMA_FILE** or **config.CREW_HOURS_SCHEMA_FILE**.
PURPOSE
Record exactly which inputs a model expects, so that a live report can be laid out to match.
REQUIREMENTS
- The folder the file belongs in exists afterwards.
- The file is JSON indented by **config.JSON_INDENT**, holding the names in their original order.

RETURNS
None.

FUNCTION SIGNATURE
`load_model(file_path)`
PARAMETERS
file_path — **str**. A model file written by **save_model**, passed by **agentic/nodes.py** from **config.REPEAT_MODEL_FILE**, **config.CREW_HOURS_MODEL_FILE** or **config.SENTENCE_MODEL_FILE**.
PURPOSE
Bring a saved model back, ready to score.
REQUIREMENTS
- What is returned is the object that was saved, able to score without any further fitting.

RETURNS
sklearn.pipeline.Pipeline. The fitted pipeline that was saved.

FUNCTION SIGNATURE
`load_feature_schema(file_path)`
PARAMETERS
file_path — **str**. A schema file written by **save_feature_schema**, passed by **agentic/nodes.py** from **config.REPEAT_SCHEMA_FILE** or **config.CREW_HOURS_SCHEMA_FILE**.
PURPOSE
Bring back the ordered list of inputs a model expects.
REQUIREMENTS
- The names come back in the order they were saved.

RETURNS
list[str]. The feature names.

FUNCTION SIGNATURE

`save_metrics(metrics)`

PARAMETERS
metrics — **dict**. The description and scores of all three models under **repeat**, **crew_hours** and **sentence**, built and passed by **run_model_training.py**.
PURPOSE
Keep the evidence of how good each model was when it was saved.
REQUIREMENTS

- The scores are written to **config.MODEL_METRICS_FILE**, whose folder exists afterwards.
- The file is JSON indented by **config.JSON_INDENT** with its keys sorted.

RETURNS
None.

MODULE: modeling/model_evaluation.py
Scores a fitted model on rows it did not learn from and on rows it did, so that the gap between the two shows how much it memorised. No function here fits anything.

FUNCTION SIGNATURE
`evaluate_classifier(model, train_features, train_target, held_out_features, held_out_target)`

PARAMETERS
model — **sklearn.pipeline.Pipeline**. The fitted repeat pipeline, whose classes are **0** and **1**, passed by **train_repeat_model** once it has fitted it.
train_features — **pandas.DataFrame**. The inputs the model was fitted on, passed on by **train_repeat_model**.
train_target — **pandas.Series**. Their answers as integers, passed on by **train_repeat_model**.
held_out_features — **pandas.DataFrame**. Inputs the model never saw, passed on by **train_repeat_model**.
held_out_target — **pandas.Series**. The held-out reports' answers as integers, encoded the same way, passed on by **train_repeat_model**.
PURPOSE
Say how well the repeat model ranks reports it has never seen, and whether its scores can be read as likelihoods.
REQUIREMENTS

- The model is scored as it was given, never fitted again.
- **train_roc_auc** and **held_out_roc_auc** are measured from the probability of the positive class, and **generalisation_gap** is the training score less the held-out score.
- **brier_score** and **mean_predicted_probability** describe the held-out probabilities, and **positive_share** the held-out answers.
- Every score is a **float** rounded to **config.METRIC_DECIMALS** places.

RETURNS
dict. The scores above.

FUNCTION SIGNATURE
`evaluate_regressor(model, train_features, train_target, held_out_features, held_out_target)`

PARAMETERS
model — **sklearn.pipeline.Pipeline**. The fitted crew hours pipeline, passed by **train_crew_hours_model** once it has fitted it.
train_features — **pandas.DataFrame**. The inputs the model was fitted on, passed on by **train_crew_hours_model**.
train_target — **pandas.Series**. Their actual crew hours as floats, passed on by **train_crew_hours_model**.
held_out_features — **pandas.DataFrame**. Inputs the model never saw, passed on by **train_crew_hours_model**.
held_out_target — **pandas.Series**. The held-out repair logs' actual crew hours as floats, passed on by **train_crew_hours_model**.
PURPOSE
Say how well the crew hours model estimates repairs it has never seen, and whether it beats the simplest possible guess.
REQUIREMENTS

- The model is scored as it was given, never fitted again.
- **train_r2** and **held_out_r2** are the R squared on each set of rows, and **generalisation_gap** is the training score less the held-out score.
- **mean_absolute_error** is the model's average held-out miss, and **average_estimate_mean_absolute_error** the average held-out miss of always answering the mean of **train_target**.
- **lowest_prediction** and **highest_prediction** are the model's lowest and highest held-out estimates.
- Every score is a **float** rounded to **config.METRIC_DECIMALS** places.

RETURNS
dict. The scores above.

FUNCTION SIGNATURE
`evaluate_text_classifier(model, train_text, train_target, held_out_text, held_out_target)`

PARAMETERS
model — **sklearn.pipeline.Pipeline**. The fitted sentence pipeline, passed by **train_sentence_model** once it has fitted it.
train_text — **pandas.Series**. The sentences it was fitted on, passed on by **train_sentence_model**.
train_target — **pandas.Series**. Their sentence types, passed on by **train_sentence_model**.
held_out_text — **pandas.Series**. Sentences it never saw, passed on by **train_sentence_model**.
held_out_target — **pandas.Series**. The held-out sentences' types as text, passed on by **train_sentence_model**.
PURPOSE

Say how well the sentence model names every type of sentence, the rare ones included.
REQUIREMENTS
- The model is scored as it was given, never fitted again.
- **train_macro_f1** and **held_out_macro_f1** are the macro-averaged F1 on each set of sentences, and **generalisation_gap** is the training score less the held-out score.
- **accuracy** and **weighted_f1** describe the held-out predictions, and **per_class_f1** gives each class the model holds its own held-out F1.
- Every score, and every value inside **per_class_f1**, is a **float** rounded to **config.METRIC_DECIMALS** places.

RETURNS
dict. The scores above.

MODULE: modeling/repeat_model.py
The model that estimates how likely a defect is to be reported again after its repair.

FUNCTION SIGNATURE
`train_repeat_model(train_features, train_target, held_out_features, held_out_target)`
PARAMETERS
train_features — **pandas.DataFrame**. The cleaned training reports of **config.CLEAN_REPORT_HISTORY_TRAIN_FILE** with their requirement counts and coordination trigger flags, laid out by **build_feature_frame** from **config.REPEAT_NUMERIC_FEATURES** and **config.REPEAT_CATEGORICAL_FEATURES**, passed by **run_model_training.py**.
train_target — **pandas.Series**. Their **reported_again** answers as integers through **config.REPEAT_TARGET_ENCODING**, passed by **run_model_training.py**.
held_out_features — **pandas.DataFrame**. The cleaned held-out reports of **config.CLEAN_REPORT_HISTORY_EVAL_FILE**, laid out the same way, passed by **run_model_training.py**.
held_out_target — **pandas.Series**. Their **reported_again** answers, encoded the same way, passed by **run_model_training.py**.
PURPOSE
Give every open report a probability that its defect is reported again.
REQUIREMENTS
- The model is a pipeline from **build_model_pipeline**, fitted on the training rows only, recording the names of the columns it was fitted on; its classes are **0** and **1** in that order.
- Its held-out ROC AUC reaches **config.MINIMUM_REPEAT_ROC_AUC**, and its mean held-out score is within **0.03** of the share of held-out defects reported again.
- Turning one requirement from Met to Not met raises the score for at least **95%** of the held-out reports with a requirement Met, and the average by at least **0.03**.
- One more conflicting permit live on the street raises the score for at least **95%** of the held-out reports with none, and the average by at least **0.05**.
- That permit adds one to the live and conflicting counts and flags the live street works trigger.
- Across the live queue its likelihoods fall in all three bands.
- Its scores come from **evaluate_classifier**.

RETURNS
tuple[sklearn.pipeline.Pipeline, dict]. The fitted model and its scores.

MODULE: modeling/crew_hours_model.py
The model that estimates how many crew hours a repair will really take.

FUNCTION SIGNATURE
`train_crew_hours_model(train_features, train_target, held_out_features, held_out_target)`
PARAMETERS
train_features — **pandas.DataFrame**. The cleaned training repair logs of **config.CLEAN_REPAIR_LOGS_TRAIN_FILE** with their coordination trigger flags, laid out by **build_feature_frame** from **config.CREW_HOURS_NUMERIC_FEATURES** and **config.CREW_HOURS_CATEGORICAL_FEATURES**, passed by **run_model_training.py**.
train_target — **pandas.Series**. Their **actual_crew_hours** as floats, passed by **run_model_training.py**.
held_out_features — **pandas.DataFrame**. The cleaned held-out repair logs of **config.CLEAN_REPAIR_LOGS_EVAL_FILE**, laid out the same way, passed by **run_model_training.py**.
held_out_target — **pandas.Series**. Their **actual_crew_hours** as floats, passed by **run_model_training.py**.
PURPOSE
Estimate the crew hours a repair needs before the crew is booked.
REQUIREMENTS
- The model is a pipeline from **build_model_pipeline**, fitted on the training rows only, recording the names of the columns it was fitted on.
- Its held-out R squared reaches **config.MINIMUM_CREW_HOURS_R2**, and its average miss is below **0.75** times that of always answering the training average.
- No held-out forecast is below the fewest actual crew hours of the training logs.
- A traffic-sensitive street in place of a local road lengthens the forecast for at least **90%** of the held-out repairs on local roads, and by at least **0.5** hours on average.

- One more conflicting permit live on the street lengthens the forecast for at least **90%** of the held-out repairs with none, and by at least **0.25** hours on average.
- That permit adds one to the live and conflicting counts; in both changes the trigger flags are worked out again for the changed repair.
- Across the live queue an overrun is expected for some reports and not for others.
- Its scores come from **evaluate_regressor**.

RETURNS
tuple[sklearn.pipeline.Pipeline, dict]. The fitted model and its scores.

MODULE: modeling/sentence_model.py
The NLP model that names the type of each sentence of a report's documents.

FUNCTION SIGNATURE
`train_sentence_model(train_text, train_target, held_out_text, held_out_target)`

PARAMETERS
train_text — **pandas.Series**. The training share of the cleaned sentences' **sentence_text**, from the split **run_model_training.py** makes with **config.SENTENCE_HELD_OUT_SHARE**, stratified by type and seeded with **config.RANDOM_SEED**.
train_target — **pandas.Series**. Their **sentence_type**, from the same split.
held_out_text — **pandas.Series**. The held-out share of **sentence_text**, from the same split.
held_out_target — **pandas.Series**. Their **sentence_type**, from the same split.
PURPOSE
Name the type of a sentence written by a resident or noted by the contact centre.
REQUIREMENTS

- The model is one pipeline fitted on the training sentences only, taking raw sentence text.
- Its first step, **config.PREPROCESSING_STEP_NAME**, weighs the words by TF-IDF; its last, **config.MODEL_STEP_NAME**, is the logistic regression.
- Its classes are the nine distinct values of **config.SENTENCE_TYPE_MAPPING**, no more and no fewer, and any text, words it never saw included, is read as one of them.
- Its held-out macro F1 reaches **config.MINIMUM_SENTENCE_MACRO_F1**; every type is predicted for some held-out sentence, and at least **75%** of the held-out sentences of each type are read as that type.
- Its scores come from **evaluate_text_classifier**.

RETURNS
tuple[sklearn.pipeline.Pipeline, dict]. The fitted model and its scores.

========== THE KNOWLEDGE BASE ==========
The sixteen documents are the only source of every requirement a standard sets, of every check a statement guide asks for, and of every answer the system draws from documents. They are read once, divided into their numbered sections, cut into passages and stored so that they can be searched by meaning, and so that any section can be read back whole. A defect type's requirements are read from its own standard alone.

MODULE: knowledge/pdf_extraction.py
Gets the text out of the PDFs and does nothing else with it.

FUNCTION SIGNATURE
`extract_text_from_directory(docs_directory)`

PARAMETERS
docs_directory — **str**. The folder holding the documents, passed from **config.DOCS_DIRECTORY** by **run_knowledge_indexing.py**, **run_model_training.py** and **agentic/nodes.py**.
PURPOSE
Read every document in a folder once.
REQUIREMENTS

- Every file whose name ends in **config.PDF_FILE_SUFFIX**, whatever its case, is read, and nothing else in the folder is.
- Each entry carries **document_name**, the file name as it is on disk, and **document_text**, every page's text in page order, one line break between pages; the entries come back in name order.
- A PDF with no pages or with a page that gives up no text is refused, naming the file; a folder with no PDF is refused, naming the folder.

RETURNS
list[dict]. One entry per document.

MODULE: knowledge/chunk_splitting.py
Cuts the documents into passages small enough for a search to return alone and whole enough that a requirement is never cut in half.

FUNCTION SIGNATURE

`split_documents_into_sections(document_entries)`

PARAMETERS
document_entries — **list[dict]**. The documents as **extract_text_from_directory** returned them, passed by **run_knowledge_indexing.py**, or by **read_defect_standard** as the single entry of its standard.
PURPOSE
Divide each document at its numbered headings, so that a passage never spans two subjects.
REQUIREMENTS

- A document carries exactly one header line recognised by **config.DOCUMENT_HEADER_PATTERN**, and sections headed as **config.SECTION_HEADING_PATTERN** recognises, numbered from **1** in order; any other layout is refused, naming the document.
- Each entry carries **document_name**, **section_number** and **section_text**, every run of **config.WHITESPACE_RUN_PATTERN** a single space and no space at either end.
- A section opens with its heading and runs to the next heading; no section carries the title or the header line.

RETURNS
list[dict]. One entry per section, in document order.

FUNCTION SIGNATURE
`build_chunk_records(section_entries)`

PARAMETERS
section_entries — **list[dict]**. The sections as **split_documents_into_sections** returned them, passed by **run_knowledge_indexing.py**.
PURPOSE
Cut each section into passages a search can return, overlapping enough that a sentence near a boundary is whole in one of them.
REQUIREMENTS

- A passage is whole sentences of one section, divided by **config.SENTENCE_BOUNDARY_PATTERN** and joined by single spaces, as many as fit within **config.CHUNK_SIZE_CHARACTERS**; it is longer only when one sentence alone is.
- Every section gives at least one passage, and no passage spans two sections.
- A passage after the first of its section opens with whole sentences repeated from the end of the one before, at most **config.CHUNK_OVERLAP_CHARACTERS** characters of them, whenever such a sentence fits.
- Each record carries **chunk_id**, **chunk_text**, **document_name**, **section_number**, **passage_number** and **carried_characters**.
- **chunk_id** is made from the document name and the passage number, so the same documents always give the same identifiers.
- **passage_number** counts the document's passages from **0**; **carried_characters** counts the characters at the start, joining space included, repeated from the passage before, **0** when none are.

RETURNS
list[dict]. One record per passage, in document and section order.

MODULE: knowledge/vector_index.py
Turns passages into vectors, stores them, reads a section back and searches them. This is the only module that talks to the vector store.

FUNCTION SIGNATURE
`build_vector_index(chunk_records, store_path)`

PARAMETERS
chunk_records — **list[dict]**. The passages as **build_chunk_records** returned them, passed by **run_knowledge_indexing.py**.
store_path — **str**. The folder the store is kept in, passed from **config.VECTOR_STORE_DIRECTORY** by **run_knowledge_indexing.py**.
PURPOSE
Build the searchable store the whole system reads from.
REQUIREMENTS

- A file given as the folder, and a folder that holds files but no **config.VECTOR_STORE_FILE_NAME**, are refused, naming the path.
- The store is persistent and holds the collection **config.VECTOR_COLLECTION_NAME**, embedded with **config.EMBEDDING_MODEL_NAME** on **config.EMBEDDING_DEVICE** and compared by **config.VECTOR_DISTANCE_METRIC** distance; every search weighs every passage, so the nearest passages are found exactly.
- Every passage is stored once under its **chunk_id**, with its text and with **document_name**, **section_number**, **passage_number** and **carried_characters** as metadata.
- Building again replaces the store rather than adding to it; if the embedding model cannot be loaded, the existing store is left as it was.

RETURNS
int. How many passages the collection holds.

FUNCTION SIGNATURE
`open_vector_index(store_path)`

PARAMETERS

store_path — **str**. The folder **build_vector_index** wrote to, passed from **config.VECTOR_STORE_DIRECTORY** by **run_knowledge_indexing.py** and **agentic/nodes.py**.
PURPOSE
Open the store for reading and searching, without changing it.
REQUIREMENTS

- A folder with no **config.VECTOR_STORE_FILE_NAME**, and a store with no **config.VECTOR_COLLECTION_NAME** collection, are refused, naming the path.
- The collection comes back with the embedding it was built with; nothing is rebuilt or written.

RETURNS
chromadb.Collection. The knowledge base, ready to read and search.

FUNCTION SIGNATURE
`read_document_section(collection, document_name, section_number)`
PARAMETERS
collection — **chromadb.Collection**. The knowledge base as **open_vector_index** returned it, passed by a function of **tools/retrieval_tools.py**.
document_name — **str**. The one document to read, passed from config, for example a value of **config.DEFECT_STANDARD_DOCUMENTS** or **config.HIGHWAYS_POLICY_DOCUMENT**.
section_number — **int**. The section wanted, passed from config, for example **config.WORKS_ORDER_SECTION_NUMBER**.
PURPOSE
Read one section of one document back, whole.
REQUIREMENTS

- Only that document's passages of that section are read, and the section comes back exactly as **split_documents_into_sections** gave it, heading included, every sentence once.
- A section the store holds no passage of is refused with **config.NO_SECTION_PASSAGES_MESSAGE**.

RETURNS
str. The section's text.

FUNCTION SIGNATURE
`search_documents(collection, query_texts)`
PARAMETERS
collection — **chromadb.Collection**. The knowledge base, passed by **answer_from_documents**.
query_texts — **list[str]**. One or more questions asked together, passed by **answer_from_documents**.
PURPOSE
Ask the store several questions at once for the chat, and get back the nearest passages with how far each one is.
REQUIREMENTS

- One list comes back for each question, in the order asked, each of **config.CHAT_RETRIEVAL_RESULT_COUNT** passages, nearest first.
- No passage of **config.CHAT_EXCLUDED_DOCUMENTS** is ever returned.
- Each passage carries **chunk_id**, **chunk_text**, **document_name** and **distance**, a plain **float**.

RETURNS
list[list[dict]]. The passages found for each question.

MODULE: knowledge/standard_reading.py
Reads a defect type's requirements out of that defect type's own defect standard. This is the module that keeps the requirements out of the code.

FUNCTION SIGNATURE
`read_defect_standard(defect_type, document_entries)`
PARAMETERS
defect_type — **str**. The defect type whose standard is wanted, one of the keys of **config.DEFECT_STANDARD_DOCUMENTS**, passed by **run_model_training.py**, **agentic/nodes.py** or **answer_from_documents**.
document_entries — **list[dict]**. The documents as **extract_text_from_directory** returned them, passed by the same callers.
PURPOSE
Establish every requirement of one defect type, with its own figures, from its own standard.
REQUIREMENTS

- A defect type with no configured standard is refused with **config.NO_STANDARD_CONFIGURED_MESSAGE**.
- The standard is read from its configured document wherever it stands among **document_entries**, and from no other; entries that do not hold that document exactly once are refused, naming it.
- The sections are those **split_documents_into_sections** gives, so a standard reads the same however its lines fall.
- Section **config.DEFECT_TYPE_SECTION_NUMBER** states the defect type itself and no other, as **config.DEFECT_TYPE_PATTERN** finds it; otherwise the document is refused, naming it.
- Section **config.REQUIREMENTS_SECTION_NUMBER** matches **config.REQUIREMENTS_SECTION_PATTERN** whole, its requirements divided by **config.REQUIREMENT_NUMBER_PATTERN** and numbered from **1** in order with no text before the first; otherwise the document is refused, naming it.
- Each requirement fits exactly one form of **config.REQUIREMENT_SENTENCE_PATTERNS**; one that fits none, and a kind stated twice, are refused, naming the document.

- Each requirement carries **requirement_kind**, the key of the form it fits, and **requirement_name** and **requirement_sentence**, the parts **config.REQUIREMENT_NAME_PART** and **config.REQUIREMENT_SENTENCE_PART**, word for word.
- Its **rules** are keyed by the form's figure parts, **config.MINIMUM_RULE**, **config.MAXIMUM_RULE**, **config.FOOTWAY_LIMIT_RULE** or **config.CARRIAGEWAY_LIMIT_RULE**, each figure an exact **decimal.Decimal**, and are empty for a requirement with no figure.
- A standard reads the same every time.

RETURNS
dict. **defect_type**, **document_name** and **requirements**, a list in the standard's own order.

============== AGENT TOOLS ==============
The seven modules below are the boundary between what a standard, the register, a model or a document says and what the review records. Everything a language model is later told is assembled here, as plain facts, by ordinary code that can be read and reproduced. The same functions serve the workflow, the service and the chat, so what a page shows before any agent has run is what the agents are given. No agent works out any of these results or figures for itself.

MODULE: tools/report_data_tools.py
Reads the live queue, its documents and the street works register through the same cleaning as the extracts, reads the report history the chat counts from, and serves one report at a time.

FUNCTION SIGNATURE
`load_open_reports()`
PARAMETERS
None.
PURPOSE
Read the live queue once, as the Report Rules read it.
REQUIREMENTS
- The table is **config.OPEN_DEFECT_REPORTS_FILE** as **load_raw_dataset** reads it and **clean_report_records** cleans it.
- Each open report appears once, at its latest revision.

RETURNS
pandas.DataFrame. The open reports.

FUNCTION SIGNATURE
`load_report_documents()`
PARAMETERS
None.
PURPOSE
Read the documents attached to the open reports once.
REQUIREMENTS
- The table is **config.REPORT_DOCUMENTS_FILE** as **load_raw_dataset** reads it and **clean_document_records** cleans it.

RETURNS
pandas.DataFrame. The documents.

FUNCTION SIGNATURE
`load_street_works_register()`
PARAMETERS
None.
PURPOSE
Read the street works register once.
REQUIREMENTS
- The table is **config.STREET_WORKS_REGISTER_FILE** as **load_raw_dataset** reads it and **clean_register_records** cleans it.

RETURNS
pandas.DataFrame. The register, one row per permit.

FUNCTION SIGNATURE
`load_report_history()`
PARAMETERS
None.
PURPOSE
Give the chat the record of every report already decided.
REQUIREMENTS
- The table is **config.CLEAN_REPORT_HISTORY_TRAIN_FILE** followed by **config.CLEAN_REPORT_HISTORY_EVAL_FILE**, as **load_processed_dataset** reads them, numbered from nought.
- The live queue is never part of it.

RETURNS
pandas.DataFrame. The reports already decided.

FUNCTION SIGNATURE

`fetch_report(reports, report_reference)`

PARAMETERS
reports — **pandas.DataFrame**. The open reports as **load_open_reports** returned them, or any cleaned table of reports, held by the caller, for example **open_reports** of **agentic/nodes.py**.
report_reference — **str**. The canonical reference of the report wanted, passed by **gather_report_context** or **answer_report_record**.
PURPOSE
Give one report's own answers to whatever is about to reason about it.
REQUIREMENTS
- The record holds every column the table carries for that report, a missing value as **None**.
- A reference the table does not hold is refused with **config.UNKNOWN_REPORT_MESSAGE**.

RETURNS
dict. The report, column by column.

FUNCTION SIGNATURE
`fetch_report_documents(documents, report)`

PARAMETERS
documents — **pandas.DataFrame**. The documents as **load_report_documents** returned them, **open_report_documents** of **agentic/nodes.py**.
report — **dict**. The report as **fetch_report** returned it, passed by **gather_report_context**.
PURPOSE
Give the documents attached to one report, in a fixed order.
REQUIREMENTS
- Only that report's documents come back, all of them, in **document_id** order whatever order they stand in the file.
- Each document holds every column of the file, a missing value as **None**.

RETURNS
list[dict]. One entry per document.

FUNCTION SIGNATURE
`rank_report_queue(reports)`

PARAMETERS
reports — **pandas.DataFrame**. The open reports, or any table of reports with canonical references, passed by **api/routes.py**.
PURPOSE
Put the queue in the order the Highways Policy sets.
REQUIREMENTS
- Reports whose defect type or linked defect type is **config.QUEUE_FIRST_DEFECT_TYPE** come before every other report; each group is in report-reference order.
- Every report appears once.

RETURNS
list[str]. The report references in queue order.

FUNCTION SIGNATURE
`describe_record(record)`

PARAMETERS
record — **dict**. A report, a document or a set of document details, passed by an agent of **agentic/agents.py**, by **api/routes.py** or by **answer_report_record**.
PURPOSE
Give a record in the words a page and a prompt show.
REQUIREMENTS
- Every value comes back as text: a missing value as **config.NOT_GIVEN_TEXT**, and anything else as its plain text.
- A whole number, held as an integer or as a float with no fraction, has its thousands separated by commas.

RETURNS
dict. The record, key by key, as text.

MODULE: tools/screening_tools.py
Counts the street works beside a report, finds its coordination triggers, and screens it against its standards. Nothing here calls a model.

FUNCTION SIGNATURE
`count_live_street_works(register, report)`

PARAMETERS
register — **pandas.DataFrame**. The register as **load_street_works_register** returned it, **street_works_register** of **agentic/nodes.py**.
report — **dict**. The report as **fetch_report** returned it, passed by **gather_report_context**.
PURPOSE
Count the permits live on a report's street, and name those that conflict with its repair.
REQUIREMENTS
- A report that gives no street is refused with **config.BLANK_STREET_MESSAGE**.

- The live permits are every permit on the report's street whose status is **config.PERMIT_STATUS_LIVE**, in permit-reference order; a cancelled permit and a permit on another street are never counted.
- A live permit conflicts when its works are in the part of the highway **config.DEFECT_TYPE_HIGHWAY_PARTS** gives the report's defect type or its linked defect type.

RETURNS
dict. The two counts under **config.LIVE_STREET_WORKS_COLUMN** and **config.CONFLICTING_STREET_WORKS_COLUMN**, and **live_permits** and **conflicting_permits**, each a list of permit references.

FUNCTION SIGNATURE
`find_coordination_triggers(report)`

PARAMETERS
report — **dict**. One cleaned report or repair log holding its count of conflicting permits, passed by **screen_report** or by **add_coordination_trigger_flags**. A blank answer arrives as **None** from **fetch_report** and as NaN from a row of a cleaned table, and either is blank.
PURPOSE
Say which coordination triggers of the Highways Policy a report meets.
REQUIREMENTS
- The triggers follow **THE STREET WORKS REGISTER AND THE COORDINATION TRIGGERS** above, the column height compared with **config.HIGH_COLUMN_TALLER_THAN_METRES** as an exact decimal.
- A blank count of conflicting permits is refused with **config.BLANK_STREET_WORKS_COUNT_MESSAGE**.
- The triggers that apply come in the order **config.LIVE_STREET_WORKS_TRIGGER**, **config.TRAFFIC_SENSITIVE_TRIGGER**, **config.HIGH_COLUMN_TRIGGER**; the list is empty when none applies.

RETURNS
list[str]. The names of the triggers that apply.

FUNCTION SIGNATURE
`screen_report(report, standards)`

PARAMETERS
report — **dict**. One cleaned report holding its counts of live and conflicting permits, from the live queue or the history, passed by **gather_report_context** or by **add_requirement_counts**. A blank answer arrives as **None** or as NaN, and either is blank.
standards — **dict[str, dict]**. Each defect type with its standard as **read_defect_standard** returns it, or any standard of the same shape, built by the caller: the report's own and linked standards from **gather_report_context**, or all five from **add_requirement_counts**; it must hold the report's defect type and any linked defect type.
PURPOSE
Screen a report against every requirement of its own standard and its linked type's, and against the coordination triggers.
REQUIREMENTS
- The defect types screened are the report's defect type and its linked defect type, unless that is **config.NO_LINKED_DEFECT**.
- A defect type with no standard handed over is refused with **config.NO_STANDARD_READ_MESSAGE**; a standard whose **defect_type** is not the one it was handed over under is refused with **config.STANDARD_OF_ANOTHER_DEFECT_TYPE_MESSAGE**.
- Every requirement of each standard appears once, the own standard's first, each in its standard's order, carrying **defect_type**, **standard_document** (the standard's **document_name**), **requirement_kind**, **requirement_name**, **requirement_sentence**, **rules**, **result** and **blank_answers**.
- Every result follows **MET, NOT MET AND NOT DOCUMENTED** above, judged on the answers **config.REQUIREMENT_ANSWER_COLUMNS** names against the figures of the standard handed over.
- Every answer and figure is compared as an exact decimal, an answer held as a number taken as the decimal it is written as, so a reach of **1.4** is exactly **1.4**.
- **blank_answers** lists the columns of **config.REQUIREMENT_ANSWER_COLUMNS** the requirement needs and the report left blank, and is empty unless the result is Not documented.
- A requirement kind the screening does not know is refused, naming the document and the kind.
- **coordination_triggers** is what **find_coordination_triggers** gives, found even when the requirements decide the assessment; **assessment** follows **THE ASSESSMENT AND THE THREE DECISIONS** above, and **decisions_open** is what **config.DECISIONS_OPEN** gives for it.

RETURNS
dict. **report_reference**, **defect_types**, **requirements**, the three counts under the keys of **config.REQUIREMENT_COUNT_RESULTS**, **coordination_triggers**, **assessment** and **decisions_open**.

MODULE: tools/evidence_tools.py
Reads the documents attached to a report sentence by sentence. A document is evidence; it never changes an answer on the report.

FUNCTION SIGNATURE
`read_document_evidence(sentence_model, documents)`

PARAMETERS
sentence_model — **sklearn.pipeline.Pipeline**. The fitted sentence model loaded from **config.SENTENCE_MODEL_FILE**, **sentence_model** of **agentic/nodes.py**, passed by **gather_report_context**.

documents — **list[dict]**. A report's documents as **fetch_report_documents** returned them, passed by **gather_report_context**.
PURPOSE
Read every sentence of a report's documents for what it is about.
REQUIREMENTS

- Each document is split into sentences at **config.STATEMENT_SENTENCE_PATTERN**, whatever the case of the next word, in the order written, the documents in the order given.
- Each sentence is kept whole, so a document's sentences joined by single spaces give its text.
- Each finding carries **document_id**, **document_type**, **sentence** and **sentence_type**, the type the model reads, as text.
- A document whose text is blank is refused with **config.NO_STATEMENT_TEXT_MESSAGE**; with no documents nothing is read.

RETURNS
list[dict]. One finding per sentence.

FUNCTION SIGNATURE
`link_evidence_to_requirements(findings, screening)`

PARAMETERS
findings — **list[dict]**. The findings as **read_document_evidence** returned them, passed by **gather_report_context**.
screening — **dict**. The report's screening as **screen_report** returned it, passed by **gather_report_context**.
PURPOSE
Record which requirements of the report's own standards each sentence bears on.
REQUIREMENTS

- Each finding keeps its own keys and gains **requirements_borne_on**: the screening's requirements, in its order, whose kind **config.SENTENCE_TYPE_REQUIREMENT_KINDS** lists for the sentence type.
- Each entry of **requirements_borne_on** is a dictionary holding only that requirement's **defect_type** and **requirement_name**; a type whose list is empty bears on none.
- A sentence type that is not a key of **config.SENTENCE_TYPE_REQUIREMENT_KINDS** is refused with **config.NO_REQUIREMENT_KINDS_MESSAGE**.

RETURNS
list[dict]. One linked finding per finding, in the same order.

FUNCTION SIGNATURE
`find_disagreements(findings, report)`

PARAMETERS
findings — **list[dict]**. The report's findings, linked or not, passed by **gather_report_context**.
report — **dict**. The report as **fetch_report** returned it, carrying its counts of live and conflicting permits, passed by **gather_report_context**.
PURPOSE
Flag every measurement sentence whose figure differs from the report's, or gives a figure the report left blank.
REQUIREMENTS

- Only findings of **config.MEASUREMENT** are read, for the figures the patterns of **config.STATEMENT_MEASUREMENT_PATTERNS** find, each for its own column.
- Each figure is compared unrounded, as an exact decimal, with the report's reading of that column, taken as the decimal it is written as.
- A sentence with no figure, or whose figures all agree with the report, is not flagged.
- A sentence is flagged **config.DESCRIBES_BLANK_ANSWER** when the report's answer for any figure it states is blank, and otherwise **config.DISAGREES_WITH_FORM** when any figure differs.
- Each flag carries **document_id**, **sentence**, **stated_readings**, **report_readings** and **flag**, the readings as plain numbers.
- **stated_readings** holds each column a figure was found for, with its figures in the order written; **report_readings** holds the same columns, each the report's reading or **None**.

RETURNS
list[dict]. One flag per flagged sentence, in the order of the findings.

MODULE: tools/prediction_tools.py
Turns a model's output into the figures the review shows. The figures inform the Highways Inspector and decide nothing.

FUNCTION SIGNATURE
`estimate_repeat_likelihood(repeat_model, feature_names, report, screening)`

PARAMETERS
repeat_model — **sklearn.pipeline.Pipeline**. The fitted repeat model loaded from **config.REPEAT_MODEL_FILE**, **repeat_model** of **agentic/nodes.py**, passed by **gather_report_context**.
feature_names — **list[str]**. Its saved schema loaded from **config.REPEAT_SCHEMA_FILE**, **repeat_feature_names** of **agentic/nodes.py**, passed by **gather_report_context**.
report — **dict**. The report as **fetch_report** returned it, with its counts of live and conflicting permits, passed by **gather_report_context**.
screening — **dict**. That report's screening as **screen_report** returned it, passed by **gather_report_context**.
PURPOSE
Give the repeat likelihood of one report, and its band.

REQUIREMENTS

- A screening of another report is refused with **config.SCREENING_OF_ANOTHER_REPORT_MESSAGE**, naming both reports.
- The inputs are the report's answers with the screening's three counts and its three trigger flags, laid out by **build_feature_frame**; inputs that are not **feature_names** in order are refused, naming both.
- **repeat_likelihood** is the model's probability rounded half up to **config.LIKELIHOOD_STEP**, as a float, and **likelihood_band** follows **THE REPEAT LIKELIHOOD AND ITS BAND** above, cut on the rounded figure.

RETURNS

dict. **repeat_likelihood** and **likelihood_band**.

FUNCTION SIGNATURE

```
estimate_expected_crew_hours(crew_hours_model, feature_names, report, screening)
```

PARAMETERS

crew_hours_model — **sklearn.pipeline.Pipeline**. The fitted crew hours model loaded from **config.CREW_HOURS_MODEL_FILE**, **crew_hours_model** of **agentic/nodes.py**, passed by **gather_report_context**.
feature_names — **list[str]**. Its saved schema loaded from **config.CREW_HOURS_SCHEMA_FILE**, **crew_hours_feature_names** of **agentic/nodes.py**, passed by **gather_report_context**.
report — **dict**. The report as **fetch_report** returned it, with its counts of live and conflicting permits, passed by **gather_report_context**.
screening — **dict**. That report's screening as **screen_report** returned it, passed by **gather_report_context**.

PURPOSE

Give the crew hours one report's repair is expected to take, and whether an overrun is expected.

REQUIREMENTS

- A screening of another report is refused with **config.SCREENING_OF_ANOTHER_REPORT_MESSAGE**, naming both reports.
- The inputs are the report's answers with the screening's three trigger flags, laid out by **build_feature_frame**; inputs that are not **feature_names** in order are refused, naming both.
- **expected_crew_hours** is the forecast rounded half up to **config.TENTH_STEP**, as a float; **planned_crew_hours** is the report's, as a float, or **None**.
- **overrun_expected** follows **THE EXPECTED CREW HOURS AND THE OVERRUN LINE** above on the rounded figure, compared as exact decimals: **True** or **False**, or **None** when nothing was planned.

RETURNS

dict. **expected_crew_hours**, **planned_crew_hours** and **overrun_expected**.

FUNCTION SIGNATURE

```
describe_expected_crew_hours(crew_hours)
```

PARAMETERS

crew_hours — **dict**. The expected crew hours as **estimate_expected_crew_hours** returned them, passed by an agent or a node.

PURPOSE

Put the expected crew hours into words.

REQUIREMENTS

- The words are **config.EXPECTED_HOURS_WITHOUT_PLAN_TEMPLATE** when nothing was planned, **config.OVERRUN_TEMPLATE** when an overrun is expected and **config.NO_OVERRUN_TEMPLATE** otherwise, every number shown to **config.CREW_HOURS_DECIMALS** decimal place.

RETURNS

str. The expected crew hours in words.

MODULE: tools/retrieval_tools.py

The five section reads the workflow and the chat need. Each returns one numbered section of one document as **read_document_section** reads it, without its heading: the number, a full stop, a space, the title and a space. A section that does not open with that heading is refused with **config.SECTION_TITLE_MISMATCH_MESSAGE**.

FUNCTION SIGNATURE

```
read_reviewer_checks(collection, sentence_type)
```

PARAMETERS

collection — **chromadb.Collection**. The knowledge base, **knowledge_collection** of **agentic/nodes.py**, passed by **analyse_report_statements_node** or **answer_from_documents**.
sentence_type — **str**. A sentence type, one of the keys of **config.SENTENCE_TYPE_GUIDE_DOCUMENTS**: each type found in the report's documents, passed by **analyse_report_statements_node**, or each key in turn, passed by **answer_from_documents**.

PURPOSE

Give what a reviewer checks in a sentence of one type, from that type's statement guide.

REQUIREMENTS

- The section is **config.REVIEWER_CHECKS_SECTION_NUMBER**, titled **config.REVIEWER_CHECKS_SECTION_TITLE**, of the guide **config.SENTENCE_TYPE_GUIDE_DOCUMENTS** names.
- A sentence type with no guide is refused with **config.NO_GUIDE_CONFIGURED_MESSAGE**.

RETURNS

str. The section's text.

FUNCTION SIGNATURE
`read_inspection_records(collection, defect_type)`

PARAMETERS
collection — **chromadb.Collection**. The knowledge base, passed by **draft_order_document_node**, **answer_from_documents** or **run_knowledge_indexing.py**.
defect_type — **str**. A defect type: each type screened for the report, passed by **draft_order_document_node**, or each key of **config.DEFECT_STANDARD_DOCUMENTS** in turn, passed by **answer_from_documents** and **run_knowledge_indexing.py**.
PURPOSE
Give what an inspection of a defect of one type records, from that type's own standard.
REQUIREMENTS
- The section is **config.INSPECTION_RECORDS_SECTION_NUMBER**, titled **config.INSPECTION_RECORDS_SECTION_TITLE**, of the standard **config.DEFECT_STANDARD_DOCUMENTS** names.
- A defect type with no standard is refused with **config.NO_STANDARD_CONFIGURED_MESSAGE**.

RETURNS
str. The section's text.

FUNCTION SIGNATURE
`read_works_order_guidance(collection, defect_type)`

PARAMETERS
collection — **chromadb.Collection**. The knowledge base, passed by **draft_order_document_node**.
defect_type — **str**. Each defect type screened for the report, from the screening's **defect_types**, passed by **draft_order_document_node**.
PURPOSE
Give what a works order for a defect of one type must say, from that type's own standard.
REQUIREMENTS
- The section is **config.WORKS_ORDER_SECTION_NUMBER**, titled **config.WORKS_ORDER_SECTION_TITLE**, of the standard **config.DEFECT_STANDARD_DOCUMENTS** names.
- A defect type with no standard is refused with **config.NO_STANDARD_CONFIGURED_MESSAGE**.

RETURNS
str. The section's text.

FUNCTION SIGNATURE
`read_resident_update_guidance(collection, defect_type)`

PARAMETERS
collection — **chromadb.Collection**. The knowledge base, passed by **draft_resident_update_node**.
defect_type — **str**. Each defect type screened for the report, from the screening's **defect_types**, passed by **draft_resident_update_node**.
PURPOSE
Give what a resident update for a defect of one type must say, from that type's own standard.
REQUIREMENTS
- The section is **config.RESIDENT_UPDATE_SECTION_NUMBER**, titled **config.RESIDENT_UPDATE_SECTION_TITLE**, of the standard **config.DEFECT_STANDARD_DOCUMENTS** names.
- A defect type with no standard is refused with **config.NO_STANDARD_CONFIGURED_MESSAGE**.

RETURNS
str. The section's text.

FUNCTION SIGNATURE
`read_policy_section(collection, section_number, section_title)`

PARAMETERS
collection — **chromadb.Collection**. The knowledge base, passed by a node of **agentic/nodes.py**.
section_number — **int**. The section wanted, passed from config, for example **config.POLICY_TRIGGERS_SECTION_NUMBER**.
section_title — **str**. The title it must carry, passed from config, for example **config.POLICY_TRIGGERS_SECTION_TITLE**.
PURPOSE
Give one section of the Highways Policy.
REQUIREMENTS
- The section is read from **config.HIGHWAYS_POLICY_DOCUMENT** and must open with **section_number** and **section_title**.

RETURNS
str. The section's text.

MODULE: tools/decision_tools.py
Decides whether a decision may be recorded, so that every refusal is given before the workflow resumes and before any language model call is spent.

FUNCTION SIGNATURE
`find_review_conflict(review_status, pending_decision, review_decision, report_reference)`

PARAMETERS
review_status — **str**. The review's stored status, passed by **submit_review_decision**.
pending_decision — **dict | None**. The decision already handed to the paused run and not yet recorded, or **None**, passed by **submit_review_decision**.
review_decision — **dict**. The decision now being made: **decision**, **decided_by**, **decided_by_role**, **decided_by_district** and **decision_reason**, built by **submit_review_decision**.
report_reference — **str**. The report the review is of, read from the stored review by **submit_review_decision**.
PURPOSE
Say whether a review may still take this decision.
REQUIREMENTS

- A review whose status is one of **config.DECIDED_REVIEW_STATUSES** gives **config.ALREADY_DECIDED_MESSAGE**.
- A pending decision that is not this decision gives **config.DECISION_IN_PROGRESS_MESSAGE**; the same decision sent again to finish it gives nothing.

RETURNS
str | None. The conflict in the configuration's words, or **None**.

FUNCTION SIGNATURE
`find_decision_refusal(review_decision, report, screening, opened_by)`

PARAMETERS
review_decision — **dict**. The decision being made, as for **find_review_conflict**, passed by **submit_review_decision** or **await_inspector_decision_node**.
report — **dict**. The report the review is of: the **report_profile** the paused run's checkpoint holds, passed by **submit_review_decision**, or the workflow state's, passed by **await_inspector_decision_node**.
screening — **dict**. That report's screening, from the same checkpoint or state, passed by the same callers.
opened_by — **str**. The username of whoever opened the review, the run's **coordinator_id**, passed by the same callers.
PURPOSE
Say why a decision may not be recorded, if it may not.
REQUIREMENTS

- A screening of another report is refused with **config.SCREENING_OF_ANOTHER_REPORT_MESSAGE**.
- Otherwise the first refusal below that applies is given, in the configuration's words; with none, the decision may be recorded.

Checked in this orderRefusalA field of **config.DECISION_RECORD_FIELDS** missing or blank**config.DECISION_FIELDS_MISSING_MESSAGE**, every such field named in that order, joined by a comma and a spaceA decision not in **config.DECISIONSconfig.UNKNOWN_DECISION_MESSAGE**, with **config.DECISIONS** joined by a comma and a spaceA role other than **config.ROLE_HIGHWAYS_INSPECTORconfig.DECIDING_ROLE_MESSAGE**A district whose streets in **config.INSPECTION_DISTRICT_STREETS** do not include the report's street**config.NOT_APPOINTED_MESSAGE**The person who opened the review**config.OWN_REVIEW_MESSAGE**A decision not in the screening's **decisions_openconfig.DECISION_NOT_OPEN_MESSAGE**, with the reason **config.ORDER_CLOSED_REASONS** gives for the assessment
RETURNS
str | None. The refusal in the configuration's words, or **None** when the decision may be recorded.

MODULE: tools/review_document_tools.py
The facts a works order and an inspection request are written from, and the coordination brief, all built by code from the report and its screening. No model writes any of them.

FUNCTION SIGNATURE
`build_works_order_details(report, screening)`

PARAMETERS
report — **dict**. The report, the **report_profile** of the workflow state, passed by **draft_order_document_node**.
screening — **dict**. That report's screening, from the workflow state, passed by the same node.
PURPOSE
Give the facts a works order is written from.
REQUIREMENTS

- A screening of another report is refused with **config.SCREENING_OF_ANOTHER_REPORT_MESSAGE**.
- The details carry the report reference under **config.REPORT_REFERENCE_COLUMN**, every column of **config.WORKS_ORDER_DETAIL_COLUMNS** (a blank as **None**), and **measurements**: for each screened defect type, the columns of **config.WORKS_ORDER_MEASUREMENT_COLUMNS** the report answers, as the report gives them.

RETURNS
dict. The works order details.

FUNCTION SIGNATURE
`build_inspection_request_details(report, screening, inspection_records, disagreements)`

PARAMETERS
report — **dict**. The report, the **report_profile** of the workflow state, passed by **draft_order_document_node**.
screening — **dict**. That report's screening, from the workflow state, passed by the same node.

inspection_records — **dict[str, str]**. Each screened defect type, in order, with what its standard says an inspection records, read with **read_inspection_records** by the same node.
disagreements — **list[dict]**. The report's flags as **find_disagreements** gave them, from the workflow state, passed by the same node.
PURPOSE
Give the facts an inspection request is written from.
REQUIREMENTS

- A screening of another report is refused with **config.SCREENING_OF_ANOTHER_REPORT_MESSAGE**, and **inspection_records** not keyed by the screened defect types in order is refused, naming both.
- **requirements_to_settle** holds every requirement not Met, with its **defect_type**, **requirement_name**, **requirement_sentence**, **result**, **report_answers** (each of its answer columns, a blank as **None**) and **blank_answers**.

RETURNS
dict. The report reference under **config.REPORT_REFERENCE_COLUMN**, **defect_types**, **requirements_to_settle**, **inspection_records** and **flagged_statements**.

FUNCTION SIGNATURE
`build_coordination_brief(report, screening, register_count, likelihood, crew_hours, disagreemen`
PARAMETERS
report — **dict**. The report, the **report_profile** of the workflow state, passed by **record_report_outcome_node**.
screening — **dict**. That report's screening, from the workflow state, passed by the same node.
register_count — **dict**. Its street works as **count_live_street_works** gave them, from the workflow state, passed by the same node.
likelihood — **dict**. Its repeat likelihood as **estimate_repeat_likelihood** gave it, from the workflow state, passed by the same node.
crew_hours — **dict**. Its expected crew hours as **estimate_expected_crew_hours** gave them, from the workflow state, passed by the same node.
disagreements — **list[dict]**. Its flags as **find_disagreements** gave them, from the workflow state, passed by the same node.
PURPOSE
Give the network coordination team the record a referral sends them.
REQUIREMENTS

- A screening of another report is refused with **config.SCREENING_OF_ANOTHER_REPORT_MESSAGE**.
- The brief carries the report reference under **config.REPORT_REFERENCE_COLUMN**, every column of **config.COORDINATION_BRIEF_DETAIL_COLUMNS** (a blank as **None**), **requirements** (each with **defect_type**, **requirement_name**, **result** and **blank_answers**), **coordination_triggers**, **live_permits**, **conflicting_permits**, **flagged_statements**, **repeat_likelihood**, **likelihood_band**, **expected_crew_hours**, **planned_crew_hours** and **overrun_expected**.

RETURNS
dict. The coordination brief.

============== PERSISTENCE ==============
Six tables hold everything the system is later asked to account for: who signed in, what ran, what was reviewed, what a Highways Inspector decided, what the language model calls of a review cost, and every question the chat replied to. Apart from the paused runs the checkpoint file holds, nothing is written anywhere else. No table records a date or a time: runs and reviews are listed by report reference, and calls, decisions and chat turns by their own numbers.

MODULE: persistence/database.py
The connection and the session factory. It defines no function: two module-level objects are built once, when the module is first imported, and everything else in the project uses them.
MODULE-LEVEL OBJECTS

- **database_engine** — **sqlalchemy.engine.Engine**. The connection to the database at **config.STREETCARE_DATABASE_URL**, usable from more than one thread, because the web framework answers on threads of its own.
- **session_factory** — **sqlalchemy.orm.sessionmaker**. Bound to that engine, and the only way the rest of the project opens a session.

REQUIREMENTS

- Both are created when the module is imported, and no other engine is created anywhere in the project.
- Importing this module neither creates tables nor writes anything; creating the tables is the work of **run_database_setup.py**.

MODULE: persistence/models.py
Declares the six tables. Table and column names are read by the marking suite and by the service's replies, and must be reproduced exactly, in lower case. The table names are also in config: **config.USER_ACCOUNTS_TABLE**, **config.AGENT_RUNS_TABLE**, **config.REPORT_REVIEWS_TABLE**, **config.REVIEW_DECISIONS_TABLE**, **config.LLM_CALLS_TABLE** and **config.CHAT_TURNS_TABLE**.

CLASS
`Base`
The declarative base every table below inherits from, and the object **run_database_setup.py** uses to create them all.

TABLE

user_accounts
CLASS
`UserAccount`

COLUMNS

Column	Type	Notes
username	**str**	The key
display_name	**str**	The name a page shows
role	**str**	One of the three roles
district	**str | None**	The district a Highways Inspector is appointed for, and nothing for the other roles
password_hash	**str**	Never the password itself
password_salt	**str**	This account's own salt
session_token	**str | None**	The current session, unique, or nothing when signed out

TABLE
agent_runs
CLASS
`AgentRun`

COLUMNS

Column	Type	Notes
run_id	**str**	The key
thread_id	**str**	The identifier the paused workflow is stored under, unique
report_reference	**str**	The report being reviewed
coordinator_id	**str**	The username of the person who opened the review
coordinator_request	**str**	What they asked to be looked at, as long text
status	**str**	One of **config.AGENT_RUN_STATUS_RUNNING**, **config.AGENT_RUN_STATUS_AWAITING_DECISION**, **config.AGENT_RUN_STATUS_COMPLETE** and **config.AGENT_RUN_STATUS_FAILED**
llm_call_count	**int**	Calls recorded against the run: nought when it opens, then written at the gate, at the end and on failure

TABLE
report_reviews
CLASS
`ReportReview`

COLUMNS

Column	Type	Notes
review_id	**str**	The key
run_id	**str**	The run that produced it, unique
report_reference	**str**	The report
revision_number	**int**	The revision that was reviewed
defect_type	**str**	Its defect type
linked_defect_type	**str**	Its linked defect type, or **config.NO_LINKED_DEFECT**
street_name	**str**	Its street
district	**str | None**	Its district
coordinator_request	**str**	What the review was opened to look at, as long text
requirements	**list**	Every requirement with only **defect_type**, **standard_document**, **requirement_kind**, **requirement_name**, **requirement_sentence**, **result** and **blank_answers**, without its **rules**, as JSON
requirements_met_count	**int**	Requirements Met
requirements_not_met_count	**int**	Requirements Not met
requirements_not_documented_count	**int**	Requirements Not documented
live_permits	**list**	The permits live on the street, as JSON
conflicting_permits	**list**	Those that conflict with the repair, as JSON
coordination_triggers	**list**	The coordination triggers that apply, as JSON
assessment	**str**	The assessment
decisions_open	**list**	The decisions open, as JSON
evidence	**list**	Every finding with the requirements it bears on, as JSON
disagreements	**list**	Every flagged sentence, as JSON
repeat_likelihood	**float**	The likelihood
likelihood_band	**str**	Its band
expected_crew_hours	**float**	The expected crew hours
planned_crew_hours	**float | None**	The planned crew hours
overrun_expected	**bool | None**	Whether an overrun is expected
crew_hours_text	**str**	The expected crew hours in words, as long text
review_plan_text	**str**	The planner's write-up, as long text
requirements_analysis_text	**str**	The requirements analyst's write-up, as long text
evidence_review_text	**str**	The evidence analyst's write-up, as long text
prediction_analysis_text	**str**	The prediction analyst's write-up, as long text
recommendation_text	**str**	The writer's recommendation, as long text
recommended_decision	**str**	One of the decisions open
status	**str**	One of **config.REVIEW_STATUS_OPTIONS**
order_document_text	**str | None**	The works order or the inspection request, where one was written, as long text
document_details	**dict | None**	The details that document carries, as JSON, and nothing where none was written
resident_update_text	**str | None**	The resident update, where one was written, as long text
coordination_brief	**dict | None**	The coordination brief, as JSON, and nothing unless referred

TABLE
review_decisions
CLASS
`ReviewDecision`

COLUMNS

Column	Type	Notes
decision_id	**int**	The key, assigned by the database
review_id	**str**	The review decided, unique
run_id	**str**	The run it belongs to
decision	**str**	One of **config.DECISIONS**
decided_by	**str**	The deciding Highways Inspector's username
decided_by_role	**str**	Their role
decided_by_district	**str**	The district they are appointed for
decision_reason	**str**	Their own words, as long text

TABLE
llm_calls

CLASS
`LlmCall`
COLUMNS

Column	Type	Notes
call_id	**int**	The key, assigned by the database
run_id	**str**	The run that made the call
agent_name	**str**	Which agent made it
model_name	**str**	Which model answered
prompt_tokens	**int**	The tokens sent, as the model counted them
response_tokens	**int**	The tokens written
total_tokens	**int**	The total the model counted
latency_milliseconds	**int**	How long it took

TABLE
chat_turns
CLASS
`ChatTurn`
COLUMNS

Column	Type	Notes
turn_id	**int**	The key, assigned by the database
username	**str**	Who asked
user_role	**str**	Their role at the time
question_text	**str**	What they asked, as long text
question_type	**str**	One of **config.CHAT_TYPE_OPTIONS**
report_reference	**str | None**	The report the question is about, if any
street_name	**str | None**	The street a figure question is about, if any
defect_type	**str | None**	The defect type a figure question is about, if any
answer_text	**str**	What they were told, refusal included, as long text
source_documents	**str**	The documents the answer rested on, written as text
closest_distance	**float**	How far the nearest passage was
was_refused	**bool**	Whether the answer was a refusal

REQUIREMENTS
- Every column above exists under exactly that name, and only a column marked as able to be empty accepts nothing.

MODULE: persistence/repository.py
Every read and write against the six tables. Nothing outside this module runs a query. Each function works inside the session its caller opened from **session_factory**: a write joins the caller's transaction, which the caller commits as one unit, so that what belongs together is saved together. A read returns plain records, never database objects.

FUNCTION SIGNATURE
`insert_user_account(session, account_record)`
PARAMETERS
session — **sqlalchemy.orm.Session**. An open session, opened by **run_database_setup.py**.
account_record — **dict**. One account's columns, built by **run_database_setup.py**, with the password already in its stored form.
PURPOSE
Create one sign-in account.
REQUIREMENTS
- The row is written into the caller's transaction; only the columns handed in are set.

RETURNS
None.

FUNCTION SIGNATURE
`fetch_user_account(session, condition)`
PARAMETERS

session — **sqlalchemy.orm.Session**. An open session, opened by **sign_in_user**, **sign_out_user**, **authorize_request** or **run_database_setup.py**.
condition — **sqlalchemy.ColumnElement**. A comparison on the **user_accounts** columns, written by the same caller: on **username** to sign in or set up, or on **session_token** to guard a request or sign out.
PURPOSE
Find one account.
REQUIREMENTS

- The record carries every column of the account, so a password can be checked or a role and a district read without a second query.
- Nothing is returned when no account matches.

RETURNS
dict | None. The account as a plain record, or nothing.

FUNCTION SIGNATURE
`update_session_token(session, username, session_token)`

PARAMETERS
session — **sqlalchemy.orm.Session**. An open session, opened by **sign_in_user** or **sign_out_user**.
username — **str**. Whose token to set, the account's **username**, passed by the same endpoint.
session_token — **str | None**. The new token **sign_in_user** made, or **None** from **sign_out_user** to end the session.
PURPOSE
Start or end one person's session.
REQUIREMENTS

- The account's token is replaced by the one given; an unknown username is refused, naming it.

RETURNS
None.

FUNCTION SIGNATURE
`insert_agent_run(session, run_record)`

PARAMETERS
session — **sqlalchemy.orm.Session**. An open session, opened by **load_report_context_node**.
run_record — **dict**. The run's columns, built by the same node.
PURPOSE
Open the record of one workflow run.
REQUIREMENTS

- The row is written into the caller's transaction.

RETURNS
None.

FUNCTION SIGNATURE
`update_agent_run(session, run_id, status_value, llm_call_count)`

PARAMETERS
session — **sqlalchemy.orm.Session**. An open session, opened by **prepare_report_recommendation_node**, **record_report_outcome_node** or **report_failed_request**.
run_id — **str**. The run to update, from the workflow state or, for **report_failed_request**, from the checkpoint of the run that failed.
status_value — **str**. Its new status, **config.AGENT_RUN_STATUS_AWAITING_DECISION**, **config.AGENT_RUN_STATUS_COMPLETE** or **config.AGENT_RUN_STATUS_FAILED**, chosen by the same caller.
llm_call_count — **int**. The calls recorded against the run so far, counted by the same caller.
PURPOSE
Move a run on, recording what it has cost.
REQUIREMENTS

- The run takes the status and the count given; an unknown run is refused, naming it.

RETURNS
None.

FUNCTION SIGNATURE
`insert_llm_call(session, run_id, call_record)`

PARAMETERS
session — **sqlalchemy.orm.Session**. An open session, opened by the node that made the call.
run_id — **str**. The run that made the call, from the workflow state, passed by the same node.
call_record — **dict**. The call's record as **call_language_model** returned it inside the agent's result, passed by the same node.
PURPOSE
Record one language model call against its run.
REQUIREMENTS

- The row carries the run and every field of the call record, written into the caller's transaction.

RETURNS
None.

FUNCTION SIGNATURE

`fetch_llm_calls(session, run_id)`

PARAMETERS
session — **sqlalchemy.orm.Session**. An open session, opened by **get_review_calls** or **report_failed_request**.
run_id — **str**. The run whose calls are wanted, read from the stored review by **get_review_calls**, or from the failed run's checkpoint by **report_failed_request**.
PURPOSE
Read back what a run's calls cost.
REQUIREMENTS

- The run's calls come back in the order they were made, each with **agent_name**, **model_name**, **prompt_tokens**, **response_tokens**, **total_tokens** and **latency_milliseconds**.

RETURNS
list[dict]. One record per call.

FUNCTION SIGNATURE
`insert_report_review(session, review_record)`

PARAMETERS
session — **sqlalchemy.orm.Session**. An open session, opened by **prepare_report_recommendation_node**.
review_record — **dict**. The review's columns, built by the same node.
PURPOSE
Store the review waiting at the gate.
REQUIREMENTS

- The row is written into the caller's transaction.

RETURNS
None.

FUNCTION SIGNATURE
`fetch_review_records(session, condition)`

PARAMETERS
session — **sqlalchemy.orm.Session**. An open session, opened by an endpoint of **api/routes.py**, or passed on by **answer_report_record** from **ask_street_care_assistant**.
condition — **sqlalchemy.ColumnElement**. A comparison on the **report_reviews** columns, written by the same caller, for example on **review_id**, on **report_reference** and **revision_number**, or on **status**.
PURPOSE
Read back reviews with everything recorded about them.
REQUIREMENTS

- Each record carries every column of the review, its run's **thread_id**, **coordinator_id** and **llm_call_count**, and its decision's **decision**, **decided_by**, **decided_by_role**, **decided_by_district** and **decision_reason**, each nothing while undecided.
- The records come in report-reference order, then revision, then review identifier.

RETURNS
list[dict]. One record per review.

FUNCTION SIGNATURE
`insert_review_decision(session, decision_record)`

PARAMETERS
session — **sqlalchemy.orm.Session**. An open session, opened by **record_report_outcome_node**.
decision_record — **dict**. The decision's columns, built by the same node.
PURPOSE
Record a Highways Inspector's decision.
REQUIREMENTS

- The row is written into the caller's transaction, one per review.

RETURNS
None.

FUNCTION SIGNATURE
`record_review_outcome(session, review_id, outcome_record)`

PARAMETERS
session — **sqlalchemy.orm.Session**. An open session, opened by **record_report_outcome_node**.
review_id — **str**. The review decided, from the workflow state, passed by **record_report_outcome_node**.
outcome_record — **dict**. The columns the decision changes, **status**, **order_document_text**, **document_details**, **resident_update_text** and **coordination_brief**, built by the same node.
PURPOSE
Write what a decision led to onto its review.
REQUIREMENTS

- Exactly the columns given are set on that review; a review identifier that matches no review is refused, naming it.

RETURNS
None.

FUNCTION SIGNATURE

`insert_chat_turn(session, turn_record)`

PARAMETERS

session — **sqlalchemy.orm.Session**. An open session, opened by **ask_street_care_assistant**.

turn_record — **dict**. The turn's columns, built by the same endpoint.

PURPOSE

Record one chat exchange.

REQUIREMENTS

- The row is written into the caller's transaction, refusals included.

RETURNS

None.

FUNCTION SIGNATURE

`fetch_chat_turns(session, condition, turn_limit)`

PARAMETERS

session — **sqlalchemy.orm.Session**. An open session, opened by **list_own_chat_turns** or **list_recent_chat_turns**, or passed on by **recall_conversation** from **ask_street_care_assistant**.

condition — **sqlalchemy.ColumnElement**. A comparison on the **chat_turns** columns, written by the caller: on **username** by **recall_conversation** and **list_own_chat_turns**, or one that every turn meets by **list_recent_chat_turns**.

turn_limit — **int**. The most turns wanted: **config.CHAT_HISTORY_TURN_LIMIT** from **recall_conversation**, or the endpoint's own **turn_limit**.

PURPOSE

Read back the most recent chat turns.

REQUIREMENTS

- The turns that meet the condition come back newest first, at most **turn_limit** of them, each with every column.

RETURNS

list[dict]. One record per turn.

========== AGENTIC WORKFLOW ==========

Five agents review a report, and up to two more write what the decision calls for. Each agent is exactly one language model call. None of them works anything out: every result, figure and passage an agent is shown was established by the standards, the register, the models, the documents and the tools, and each prompt tells its agent to use only what it is given. A run makes **config.LLM_CALLS_BEFORE_DECISION** calls before the gate and, in all, the number **config.LLM_CALLS_BY_DECISION** gives for its decision.

The workflow stops before any decision is recorded and before any document is written, and the pause is written to disk so that it survives a restart.

MODULE: agentic/state.py

Declares what the workflow carries from one node to the next, as a **typing.TypedDict**, so every node and the router read the state as a dictionary.

CLASS

`ReportReviewState`

STATE KEYS

Key	Type	Written by	What it carries
report_reference	**str**	**start_report_review**	The report being reviewed
coordinator_request	**str**	**start_report_review**	What the person who opened the review asked to be looked at
coordinator_id	**str**	**start_report_review**	The username of whoever opened the review
thread_id	**str**	**start_report_review**	The identifier the pause is stored under
run_id	**str**	**load_report_context_node**	The run, as recorded in **agent_runs**
review_id	**str**	**load_report_context_node**	The review this run will write
report_profile	**dict**	**load_report_context_node**	The report as **fetch_report** returned it, with its counts of live and conflicting permits
report_documents	**list**	**load_report_context_node**	Its documents as **fetch_report_documents** returned them
register_count	**dict**	**load_report_context_node**	The street works as **count_live_street_works** returned them
screening	**dict**	**load_report_context_node**	The screening under its own standard and its linked type's
evidence_findings	**list**	**load_report_context_node**	Every sentence read, with the requirements it bears on
disagreements	**list**	**load_report_context_node**	Every flagged sentence
likelihood	**dict**	**load_report_context_node**	The repeat likelihood and its band
crew_hours	**dict**	**load_report_context_node**	The expected crew hours
review_plan	**dict**	**plan_report_review_node**	The planner's reply
requirements_analysis	**dict**	**assess_report_requirements_node**	The requirements analyst's reply
evidence_review	**dict**	**analyse_report_statements_node**	The evidence analyst's reply
prediction_analysis	**dict**	**interpret_model_predictions_node**	The prediction analyst's reply
review_decision	**dict**	**await_inspector_decision_node**	**decision**, one of **config.DECISIONS**; **decided_by**, the deciding Highways Inspector's username; **decided_by_role**; **decided_by_district**; **decision_reason**, their own words
order_document	**dict**	**draft_order_document_node**	The works order's or the inspection request's fields
document_details	**dict**	**draft_order_document_node**	The details that document carries
resident_update	**dict**	**draft_resident_update_node**	The resident update's fields
llm_call_count	**int**	**start_report_review**, as **0**, and every agent node	The calls recorded so far

REQUIREMENTS

- Every key above is declared, and nothing else is.
- **llm_call_count** is added to by each node that returns it rather than replaced; every other key is replaced by whichever node writes it.

MODULE: agentic/language_model.py

The language model client and the reading of a labelled reply. The key and the model name are read from the environment, through **config.GEMINI_API_KEY_VARIABLE** and **config.GEMINI_MODEL_VARIABLE**, which are set in **.env** and loaded from **config.ENV_FILE** when the module is imported. No key or model name is written into the code.

MODULE-LEVEL OBJECTS

- **generation_settings** — **google.genai.types.GenerateContentConfig**. The one generation configuration every call uses, holding **config.GEMINI_MAX_OUTPUT_TOKENS** and **config.GEMINI_TEMPERATURE**, with automatic function calling turned off.

FUNCTION SIGNATURE

```
call_language_model(prompt_text, agent_name)
```

PARAMETERS

prompt_text — **str**. The whole prompt, assembled and passed by an agent function of **agentic/agents.py** or by **answer_from_documents**.
agent_name — **str**. The calling agent's name from config, for example **config.AGENT_NAME_PLANNER**, passed by the same caller.
PURPOSE
Make one language model call and measure what it cost.
REQUIREMENTS

- The key and the model name are read at the moment of the call; when either is missing the call is refused, naming both settings and **.env**.
- The reply is generated under **generation_settings**, and the client is open for the call alone.
- An empty reply, a reply that did not finish normally, and a reply that does not say how many tokens it used are refused, naming the agent and the model.
- **call_record** carries **agent_name**, **model_name**, **prompt_tokens**, **response_tokens** and **total_tokens** as the reply counts them, and **latency_milliseconds**, whole milliseconds measured around the call itself through **config.MILLISECONDS_PER_SECOND**.

RETURNS
dict. **response_text**, the reply with surrounding whitespace removed, and **call_record**.

FUNCTION SIGNATURE
`parse_labelled_response(response_text, field_labels, agent_name)`
PARAMETERS
response_text — **str**. What the model replied, passed by an agent function or by **answer_from_documents**, or the stored **resident_update_text**, passed by **answer_report_record**.
field_labels — **list[str]**. The labels the prompt asked for, in upper case and in order, written by the same caller.
agent_name — **str**. Which agent replied, a name from config such as **config.AGENT_NAME_WRITER**, passed by the same caller for the refusal.
PURPOSE
Read a labelled reply back into its fields, tolerantly enough to survive the way models decorate their answers.
REQUIREMENTS

- Bold markers are dropped from every line, and every line is trimmed.
- A line starts a field when its text before the first colon is one of **field_labels**, ignoring case, hyphens written for spaces and the decoration **config.RESPONSE_DECORATION_EDGE_PATTERN** finds at its edges.
- The text after that colon, less any of **config.RESPONSE_DECORATION_CHARACTERS** at either end, opens the field; each later line up to the next label is added on a new line.
- A line of decoration alone adds nothing, text before the first label is ignored, and a label written again adds its text to its field on a new line.
- A label that is missing, or answered with nothing, is refused, naming the agent and every such label.

RETURNS
dict. Exactly the labels asked for, in the order asked.

FUNCTION SIGNATURE
`join_labelled_response(labelled_fields)`
PARAMETERS
labelled_fields — **dict**. An agent's fields as **parse_labelled_response** returned them, passed by **prepare_report_recommendation_node** or **record_report_outcome_node**.
PURPOSE
Write an agent's fields back as one text for the record.
REQUIREMENTS

- Each field is written as its label, a colon, a space and its text, in the order given, with one blank line between fields.

RETURNS
str. The fields as one text.

MODULE: agentic/agents.py
The seven agents. Each hands the model facts that are already established and asks for prose in a fixed layout of labelled fields, read back by **parse_labelled_response**. Every prompt speaks for the **config.OFFICE_NAME** of **config.ORGANISATION_NAME**, states each value as **describe_record** words it and each answer as **config.FORM_ANSWER_LABELS** labels it, and states an empty list of documents, permits, triggers or flags as none.

FUNCTION SIGNATURE
`plan_review(report_profile, report_documents, screening, register_count, coordinator_request)`
PARAMETERS
report_profile — **dict**. The report, passed by **plan_report_review_node** from the workflow state.
report_documents — **list[dict]**. The report's documents, from the workflow state, passed by the same node.
screening — **dict**. The report's screening, from the workflow state, passed by the same node.
register_count — **dict**. The street works on its street, from the workflow state, passed by the same node.
coordinator_request — **str**. What the person who opened the review asked, from the workflow state, passed by the same node.
PURPOSE

Decide what the review has to establish about this report.

REQUIREMENTS

- The prompt carries the report, its documents' identifiers and titles in order, and every requirement's defect type, name, result and blank answers.
- It also carries the counts, the permits live and conflicting, the triggers, the assessment, the decisions open and the coordinator's request.
- It forbids altering any screening result or naming any figure it does not carry.
- The reply carries **PRIORITY**, **CHECKS** and **RATIONALE**; the call is made as **config.AGENT_NAME_PLANNER**.

RETURNS

dict. **review_plan**, holding the three fields, and **call_record**.

FUNCTION SIGNATURE

```
analyse_requirements(screening, report_profile, register_count, review_plan, trigger_rules)
```

PARAMETERS

screening — **dict**. The report's screening, passed by **assess_report_requirements_node** from the workflow state.
report_profile — **dict**. The report, from the workflow state, passed by the same node.
register_count — **dict**. The street works on its street, from the workflow state, passed by the same node.
review_plan — **dict**. The planner's three fields, from the workflow state, passed by the same node.
trigger_rules — **str**. The Highways Policy's section on the coordination triggers, read by the same node with **read_policy_section**.

PURPOSE

Account for each requirement's result under each standard, and for the coordination triggers that apply, in words a Highways Inspector can check against the report.

REQUIREMENTS

- The prompt carries every requirement in the screening's order with its standard, its sentence, the answers it reads (**config.REQUIREMENT_ANSWER_COLUMNS**), its result and its blank answers.
- It also carries the counts, **trigger_rules**, the answers the triggers read (**config.TRIGGER_ANSWER_COLUMNS**), the permits live and conflicting, the triggers that apply, the assessment, the decisions open and the plan's **CHECKS**.
- It binds the agent to the results as given: a blank answer stays blank, and the report is never presented as decided, rejected or closed.
- The reply carries **REQUIREMENTS MET**, **REQUIREMENTS NOT MET**, **NOT DOCUMENTED**, **COORDINATION TRIGGERS** and **REQUIREMENTS SUMMARY**; the call is made as **config.AGENT_NAME_REQUIREMENTS**.

RETURNS

dict. **requirements_analysis** and **call_record**.

FUNCTION SIGNATURE

```
review_evidence(evidence_findings, disagreements, reviewer_checks, report_profile, screening, r
```

PARAMETERS

evidence_findings — **list[dict]**. Every sentence read, with the requirements it bears on, passed by **analyse_report_statements_node** from the workflow state.
disagreements — **list[dict]**. Every flagged sentence, from the workflow state, passed by the same node.
reviewer_checks — **dict[str, str]**. Each sentence type found with the reviewer's checks its guide sets, read by the same node.
report_profile — **dict**. The report, from the workflow state, passed by the same node.
screening — **dict**. The report's screening, from the workflow state, passed by the same node.
review_plan — **dict**. The planner's three fields, from the workflow state, passed by the same node.

PURPOSE

Set the evidence in the documents beside the report, with the flagged sentences and the guides' checks.

REQUIREMENTS

- The prompt carries each answer the screened requirements read and those of **config.STATEMENT_GUIDE_ANSWER_COLUMNS**, each once.
- It also carries every sentence, document by document, with its type and the requirements it bears on; every flag with its figures beside the report's; each guide's checks with their guide; and the plan's **CHECKS**.
- It treats a document as evidence only, never in place of an answer on the report; flags are left to the Highways Inspector, and any other difference is advice only.
- The reply carries **EVIDENCE FOUND**, **DISAGREEMENTS**, **CHECKS TO MAKE** and **EVIDENCE SUMMARY**; the call is made as **config.AGENT_NAME_EVIDENCE**.

RETURNS

dict. **evidence_review** and **call_record**.

FUNCTION SIGNATURE

```
analyse_predictions(likelihood, crew_hours, report_profile, screening, review_plan)
```

PARAMETERS

likelihood — **dict**. The repeat likelihood and its band, passed by **interpret_model_predictions_node** from the workflow state.
crew_hours — **dict**. The expected crew hours, from the workflow state, passed by the same node.
report_profile — **dict**. The report, from the workflow state, passed by the same node.
screening — **dict**. The report's screening, from the workflow state, passed by the same node.
review_plan — **dict**. The planner's three fields, from the workflow state, passed by the same node.

PURPOSE

Interpret the repeat likelihood and the expected crew hours, with the inputs behind each.

REQUIREMENTS

- The prompt carries the likelihood to **config.LIKELIHOOD_DECIMALS** places with its band and the bands' thresholds, and every input of the repeat model.
- It also carries the expected crew hours as **describe_expected_crew_hours** words them with **config.OVERRUN_RATIO**, every input of the crew hours model, how a blank input is filled and the plan's **CHECKS**.
- It states that neither figure decides anything or appears in any document, and forbids recalculating either.
- The reply carries **LIKELIHOOD**, **EXPECTED CREW HOURS** and **PREDICTION SUMMARY**; the call is made as **config.AGENT_NAME_RISK**.

RETURNS

dict. **prediction_analysis** and **call_record**.

FUNCTION SIGNATURE

```
write_review_recommendation(report_profile, coordinator_request, screening, register_count, lik
```

PARAMETERS

report_profile — **dict**. The report, passed by **prepare_report_recommendation_node** from the workflow state.
coordinator_request — **str**. What the person who opened the review asked, from the workflow state, passed by the same node.
screening — **dict**. The report's screening, from the workflow state, passed by the same node.
register_count — **dict**. The street works on its street, from the workflow state, passed by the same node.
likelihood — **dict**. The repeat likelihood and its band, from the workflow state, passed by the same node.
crew_hours — **dict**. The expected crew hours, from the workflow state, passed by the same node.
review_plan — **dict**. The planner's three fields, from the workflow state, passed by the same node.
requirements_analysis — **dict**. The requirements analyst's five fields, from the workflow state, passed by the same node.
evidence_review — **dict**. The evidence analyst's four fields, from the workflow state, passed by the same node.
prediction_analysis — **dict**. The prediction analyst's three fields, from the workflow state, passed by the same node.
decision_rules — **str**. The Highways Policy's section on the three decisions, read by the same node with **read_policy_section**.

PURPOSE

Write the review a Highways Inspector decides from, and recommend one of the decisions open.

REQUIREMENTS

- The prompt carries the report, the coordinator's request, the planner's **PRIORITY** and **CHECKS** and every field of the other three analysts.
- It also carries each requirement's result, the permits live and conflicting, the triggers, the assessment, the two figures, **decision_rules** and the decisions open.
- It keeps the agent to a recommendation: the assessment, unless the review justifies another open decision and says why; neither figure may be a reason, and nothing is presented as decided, rejected or closed.
- The reply carries **RECOMMENDATION** and **RECOMMENDED DECISION**; the call is made as **config.AGENT_NAME_WRITER**.
- A **RECOMMENDED DECISION** over more than one line, or not among the decisions open once in upper case and without the edge decoration **config.RESPONSE_DECORATION_EDGE_PATTERN** finds, is refused, naming the agent.
- The **RECOMMENDED DECISION** returned is that checked form.

RETURNS

dict. **review_recommendation**, holding the two fields, and **call_record**.

FUNCTION SIGNATURE

```
draft_order_document(decision, report_profile, document_details, document_guidance)
```

PARAMETERS

decision — **str**. **config.DECISION_ORDER** or **config.DECISION_INSPECT**, passed by **draft_order_document_node** from the workflow state.
report_profile — **dict**. The report, from the workflow state, passed by the same node.
document_details — **dict**. The works order details or the inspection request details, built by the same node with **build_works_order_details** or **build_inspection_request_details**.
document_guidance — **str**. For an order, what section **config.WORKS_ORDER_SECTION_NUMBER** of each standard screened says, each written as **config.STANDARD_PASSAGE_TEMPLATE**; for an inspection, the Highways Policy's section on the documents; read and passed by the same node.

PURPOSE

Write the works order to the repair contractor, or the inspection request to the inspector, the kind **config.DOCUMENT_KINDS** names.

REQUIREMENTS

- The prompt carries the decision with **config.DECISION_DESCRIPTIONS**, the recipient **config.DOCUMENT_RECIPIENTS** names, the report's channel, the details, the guidance and **config.DOCUMENT_SIGN_OFF**.
- It keeps out every model figure, every staff name and any invented time, date or contact detail.
- An order's details are labelled by **config.DOCUMENT_DETAIL_LABELS**, followed by each defect type's own measurements labelled by **config.FORM_ANSWER_LABELS**.
- An inspection's details give the report reference, the standards screened, and each requirement to settle with its standard, result and sentence.
- They also give the report's answers, the answers needed, what each standard says an inspection records, and every flagged sentence.
- The reply carries **SUBJECT** and **DOCUMENT**; the call is made as **config.AGENT_NAME_ORDER_WRITER**. A **SUBJECT** over more than one line, or not naming the report reference, is refused, naming the agent.

- Read without case and with each run of whitespace as one space, the subject and document name every defect type screened and the street for an order, or every requirement to settle for an inspection.
- They name none of **config.DOCUMENT_FORBIDDEN_TERMS**, and the **DOCUMENT** holds none of **config.TEMPLATE_SLOT_CHARACTERS**.
- The **DOCUMENT** ends with **config.DOCUMENT_SIGN_OFF**, compared without case and with each run of **config.SIGN_OFF_SPACING_PATTERN** as one space.
- A reply breaking any of the three rules above is refused, naming the agent.

RETURNS
dict. **order_document**, holding the two fields, and **call_record**.

FUNCTION SIGNATURE
`draft_resident_update(works_order_details, update_guidance)`

PARAMETERS
works_order_details — **dict**. The works order details **build_works_order_details** built, the run's **document_details**, passed by **draft_resident_update_node**.
update_guidance — **str**. What section **config.RESIDENT_UPDATE_SECTION_NUMBER** of each standard screened says the update tells the reporter, each written as **config.STANDARD_PASSAGE_TEMPLATE**, read and passed by the same node.
PURPOSE
Write the resident update for the person who reported the defect, after the works order.
REQUIREMENTS

- The prompt carries the details labelled by **config.DOCUMENT_DETAIL_LABELS**, the guidance and **config.DOCUMENT_SIGN_OFF**.
- It keeps out every model figure, the review's results, any promise of when the work will be done, every staff name and any invented time, date or contact detail.
- It never says the defect was visited, inspected or measured.
- The reply carries **HEADLINE** and **UPDATE**; the call is made as **config.AGENT_NAME_UPDATE_WRITER**. A **HEADLINE** over more than one line is refused, naming the agent.
- Read without case and with each run of whitespace as one space, the headline and update name the report reference, the street and any linked defect type.
- They name none of **config.DOCUMENT_FORBIDDEN_TERMS** and no account of **config.SEED_USER_ACCOUNTS**, by its username as a whole word (**config.NAME_IN_TEXT_PATTERN**) or its display name.
- The **UPDATE** holds none of **config.TEMPLATE_SLOT_CHARACTERS** and ends with **config.DOCUMENT_SIGN_OFF**, compared as for the works order.
- A reply breaking any of the three rules above is refused, naming the agent.

RETURNS
dict. **resident_update**, holding the two fields, and **call_record**.

MODULE: agentic/nodes.py
The ten steps of the workflow, the router, and the context every node and the service share. Each node takes the state, does one thing and returns only the keys it wrote. A node that calls an agent records the call in **llm_calls** against the run as soon as the agent function returns it, and returns **llm_call_count** as **1**.
MODULE-LEVEL RESOURCES
Everything below is loaded once, when the module is first imported, from the paths in config.

Name	Type	What it holds
repeat_model, **crew_hours_model**, **sentence_model**	**sklearn.pipeline.Pipeline**	The saved models, loaded by **load_model**
repeat_feature_names, **crew_hours_feature_names**	**list[str]**	Their saved schemas, loaded by **load_feature_schema**
open_reports	**pandas.DataFrame**	The live queue, loaded by **load_open_reports**
open_report_documents	**pandas.DataFrame**	Its documents, loaded by **load_report_documents**
street_works_register	**pandas.DataFrame**	The register, loaded by **load_street_works_register**
knowledge_documents	**list[dict]**	The sixteen documents' text, read by **extract_text_from_directory** from **config.DOCS_DIRECTORY**
knowledge_collection	**chromadb.Collection**	The knowledge base, opened by **open_vector_index**

FUNCTION SIGNATURE
`gather_report_context(report_reference)`

PARAMETERS
report_reference — **str**. A report in the live queue, passed by **load_report_context_node**, and by **api/routes.py** for every report it ranks.

PURPOSE
Establish everything about one report that needs no language model.
REQUIREMENTS

- **report_profile** is the report **fetch_report** reads from **open_reports**, carrying the two counts of **register_count** under **config.LIVE_STREET_WORKS_COLUMN** and **config.CONFLICTING_STREET_WORKS_COLUMN**.
- **report_documents** are what **fetch_report_documents** reads from **open_report_documents**; **register_count** is what **count_live_street_works** gives from **street_works_register**.
- **screening** is what **screen_report** gives for that profile under the standards of its defect type and any linked defect type, each read by **read_defect_standard** from **knowledge_documents**.
- **evidence_findings** are the findings of **read_document_evidence** with **sentence_model**, linked by **link_evidence_to_requirements**; **disagreements** are what **find_disagreements** gives for them and the profile.
- **likelihood** and **crew_hours** come from **estimate_repeat_likelihood** and **estimate_expected_crew_hours**, each given its model, its schema, the profile and the screening.
- No language model is called and nothing is written.

RETURNS
dict. **report_profile**, **report_documents**, **register_count**, **screening**, **evidence_findings**, **disagreements**, **likelihood** and **crew_hours**.

FUNCTION SIGNATURE
`load_report_context_node(state)`
PARAMETERS
state — **ReportReviewState**. The workflow's state, handed to the node by LangGraph; the node reads **report_reference**, **coordinator_request**, **coordinator_id** and **thread_id**.
PURPOSE
Gather the report's context and open the run.
REQUIREMENTS

- The run identifier is **config.RUN_ID_PREFIX** and the review identifier **config.REVIEW_ID_PREFIX**, each followed by **config.IDENTIFIER_HEX_LENGTH** random upper-case hexadecimal characters.
- The run is written to **agent_runs** with the four keys read, the status **config.AGENT_RUN_STATUS_RUNNING** and no calls; no language model is called.

RETURNS
dict. **run_id**, **review_id** and the eight keys of **gather_report_context**.

FUNCTION SIGNATURE
`plan_report_review_node(state)`
PARAMETERS
state — **ReportReviewState**. The workflow's state, handed to the node by LangGraph; the node reads **report_profile**, **report_documents**, **screening**, **register_count**, **coordinator_request** and **run_id**.
PURPOSE
Ask the planner what the review has to establish.
REQUIREMENTS

- **plan_review** is called once and its call recorded.

RETURNS
dict. **review_plan** and **llm_call_count**.

FUNCTION SIGNATURE
`assess_report_requirements_node(state)`
PARAMETERS
state — **ReportReviewState**. The workflow's state, handed to the node by LangGraph; the node reads **screening**, **report_profile**, **register_count**, **review_plan** and **run_id**.
PURPOSE
Ask the requirements analyst to explain the screening and the coordination triggers.
REQUIREMENTS

- The policy's section **config.POLICY_TRIGGERS_SECTION_NUMBER**, titled **config.POLICY_TRIGGERS_SECTION_TITLE**, is read with **read_policy_section** from **knowledge_collection**.
- **analyse_requirements** is called once and its call recorded.

RETURNS
dict. **requirements_analysis** and **llm_call_count**.

FUNCTION SIGNATURE
`analyse_report_statements_node(state)`
PARAMETERS
state — **ReportReviewState**. The workflow's state, handed to the node by LangGraph; the node reads **evidence_findings**, **disagreements**, **report_profile**, **screening**, **review_plan** and **run_id**.
PURPOSE
Ask the evidence analyst to review the documents with the guides' checks in hand.
REQUIREMENTS

- The reviewer's checks are read with **read_reviewer_checks** for each sentence type found that **config.SENTENCE_TYPE_GUIDE_DOCUMENTS** covers, in that order, and for no other.

- **review_evidence** is called once and its call recorded.

RETURNS
dict. **evidence_review** and **llm_call_count**.

FUNCTION SIGNATURE
`interpret_model_predictions_node(state)`

PARAMETERS
state — **ReportReviewState**. The workflow's state, handed to the node by LangGraph; the node reads **likelihood**, **crew_hours**, **report_profile**, **screening**, **review_plan** and **run_id**.
PURPOSE
Ask the prediction analyst to explain the two figures.
REQUIREMENTS

- **analyse_predictions** is called once and its call recorded.

RETURNS
dict. **prediction_analysis** and **llm_call_count**.

FUNCTION SIGNATURE
`prepare_report_recommendation_node(state)`

PARAMETERS
state — **ReportReviewState**. The workflow's state, handed to the node by LangGraph; the node reads the report's context, the four agents' replies, **coordinator_request**, **report_reference**, **review_id**, **run_id** and **llm_call_count**.
PURPOSE
Ask the writer for the recommendation, and store the review waiting at the gate.
REQUIREMENTS

- The decision rules are the policy's section **config.POLICY_DECISIONS_SECTION_NUMBER**, titled **config.POLICY_DECISIONS_SECTION_TITLE**, read with **read_policy_section**; **write_review_recommendation** is called once.
- The call, the review and the run are written in one transaction: the review to **report_reviews** at **config.REVIEW_STATUS_AWAITING**, the run at **config.AGENT_RUN_STATUS_AWAITING_DECISION** with every call so far, this node's included.
- The review holds the report's revision, defect type, linked defect type, street and district, and the coordinator's request.
- It holds the screening, each requirement without its **rules**, the permits live and conflicting, the findings, the flags and the two figures, with **crew_hours_text** as **describe_expected_crew_hours** words it.
- Each earlier agent's fields are stored joined by **join_labelled_response**; the writer's **RECOMMENDATION** is **recommendation_text** and its **RECOMMENDED DECISION** is **recommended_decision**.
- The five write-ups each hold at least **8** words, and no two are the same text.

RETURNS
dict. **llm_call_count**.

FUNCTION SIGNATURE
`await_inspector_decision_node(state)`

PARAMETERS
state — **ReportReviewState**. The workflow's state, handed to the node by LangGraph; the node reads **review_id**, **report_profile**, **screening** and **coordinator_id**.
PURPOSE
Stop the run until a Highways Inspector decides, and accept only a decision that may be recorded.
REQUIREMENTS

- The run stops here, handing out **review_id**, and resumes with the decision.
- A decision **find_decision_refusal** refuses is refused with its reason.
- Nothing is written: a paused node runs again from its start when the run resumes.

RETURNS
dict. **review_decision**.

FUNCTION SIGNATURE
`route_report_decision(state)`

PARAMETERS
state — **ReportReviewState**. The workflow's state, handed to the router by LangGraph; the router reads **review_decision**.
PURPOSE
Choose the path out of the gate, and out of the order document.
REQUIREMENTS

- The path is named by the decision itself.

RETURNS
str. The decision.

FUNCTION SIGNATURE
`draft_order_document_node(state)`

PARAMETERS

state — **ReportReviewState**. The workflow's state, handed to the node by LangGraph; the node reads **review_decision**, **report_profile**, **screening**, **disagreements** and **run_id**.
PURPOSE
Write the works order or the inspection request.
REQUIREMENTS

- For **config.DECISION_ORDER** the details come from **build_works_order_details**, and the guidance is **read_works_order_guidance** for each defect type screened, each written as **config.STANDARD_PASSAGE_TEMPLATE**, joined by a blank line.
- For **config.DECISION_INSPECT** the details come from **build_inspection_request_details**, with **read_inspection_records** for each defect type screened, and the guidance is the policy's section **config.POLICY_DOCUMENTS_SECTION_NUMBER**, titled **config.POLICY_DOCUMENTS_SECTION_TITLE**.
- **draft_order_document** is called once and its call recorded.

RETURNS
dict. **order_document**, **document_details** and **llm_call_count**.

FUNCTION SIGNATURE
`draft_resident_update_node(state)`
PARAMETERS
state — **ReportReviewState**. The workflow's state, handed to the node by LangGraph; the node reads **screening**, **document_details** and **run_id**.
PURPOSE
Write the resident update, after a works order.
REQUIREMENTS

- The guidance is **read_resident_update_guidance** for each defect type screened, each written as **config.STANDARD_PASSAGE_TEMPLATE**, joined by a blank line; **draft_resident_update** is called once and its call recorded.

RETURNS
dict. **resident_update** and **llm_call_count**.

FUNCTION SIGNATURE
`record_report_outcome_node(state)`
PARAMETERS
state — **ReportReviewState**. The workflow's state, handed to the node by LangGraph; the node reads **review_decision**, **order_document**, **document_details**, **resident_update**, **report_profile**, **screening**, **register_count**, **likelihood**, **crew_hours**, **disagreements**, **review_id**, **run_id** and **llm_call_count**.
PURPOSE
Record the decision and everything it led to, and close the run.
REQUIREMENTS

- The decision, the review's outcome and the closed run are written in one transaction: the decision to **review_decisions**, and the run at **config.AGENT_RUN_STATUS_COMPLETE** with the total calls recorded.
- The review takes the status **config.REVIEW_STATUS_BY_DECISION** gives; each text is stored joined by **join_labelled_response**, and nothing is stored for what was not written.
- An order or an inspection stores **order_document_text** and **document_details**, an order also **resident_update_text**, and a referral **coordination_brief** from **build_coordination_brief**.
- An order's two texts each hold at least **8** words, and they are not the same text.

RETURNS
dict. An empty dictionary: the work is in the database.

MODULE: agentic/graph.py
The order the nodes run in, the gate, the branches and the checkpoint.
FUNCTION SIGNATURE
`build_workflow(checkpoint_path)`
PARAMETERS
checkpoint_path — **str**. The file paused runs are written to, passed by **api/routes.py** from **config.CHECKPOINT_DATABASE_FILE**.
PURPOSE
Assemble the ten nodes into a workflow that can be run, paused and resumed.
REQUIREMENTS

- The ten nodes are registered under exactly the names above, and the first seven run in the order shown, from the start.
- There are exactly two conditional branches, both chosen by **route_report_decision**.
- One leaves the gate, for each of **config.DECISIONS**; the other leaves **draft_order_document**, a single node, for **config.DECISION_ORDER** and **config.DECISION_INSPECT**.
- Every path ends at **record_report_outcome**, the last node, which has its own edge to the end of the workflow.
- Every node is reachable from the start, and every connection outside the two branches is a plain edge.
- The workflow is compiled with a checkpointer writing to **checkpoint_path** through a connection usable from more than one thread.
- A run paused at the gate can therefore be resumed by another process, after a restart.
- Building the workflow runs nothing and calls no model.

RETURNS
langgraph.graph.state.CompiledStateGraph. The compiled LangGraph workflow.

========== THE REST SERVICE ==========
Every figure about a report that the dashboard shows comes from this service. The dashboard holds no business rule and reads no file of its own, so the same report gives the same answer whether it is reached through a browser, a script or the marking suite. Every refusal the service makes itself is answered with its status and a JSON body whose **detail** is the message; a request refused with **422** for its shape keeps FastAPI's own body, whose **detail** lists what is wrong.
MODULE: api/schemas.py
The shape of every request the service accepts and every reply it returns, declared once. A request that does not meet the conditions stated is refused with status **422** before any work is done, and a reply carries exactly the fields listed. Every dictionary a reply carries holds only plain Python values, never a pandas or NumPy one. Field names are read by the dashboard and by the marking suite and must be reproduced exactly.
MODULE-LEVEL TYPES

- **DefectTypeFilter** — **typing.Literal**. The values the queue may be filtered by, from **config.QUEUE_DEFECT_TYPE_OPTIONS**: **config.QUEUE_DEFECT_TYPE_ALL**, or one of the five defect types.
- **ReviewStatusValue** — **typing.Literal**. The statuses a review may take, from **config.REVIEW_STATUS_OPTIONS**.
- **DecisionValue** — **typing.Literal**. The decisions of **config.DECISIONS**.
- **CoordinatorRequestText** — **typing.Annotated[str, ...]**. Text with surrounding whitespace removed, from **config.COORDINATOR_REQUEST_MINIMUM_CHARACTERS** to **config.COORDINATOR_REQUEST_MAXIMUM_CHARACTERS** characters.
- **DecisionReasonText** — **typing.Annotated[str, ...]**. The same, from **config.DECISION_REASON_MINIMUM_CHARACTERS** to **config.DECISION_REASON_MAXIMUM_CHARACTERS** characters.
- **QuestionText** — **typing.Annotated[str, ...]**. The same, from **config.CHAT_QUESTION_MINIMUM_CHARACTERS** to **config.CHAT_QUESTION_MAXIMUM_CHARACTERS** characters.

REQUEST SHAPE
`SignInRequest`
FIELDS

Field	Type	What it carries
username	**str**	The account signing in
password	**str**	The password as typed

VALIDATION

- Both fields are required.

RESPONSE SHAPE
`SignedInUserResponse`
FIELDS

Field	Type	What it carries
username	**str**	The account that signed in
display_name	**str**	The person's name as the dashboard shows it
role	**str**	The account's role
district	**str | None**	The district a Highways Inspector is appointed for, and **null** for the other roles
session_token	**str**	The token every later request carries in **config.SESSION_TOKEN_HEADER**
pages	**list[str]**	The pages the role is offered, from **config.ROLE_PAGE_ACCESS**
permissions	**list[str]**	The permissions the role holds, from **config.ROLE_PERMISSIONS**

VALIDATION
None beyond the field types above.

RESPONSE SHAPE
`SignedOutResponse`
FIELDS

Field	Type	What it carries
username	**str**	The account whose session ended
signed_out	**bool**	That the session is over

VALIDATION
None beyond the field types above.

RESPONSE SHAPE

QueueRow

FIELDS

Field	Type	What it carries
queue_position	**int**	The report's place in the queue, counting from **1**
report_reference	**str**	The report
revision_number	**int**	Its latest revision
defect_type	**str**	Its defect type
linked_defect_type	**str**	Its linked defect type, or **config.NO_LINKED_DEFECT**
street_name	**str**	Its street
district	**str**	Its district, in the words **describe_record** gives
report_channel	**str**	Its channel, in the words **describe_record** gives
report_status	**str**	**config.REPORT_STATUS_AWAITING_REVIEW** until that revision is reviewed, and then its review's status
document_count	**int**	How many documents are attached to it
requirements_met_count	**int**	Requirements Met
requirements_not_met_count	**int**	Requirements Not met
requirements_not_documented_count	**int**	Requirements Not documented
live_permit_count	**int**	How many permits are live on its street
conflicting_permit_count	**int**	How many of those conflict with its repair
coordination_trigger_count	**int**	How many coordination triggers apply
assessment	**str**	The assessment
repeat_likelihood	**float**	The repeat likelihood
likelihood_band	**str**	Its band
expected_crew_hours	**float**	The expected crew hours
overrun_expected	**bool | None**	Whether an overrun is expected, **null** with no planned crew hours
disagreement_count	**int**	How many document sentences were flagged

VALIDATION

None beyond the field types above.

RESPONSE SHAPE

ReportDetailResponse

FIELDS

Field	Type	What it carries
queue_position	**int**	The report's place in the queue
report_status	**str**	As for **QueueRow**
report	**dict**	Every column of the report, its counts of live and conflicting permits included, as **describe_record** words it
report_documents	**list[dict]**	Every document attached, each as **describe_record** words it
screening	**dict**	The screening as **screen_report** gave it
register_count	**dict**	The street works as **count_live_street_works** gave them
evidence_findings	**list[dict]**	Every sentence read, with the requirements it bears on
disagreements	**list[dict]**	Every flagged sentence
likelihood	**dict**	The whole result of **estimate_repeat_likelihood**
crew_hours	**dict**	The whole result of **estimate_expected_crew_hours**

VALIDATION

None beyond the field types above.

REQUEST SHAPE

ReviewRequest

FIELDS

Field	Type	What it carries
report_reference	**str**	The report to review
coordinator_request	**CoordinatorRequestText**	What the person opening the review wants looked at

VALIDATION

- **coordinator_request** meets **CoordinatorRequestText**.

REQUEST SHAPE

`DecisionRequest`

FIELDS

Field	Type	What it carries
decision	**DecisionValue**	One of **config.DECISIONS**
decision_reason	**DecisionReasonText**	The Highways Inspector's reason, in their own words

VALIDATION

- **decision** is one of **config.DECISIONS**, and **decision_reason** meets **DecisionReasonText**.

RESPONSE SHAPE

`ReviewRecordResponse`

FIELDS

Field	Type	What it carries
review_id	**str**	The review
run_id	**str**	The run that produced it
thread_id	**str**	The thread its run is checkpointed under
coordinator_id	**str**	Who opened the review
report_reference	**str**	The report
revision_number	**int**	The revision reviewed
defect_type	**str**	Its defect type
linked_defect_type	**str**	Its linked defect type
street_name	**str**	Its street
district	**str | None**	Its district
coordinator_request	**str**	What the review was opened to look at
requirements	**list[dict]**	Every requirement with its result
requirements_met_count	**int**	Requirements Met
requirements_not_met_count	**int**	Requirements Not met
requirements_not_documented_count	**int**	Requirements Not documented
live_permits	**list[str]**	The permits live on the street
conflicting_permits	**list[str]**	Those that conflict with the repair
coordination_triggers	**list[str]**	The coordination triggers that apply
assessment	**str**	The assessment
decisions_open	**list[str]**	The decisions open
evidence	**list[dict]**	Every finding with the requirements it bears on
disagreements	**list[dict]**	Every flagged sentence
repeat_likelihood	**float**	The likelihood
likelihood_band	**str**	Its band
expected_crew_hours	**float**	The expected crew hours
planned_crew_hours	**float | None**	The planned crew hours
overrun_expected	**bool | None**	Whether an overrun is expected
crew_hours_text	**str**	The expected crew hours in words
review_plan_text	**str**	The planner's write-up
requirements_analysis_text	**str**	The requirements analyst's write-up
evidence_review_text	**str**	The evidence analyst's write-up
prediction_analysis_text	**str**	The prediction analyst's write-up
recommendation_text	**str**	The writer's recommendation
recommended_decision	**str**	One of the decisions open
status	**str**	One of **config.REVIEW_STATUS_OPTIONS**
order_document_text	**str | None**	The works order or inspection request, once written
document_details	**dict | None**	That document's details, once written
resident_update_text	**str | None**	The resident update, once written
coordination_brief	**dict | None**	The coordination brief, for a referral
llm_call_count	**int**	The run's call count, as **agent_runs** holds it
decision	**str | None**	The decision, once taken
decided_by	**str | None**	Who took it
decided_by_role	**str | None**	Their role
decided_by_district	**str | None**	The district they are appointed for
decision_reason	**str | None**	Their reason

VALIDATION

None beyond the field types above.

RESPONSE SHAPE

`ReviewCallsResponse`

FIELDS

Field	Type	What it carries
review_id	**str**	The review
run_id	**str**	Its run
call_count	**int**	The calls recorded against the run
expected_call_count	**int**	The calls the workflow allows at this point
total_tokens	**int**	The calls' total tokens added together
total_latency_milliseconds	**int**	The calls' latencies added together
calls	**list[dict]**	Every call recorded, as **fetch_llm_calls** returns them

VALIDATION

None beyond the field types above.

REQUEST SHAPE

`ChatRequest`

FIELDS

Field	Type	What it carries
question	**QuestionText**	The question

VALIDATION

- **question** meets **QuestionText**.

RESPONSE SHAPE

`ChatResponse`

FIELDS

Field	Type	What it carries
question_type	**str**	One of **config.CHAT_TYPE_OPTIONS**
report_reference	**str | None**	The report the question is about, if any
street_name	**str | None**	The street a figure question is about, if any
defect_type	**str | None**	The defect type a figure question is about, if any
answer	**str**	The answer, or the refusal
source_documents	**list[str]**	The documents the answer rested on
was_refused	**bool**	Whether it was a refusal

VALIDATION

None beyond the field types above.

RESPONSE SHAPE

`ChatTurnResponse`

FIELDS

Field	Type	What it carries
turn_id	**int**	The turn
username	**str**	Who asked
user_role	**str**	Their role at the time
question_text	**str**	What they asked
question_type	**str**	How it was classified
report_reference	**str | None**	The report it was about, if any
street_name	**str | None**	The street a figure question was about, if any
defect_type	**str | None**	The defect type a figure question was about, if any
answer_text	**str**	What they were told
source_documents	**str**	The documents, written as text
closest_distance	**float**	How far the nearest passage was
was_refused	**bool**	Whether it was a refusal

VALIDATION

None beyond the field types above.

REQUIREMENTS

- Only a field that may legitimately be empty is declared as able to be empty.

MODULE: api/routes.py

PURPOSE

Expose the queue, the report detail, the review workflow, the decision, the model calls, the chat and the audit trail over HTTP, behind one sign-in and permission guard and one failure handler.

MODULE-LEVEL OBJECTS

- **review_workflow** — **langgraph.graph.state.CompiledStateGraph**. The workflow, built once by **build_workflow** from **config.CHECKPOINT_DATABASE_FILE**.
- **report_history** — **pandas.DataFrame**. The reports already decided, loaded once by **load_report_history**, which the chat counts street figures from.

- **report_profiles** — **dict[str, dict]**. Every queued report's profile, with its counts of live and conflicting permits, keyed by its report reference, as **gather_report_context** gave it.
- **report_details** — **dict[str, dict]**. Every queued report's detail, keyed by its report reference, built once with **gather_report_context**, in the shape of **ReportDetailResponse** without **report_status**.
- **report_queue** — **list[dict]**. One queue row per report, in the shape of **QueueRow** without **report_status**, in the order **rank_report_queue** gives, built at the same time.
- **report_locks** — **dict[str, threading.Lock]**. One lock per queued report, keyed by its report reference, so that no report is reviewed twice at once and no review is decided twice at once.
- **public_router** — **fastapi.APIRouter**. Signing in and signing out, reachable without a token.
- **guarded_router** — **fastapi.APIRouter**. Every other endpoint, included behind **authorize_request** so that the guard cannot be forgotten on a new endpoint.
- **SessionTokenHeader** — a **typing.Annotated** alias for **str | None**. The header **config.SESSION_TOKEN_HEADER**, which a request may leave out.
- **SignedInUser** — a **typing.Annotated** alias for **dict**. The parameter type that receives the account **authorize_request** returned.
- **service_application** — **fastapi.FastAPI**. The application, titled **config.API_TITLE**, holding both routers and **report_failed_request** as its failure handler.

REQUIREMENTS

- The models, the live files, the register and the knowledge base are not loaded again here: the service uses those **agentic/nodes.py** holds.
- The queue and every detail are worked out once, when the module is imported, from the same tools the workflow uses, so the queue, the detail and the stored review agree; no language model is called.
- Every endpoint answers **200** when it succeeds, except **POST /reviews**, which answers **201**.
- No endpoint carries a default parameter value except **session_token** of **sign_out_user**.
- Only **POST /reviews**, **POST /reviews/{review_id}/decision** and **POST /chat/ask** may cost language model calls.

sign_in_user
ENDPOINT
POST /auth/signin
FUNCTION SIGNATURE
`sign_in_user(sign_in_request)`

PARAMETERS
sign_in_request — **SignInRequest**. The username and password, read from the request body.
PURPOSE
Exchange a username and password for a session token.
REQUIREMENTS

- An unknown username and a wrong password are answered identically, with **401** and **config.SIGN_IN_FAILED_MESSAGE**.
- The password is checked with **hash_password** and the account's own salt, compared in constant time.
- A correct password issues a fresh random, URL-safe token made from **config.SESSION_TOKEN_BYTES** random bytes and stores it against the account, replacing any token it held, so an account holds one session.

RETURNS
dict. A **SignedInUserResponse**.

sign_out_user
ENDPOINT
POST /auth/signout
FUNCTION SIGNATURE
`sign_out_user(session_token=None)`

PARAMETERS
session_token — **str | None**. The token to end, declared as **SessionTokenHeader** and read from the header **config.SESSION_TOKEN_HEADER**, **None** when the header is absent.
PURPOSE
End a session.
REQUIREMENTS

- A request with no token, or a token no account holds, is answered **401** with **config.SIGN_IN_REQUIRED_MESSAGE**.
- The account's token is cleared, so the same token opens nothing afterwards.

RETURNS
dict. A **SignedOutResponse**, with **signed_out True**.

list_report_queue
ENDPOINT
GET /queue
FUNCTION SIGNATURE
`list_report_queue(defect_type)`

PARAMETERS
defect_type — **str**. One of **config.QUEUE_DEFECT_TYPE_OPTIONS**, declared as **DefectTypeFilter**, required, taken from the query string.
PURPOSE

Give the Street Care Office its ranked list of open reports.
REQUIREMENTS
- **config.QUEUE_DEFECT_TYPE_ALL** gives every row; a defect type gives exactly the rows whose defect type or linked defect type it is, in the same order and with the same positions.
- Each row's **report_status** is the stored status of the review of that report's latest revision, or **config.REPORT_STATUS_AWAITING_REVIEW** when it has none.

RETURNS
list[dict]. One **QueueRow** per report.

get_report_detail
ENDPOINT
GET /queue/{report_reference}
FUNCTION SIGNATURE
`get_report_detail(report_reference)`
PARAMETERS
report_reference — **str**. The report, taken from the path.
PURPOSE
Show one report with everything the review would work from, before any language model call is spent on it.
REQUIREMENTS
- A report not in the queue is answered **404** with **config.UNKNOWN_REPORT_MESSAGE** filled with the reference.
- **report_status** is worked out as for the queue.

RETURNS
dict. A **ReportDetailResponse**.

start_report_review
ENDPOINT
POST /reviews
FUNCTION SIGNATURE
`start_report_review(review_request, signed_in_user, http_request)`
PARAMETERS
review_request — **ReviewRequest**. The report and what is to be looked at, read from the request body.
signed_in_user — **dict**, declared as **SignedInUser**. The account of whoever is opening the review, as **authorize_request** returned it, supplied by FastAPI.
http_request — **fastapi.Request**. The HTTP request itself, supplied by FastAPI, through which **report_failed_request** learns the thread of a run that fails.
PURPOSE
Run the five reviewing agents on one report and leave it waiting for a Highways Inspector.
REQUIREMENTS
- A report not in the queue is answered **404** with **config.UNKNOWN_REPORT_MESSAGE** filled with the reference, and a Highways Inspector outside the report's district is refused by **authorize_district**, both before any run starts.
- Under the report's lock, a report whose latest revision already has a stored review is answered **409** with **config.REPORT_ALREADY_REVIEWED_MESSAGE**, filled with that revision number and the reference, before the workflow starts, so no run is opened.
- The run's thread is the reference, a hyphen and **config.IDENTIFIER_HEX_LENGTH** random upper-case hexadecimal characters, kept on the HTTP request's state as **opened_thread_id** so that **report_failed_request** can close a run that fails.
- The workflow runs to the gate with the signed-in username as **coordinator_id**.

RETURNS
dict. A **ReviewRecordResponse**, the stored review, with status **201**.

list_report_reviews
ENDPOINT
GET /reviews
FUNCTION SIGNATURE
`list_report_reviews(status_value)`
PARAMETERS
status_value — **list[ReviewStatusValue]**. Required, taken from the query string, each status given as its own **status_value**.
PURPOSE
List reviews by status, for the decision queue, the report detail and the audit trail.
REQUIREMENTS
- Every review whose status is one of those asked for is returned once, in report-reference order.

RETURNS
list[dict]. One **ReviewRecordResponse** per review.

submit_review_decision
ENDPOINT
POST /reviews/{review_id}/decision
FUNCTION SIGNATURE
`submit_review_decision(review_id, decision_request, signed_in_user)`

PARAMETERS
review_id — **str**. The review, taken from the path.
decision_request — **DecisionRequest**. The decision and the reason, read from the request body.
signed_in_user — **dict**, declared as **SignedInUser**. The account of the Highways Inspector deciding, as **authorize_request** returned it, supplied by FastAPI.
PURPOSE
Record a Highways Inspector's decision on a paused review and finish its run.
REQUIREMENTS

- An unknown review is answered **404** with **config.REVIEW_NOT_FOUND_MESSAGE**.
- A review whose report is not in the queue is refused with **config.REVIEW_OUTSIDE_QUEUE_MESSAGE**, filled with the review identifier and the reference, answered **502** by **report_failed_request**.
- A Highways Inspector outside the report's district is refused by **authorize_district**.
- Under the report's lock, a review whose checkpoint holds nothing is refused with **config.CHECKPOINT_MISSING_MESSAGE**, filled with the review identifier and **config.CHECKPOINT_DATABASE_FILE**, answered **502** by **report_failed_request**.
- The decision carries the signed-in username as **decided_by**, their role as **decided_by_role** and their district as **decided_by_district**.
- Under the same lock, a conflict from **find_review_conflict** (the review's status and the decision the checkpoint holds) takes precedence over a refusal from **find_decision_refusal** (the checkpoint's report, screening and opener).
- Either is answered **409** with its reason, before the run resumes and before any call is spent.
- The run resumes from its checkpoint with the decision; the same decision sent again after a failure finishes it.

RETURNS
dict. A **ReviewRecordResponse**, the decided review.

get_review_calls
ENDPOINT
GET /reviews/{review_id}/calls
FUNCTION SIGNATURE
`get_review_calls(review_id)`
PARAMETERS
review_id — **str**. The review, taken from the path.
PURPOSE
Show the language model calls recorded for the review's run, against the number the workflow allows.
REQUIREMENTS

- An unknown review is answered **404** with **config.REVIEW_NOT_FOUND_MESSAGE**.
- **expected_call_count** is the figure **config.LLM_CALLS_BY_DECISION** gives for the decision recorded, or for the decision the checkpoint holds while it is being recorded, and **config.LLM_CALLS_BEFORE_DECISION** otherwise.
- An undecided review whose checkpoint holds nothing is refused with **config.CHECKPOINT_MISSING_MESSAGE**, filled with the review identifier and **config.CHECKPOINT_DATABASE_FILE**, answered **502** by **report_failed_request**.
- **total_tokens** and **total_latency_milliseconds** are the sums over the calls.

RETURNS
dict. A **ReviewCallsResponse**.

ask_street_care_assistant
ENDPOINT
POST /chat/ask
FUNCTION SIGNATURE
`ask_street_care_assistant(chat_request, signed_in_user)`
PARAMETERS
chat_request — **ChatRequest**. The question, read from the request body.
signed_in_user — **dict**, declared as **SignedInUser**. The account of whoever is asking, as **authorize_request** returned it, supplied by FastAPI.
PURPOSE
Answer one question from the documents or the records, or refuse, and record the exchange.
REQUIREMENTS

- The asker's own conversation is recalled with **recall_conversation**, and the question is answered by **answer_from_documents** in exactly one call.
- A question of **config.CHAT_REPORT_TYPES** has its answer replaced by **answer_report_record**, with the asker's account, the open reports and the report the reply names.
- A question of **config.CHAT_TYPE_STREET_FIGURE** has its answer replaced by **answer_street_figure**, with **report_history** and the street and defect type the reply names.
- Every question the chat replies to, from the documents or the records or with a refusal, is recorded, with the documents joined by a comma and a space; a request that fails records nothing.

RETURNS
dict. A **ChatResponse**.

list_own_chat_turns
ENDPOINT

GET /chat/turns
FUNCTION SIGNATURE
`list_own_chat_turns(signed_in_user, turn_limit)`
PARAMETERS
signed_in_user — **dict**, declared as **SignedInUser**. The account of whoever is asking, as **authorize_request** returned it, supplied by FastAPI.
turn_limit — **int**. Required, taken from the query string, from **1** to **config.CHAT_AUDIT_MAX_ROWS**.
PURPOSE
Give a person their own conversation back.
REQUIREMENTS
- Only the signed-in person's own turns are returned, newest first, at most **turn_limit** of them.

RETURNS
list[dict]. One **ChatTurnResponse** per turn.

list_recent_chat_turns
ENDPOINT
GET /audit/chat-turns
FUNCTION SIGNATURE
`list_recent_chat_turns(turn_limit)`
PARAMETERS
turn_limit — **int**. Required, taken from the query string, from **1** to **config.CHAT_AUDIT_MAX_ROWS**.
PURPOSE
Give the audit trail every recent question, whoever asked it.
REQUIREMENTS
- Turns from every person are returned, newest first, at most **turn_limit** of them.

RETURNS
list[dict]. One **ChatTurnResponse** per turn.

report_failed_request
FUNCTION SIGNATURE
`report_failed_request(http_request, failure)`
PARAMETERS
http_request — **fastapi.Request**. The HTTP request that failed, supplied by FastAPI to the failure handler.
failure — **Exception**. The failure raised while the request was served, supplied by FastAPI: a **ValueError** from any part of the system, a **google.genai.errors.APIError** or an **httpx.HTTPError**.
PURPOSE
Turn every failure into one reply the dashboard can show, and never leave a failed run looking as though it is still running.
REQUIREMENTS
- The reply has status **502** and a **detail** that is the failure's own message.
- Where the failing request opened a run, that run is recorded as **config.AGENT_RUN_STATUS_FAILED** with the number of calls recorded against it.

RETURNS
fastapi.responses.JSONResponse. The failure reply.

========== ACCESS CONTROL ==========

Everyone signs in, and the service, not the dashboard, decides what each role may reach and which streets a Highways Inspector may act on.

MODULE: access/password_security.py

FUNCTION SIGNATURE
`hash_password(plain_password, password_salt)`
PARAMETERS
plain_password — **str**. The password as typed, passed by **sign_in_user**, or by **run_database_setup.py** from **config.SEED_USER_ACCOUNTS**.
password_salt — **str**. The account's own salt, read by **sign_in_user** from **user_accounts**, or made by **run_database_setup.py** from **config.PASSWORD_SALT_BYTES**.
PURPOSE
Turn a password into the form the database stores, so that the database never holds a password.
REQUIREMENTS
- The stored form is the PBKDF2-HMAC derivation with **config.PASSWORD_HASH_ALGORITHM** over **config.PASSWORD_HASH_ROUNDS** rounds of the password and **password_salt**, both as UTF-8 text, written as hexadecimal text.
- The same password and salt always give the same result, and a different salt gives a different result.

RETURNS
str. The stored form.

MODULE: access/permissions.py

FUNCTION SIGNATURE
`authorize_request(request, session_token=None)`

PARAMETERS
request — **fastapi.Request**. The incoming request, supplied by FastAPI, read for its method and its route's path template.
session_token — **str | None**. The token sent under the header **config.SESSION_TOKEN_HEADER**, taken by FastAPI from the request, **None** when the header is absent.
PURPOSE
Decide, for every guarded request, who is asking and whether their role holds the permission that request needs.
REQUIREMENTS

- A request with no token, or a token no account holds, is refused **401** with **config.SIGN_IN_REQUIRED_MESSAGE**.
- The permission is the one **config.PERMISSION_BY_ROUTE** gives for the method and the route's path template, for example **/reviews/{review_id}/decision**.
- A role whose **config.ROLE_PERMISSIONS** does not hold that permission is refused **403** with **config.PERMISSION_REFUSED_MESSAGE**, naming the role and the permission, before the endpoint runs and whether or not what it asks for exists.

RETURNS
dict. The signed-in person's account record.

FUNCTION SIGNATURE
`authorize_district(signed_in_user, street_name, report_reference)`

PARAMETERS
signed_in_user — **dict**. The account **authorize_request** returned, passed by **start_report_review** and **submit_review_decision**.
street_name — **str**. The report's street, passed by **start_report_review** from the report's profile, or by **submit_review_decision** from the stored review.
report_reference — **str**. The report being acted on, passed by the same endpoints for the refusal.
PURPOSE
Hold a Highways Inspector to the streets of their own district.
REQUIREMENTS

- A **config.ROLE_HIGHWAYS_INSPECTOR** whose district's streets in **config.INSPECTION_DISTRICT_STREETS** do not include **street_name** is refused **403** with **config.DISTRICT_REFUSED_MESSAGE**, naming the username, the district, the street and the report.
- Any other role belongs to no district and is not held to one.
- It is called before a review run starts and before a decision is looked at, so a refusal costs no language model call.

RETURNS
None.

========== THE STREET CARE CHAT ==========
One question, one language model call, answered from the documents and from nothing else. The assistant classifies what it was asked, answers the kinds it may answer, and refuses the rest. It knows nothing of any single report or street; the service answers questions about a report from the review records, within what the asker's role allows, and questions about a street from the report history, within the figure line.

Kind	What it covers	Answered from
config.CHAT_TYPE_DEFECT_STANDARD	What one defect standard sets out, from its scope and requirements to the works order and the resident update	The standards and the passages found
config.CHAT_TYPE_STREET_CARE_RULES	How reports are read and decided: roles, districts, triggers, decisions, documents and records	The passages found
config.CHAT_TYPE_STATEMENT_GUIDANCE	What a statement guide says about one type of sentence	The guides and the passages found
config.CHAT_TYPE_REPORT_STATUS	Where one named report stands	The review records, by **answer_report_record**
config.CHAT_TYPE_OUTCOME_PREDICTION	Any forecast about one named report	The review records, by **answer_report_record**
config.CHAT_TYPE_STREET_FIGURE	A count of one defect type's reports on a street	The report history, by **answer_street_figure**
config.CHAT_TYPE_OUT_OF_SCOPE	A question the documents cannot answer	Nothing: it is refused

MODULE: chat/conversation_memory.py

FUNCTION SIGNATURE

```
recall_conversation(session, username, question_text)
```

PARAMETERS

session — **sqlalchemy.orm.Session**. An open session, opened by **ask_street_care_assistant**.

username — **str**. Whose conversation to recall, the signed-in person's **username**, passed by **ask_street_care_assistant**.

question_text — **str**. The question just asked, from the request body, passed by the same endpoint.

PURPOSE

Give the assistant the last few exchanges of this person's own conversation, as history to read and as words to search with.

REQUIREMENTS

- At most **config.CHAT_HISTORY_TURN_LIMIT** of that person's own most recent turns are recalled, and nobody else's.
- **history_text** shows them oldest first, each question with its stored answer as written; with no earlier turn it is **config.CHAT_FIRST_QUESTION_TEXT**.
- An answer to a question of **config.CHAT_RECORD_TYPES** appears as **config.CHAT_RECORD_ANSWER_IN_HISTORY**, so nothing from the records reaches a prompt.
- **search_text** is the question asked just before, where there is one, together with the new question.
- **previous_question** is the question asked just before, or empty text; **previous_street** is the street that turn was about, or **None**.

RETURNS

dict. **history_text**, **search_text**, **previous_question** and **previous_street**.

MODULE: chat/street_care_assistant.py

FUNCTION SIGNATURE

```
answer_from_documents(collection, document_entries, question_text, conversation)
```

PARAMETERS

collection — **chromadb.Collection**. The knowledge base, **knowledge_collection** of **agentic/nodes.py**, passed by **ask_street_care_assistant**.

document_entries — **list[dict]**. The sixteen documents' text, **knowledge_documents** of **agentic/nodes.py**, passed by the same endpoint.

question_text — **str**. The question as asked, from the request body, passed by **ask_street_care_assistant**.

conversation — **dict**. **history_text**, **search_text**, **previous_question** and **previous_street** as **recall_conversation** returned them, passed by the same endpoint.

PURPOSE

Settle what kind of question was asked and answer it from the documents in one language model call, refusing when they do not ground an answer.

REQUIREMENTS

- Two searches with **search_documents**, one for the question and one for **search_text**, are made together; their passages are offered once each, nearest first.
- The prompt carries those passages, the question, **history_text**, each defect type's requirements and what an inspection records, from its own standard, and every statement guide's reviewer checks.
- The prompt names the seven kinds of **config.CHAT_TYPE_OPTIONS** and keeps the model to what it is given and to each requirement's own standard.
- It reads a question that leaves out its subject as carrying on from the last exchange, and leaves questions about a single report or street to the records.
- The call is made as **config.AGENT_NAME_STREET_CARE_ASSISTANT**; the reply carries **QUESTION TYPE**, **REPORT**, **DEFECT TYPE** and **ANSWER**, with **config.CHAT_NO_VALUE_REPLY** in a field that does not apply.
- Each value is read ignoring case and the edge decoration **config.RESPONSE_DECORATION_EDGE_PATTERN** finds.
- A question of **config.CHAT_ANSWERED_TYPES** that names a report reference, as **config.REPORT_REFERENCE_IN_TEXT_PATTERN** finds one, becomes **config.CHAT_TYPE_REPORT_STATUS**.
- A reply is refused, naming the agent, when any of its first three fields runs over more than one line, its type is outside **config.CHAT_TYPE_OPTIONS** or its **REPORT** is empty.
- It is also refused when its **ANSWER** is **config.CHAT_NO_VALUE_REPLY** for a question still of **config.CHAT_ANSWERED_TYPES**.
- It is also refused when the report reference settled below is named by neither the question nor **history_text**, in full or, for a canonical one, by its number as **config.NUMBER_IN_TEXT_PATTERN** finds it.
- Three fields are settled for the question types below, and are **None** for any other type:

Field	Question type	Settled as
report_reference	**config.CHAT_REPORT_TYPES**	The first reference the question names, or else the model's **REPORT**, and **None** when that is **config.CHAT_NO_VALUE_REPLY**; made canonical when **config.REPORT_REFERENCE_PATTERN** reads it in upper case, and otherwise kept as written
street_name	**config.CHAT_TYPE_STREET_FIGURE**	The one street of **config.INSPECTION_DISTRICT_STREETS** named as a whole word, as **config.NAME_IN_TEXT_PATTERN** finds it in the question in small letters, or else in **previous_question**, or else **previous_street**; **None** unless exactly one street is found
defect_type	**config.CHAT_TYPE_STREET_FIGURE**	The model's **DEFECT TYPE** when it is one of the five defect types, written as the defect type, and **None** otherwise

- A question is answered when its type is one of **config.CHAT_ANSWERED_TYPES** and the nearest passage is within **config.CHAT_MAX_RELEVANCE_DISTANCE**; its answer is the model's **ANSWER**.
- Any other question, except one of **config.CHAT_RECORD_TYPES**, is a refusal: **was_refused** is **True**, its type becomes **config.CHAT_TYPE_OUT_OF_SCOPE** and its answer **config.CHAT_REFUSAL_MESSAGE**.
- **source_documents** names, once each, every standard for a **config.CHAT_TYPE_DEFECT_STANDARD** answer, every guide for a **config.CHAT_TYPE_STATEMENT_GUIDANCE** answer, and the documents of the passages found.
- **source_documents** is empty unless the question was answered; **closest_distance** is the nearest passage's distance.

RETURNS

dict. **question_type**, **report_reference**, **street_name**, **defect_type**, **answer**, **source_documents**, **closest_distance** and **was_refused**.

MODULE: chat/record_answer.py

Answers from the records, never from a model, so that a model cannot give away what the rules withhold.

FUNCTION SIGNATURE

```
answer_report_record(session, signed_in_user, open_reports, question_type, report_reference)
```

PARAMETERS

session — **sqlalchemy.orm.Session**. An open session, opened by **ask_street_care_assistant**.
signed_in_user — **dict**. The asker's account, with **username** and **role**, as **authorize_request** returned it, passed by **ask_street_care_assistant**.
open_reports — **pandas.DataFrame**. The live queue, **open_reports** of **agentic/nodes.py**, passed by **ask_street_care_assistant**.
question_type — **str**. One of **config.CHAT_REPORT_TYPES**, as **answer_from_documents** classified the question, passed by the same endpoint.
report_reference — **str | None**. The report the question is about, as **answer_from_documents** returned it, passed by the same endpoint.

PURPOSE

Answer a question about one report from the review records, in the way the asker's role allows.

REQUIREMENTS

- A Customer Services Adviser asking a question of **config.CHAT_TYPE_OUTCOME_PREDICTION** hears **config.OUTCOME_NOT_PREDICTED_LINE**, before anything else is looked at; no language model is called.
- Any role hears **config.CHAT_REPORT_NOT_NAMED_MESSAGE** when no report is named, and **config.CHAT_NO_REPORT_TEMPLATE** for a reference that is not an open report.
- Only a review of the report's current revision counts. Its defect types are its defect type as **describe_record** words it or, when it has a linked defect type, **config.CHAT_LINKED_DEFECT_TEMPLATE**.
- A Customer Services Adviser hears **config.CHAT_ADVISER_STATUS_TEMPLATE**, with the phrase **config.REPORT_STATUS_PHRASES** gives for the stored review's status, or for **config.REPORT_STATUS_AWAITING_REVIEW** when none is stored.
- Only a repair ordered, **config.REVIEW_STATUS_REPAIR_ORDERED**, adds for the adviser, after a space, **config.CHAT_ORDERED_REPAIR_TEMPLATE**.
- It is filled with the location of the review's **document_details** as **describe_record** words it, and the stored resident update's **UPDATE** field with every run of whitespace as one space.
- A Highways Inspector or a Street Care Coordinator hears **config.CHAT_NO_REVIEW_TEMPLATE** when no review is stored, and otherwise the six lines below, filled from the review and the defect types, in this order.

Line	Filled with
config.CHAT_REVIEW_TEMPLATE	The review, with its status from **config.REPORT_STATUS_PHRASES**
config.CHAT_SCREENING_TEMPLATE	Each standard's requirements written as the defect type, **standard:** and each requirement's name and result
config.CHAT_PERMITS_TEMPLATE	The live permits, each conflicting one written as **config.CONFLICTING_PERMIT_TEMPLATE**
config.CHAT_FIGURES_TEMPLATE	The likelihood to **config.LIKELIHOOD_DECIMALS** places, its band and the crew hours text
config.CHAT_EVIDENCE_TEMPLATE	The number of sentences read, those sentences counted by type, each as the type and its count in name order, and the number of flags
config.CHAT_AWAITING_DECISION_TEMPLATE or **config.CHAT_DECIDED_TEMPLATE**	The recommended decision while no decision is recorded, and otherwise the decision

- In those lines items are joined by a comma and a space, triggers and standards by a semicolon and a space, and an empty list reads **config.CHAT_NONE_TEXT**, or **config.CHAT_NO_DOCUMENTS_TEXT** for the sentences.

RETURNS
str. The answer, its lines joined by line breaks.

FUNCTION SIGNATURE
`answer_street_figure(report_history, street_name, defect_type)`

PARAMETERS
report_history — **pandas.DataFrame**. The reports already decided, **report_history** of **api/routes.py**, passed by **ask_street_care_assistant**.
street_name — **str | None**. The street the question is about, as **answer_from_documents** returned it, passed by **ask_street_care_assistant**.
defect_type — **str | None**. The defect type the question is about, as **answer_from_documents** returned it, passed by the same endpoint.
PURPOSE
Give a figure about a street only where it cannot point to the household that made a report.
REQUIREMENTS

- With either the street or the defect type missing, the answer is **config.CHAT_STREET_NOT_NAMED_MESSAGE**.
- The figure is the number of distinct **report_reference** values among the rows of **report_history** on that street whose **defect_type** is that defect type; a report only linked to the defect type does not count.
- A figure below **config.GROUP_SIZE_MINIMUM** is answered with **config.GROUP_SIZE_LINE**; any other with **config.CHAT_STREET_FIGURE_TEMPLATE**, filled with the figure, the defect type in small letters and the street.
- The answer is the same whoever asks, and no language model is called.

RETURNS
str. The answer.

========== THE DASHBOARD ==========
A sign-in page and five pages over the service, named by **config.PAGE_QUEUE**, **config.PAGE_DETAIL**, **config.PAGE_DECISION**, **config.PAGE_AUDIT** and **config.PAGE_CHAT**. The dashboard holds no business rule: every figure on every page was produced by the service. The only checks a page makes itself are the service's own input limits, a Highways Inspector's own district and the refusal of a decision on one's own review, repeated so that a form the service would refuse is never sent. Each page is a function that draws itself and returns nothing.
A decision, a status, a band, a question type, a requirement result and a sentence type are badged with **config.DECISION_BADGE_COLOURS**, **config.STATUS_BADGE_COLOURS**, **config.LIKELIHOOD_BAND_BADGE_COLOURS**, **config.CHAT_TYPE_BADGE_COLOURS**, **config.REQUIREMENT_RESULT_BADGE_COLOURS** and **config.SENTENCE_TYPE_BADGE_COLOUR**, an expected overrun with **config.OVERRUN_COLOUR**, and everything else with **config.NEUTRAL_BADGE_COLOUR**. The theme and the port are pre-loaded in **.streamlit/config.toml**.

MODULE: dashboard/api_client.py
Every request the dashboard sends to the service passes through this module, so no page holds an address, a timeout or a session token.

FUNCTION SIGNATURE
`call_service(method, path, query=None, payload=None, waiting_message=None, failure_message=None`

PARAMETERS
method — **str**. The HTTP method, for example **GET** or **POST**, written by the page making the call, or by **render_sign_in_page** and **run_dashboard.py** to sign in and out.
path — **str**. The path beneath **config.API_BASE_URL**, for example **/queue**, written by the same caller.
query — **dict | None**. Query parameters built by the calling page from config or from what the reader chose, **None** for an endpoint that takes none.

payload — **dict | None**. The request body built by the calling page from what the reader typed, **None** for an endpoint that takes none.
waiting_message — **str | None**. What the calling page tells the reader while a call that costs language model calls runs, **None** for a call that returns at once.
failure_message — **str | None**. The words the calling page shows a **502** reply in, **config.REVIEW_FAILED_MESSAGE**, **config.DECISION_FAILED_MESSAGE** or **config.CHAT_FAILED_MESSAGE**, **None** for a call that costs no language model call.
PURPOSE
Make one call to the service and give the page back the reply.
REQUIREMENTS

- The call goes to **config.API_BASE_URL** with **config.API_REQUEST_TIMEOUT_SECONDS**, and the signed-in person's token travels under **config.SESSION_TOKEN_HEADER** whenever there is a session.
- While the call runs, and only then, a spinner shows **waiting_message** with the time elapsed; with no message nothing is shown.
- A **401** while signed in ends the session in the browser and returns to signing in, where **config.SESSION_ENDED_MESSAGE** is shown.
- Any other reply of status **400** or more stops the page, shown as the service's own **detail**, or as the status and the text when the reply is not JSON.
- A **502** on a call given **failure_message** is shown in those words, with that text, less its closing full stop, as the reason.

RETURNS
dict | list. The service's reply read from JSON.

MODULE: dashboard/sign_in.py

FUNCTION SIGNATURE
`render_sign_in_page()`
PARAMETERS
None. The page reads what the reader types, and from the browser session whether a session has just ended.
PURPOSE
Let a person sign in.
REQUIREMENTS

- The page shows **config.DASHBOARD_TITLE**, **config.DASHBOARD_SUBTITLE** and a form for the username and the password, and, once after a session has ended, **config.SESSION_ENDED_MESSAGE**.
- Signing in calls the service once, with the username stripped of surrounding space, and keeps the reply in the browser session; a refused sign-in shows the service's own message.

RETURNS
None. The page draws itself.

MODULE: dashboard/report_queue.py

FUNCTION SIGNATURE
`render_report_queue_page()`
PARAMETERS
None. The page reads what it needs from the service through **call_service**, and what the reader has already chosen from the browser session.
PURPOSE
Show the open queue in the order the Street Care Office works it, and let a person pick one report.
REQUIREMENTS

- Counts along the top, from the whole queue, give the number of reports, of those awaiting review and of those awaiting a decision.
- They also count the reports with **config.QUEUE_FIRST_DEFECT_TYPE** as defect type or linked defect type, and those assessed each of **config.DECISIONS**.
- The reader filters by one of **config.QUEUE_DEFECT_TYPE_OPTIONS**, matched by defect type or linked defect type; the service does the filtering and the ordering.
- The table shows each report's position, reference, revision, defect type, linked defect type, street, district, channel, status labelled by **config.REPORT_STATUS_LABELS** and assessment.
- It also shows the three requirement counts, the live and conflicting permits, the coordination triggers and the numbers of flags and documents.
- It shows the likelihood to **config.LIKELIHOOD_DECIMALS** places with its band, the expected crew hours to **config.CREW_HOURS_DECIMALS** place, and whether an overrun is expected as **config.BOOLEAN_YES**, **config.BOOLEAN_NO** or **config.NOT_GIVEN_TEXT**.
- Selecting a row remembers that report, so that the detail page opens on it.

RETURNS
None. The page draws itself.

MODULE: dashboard/report_detail.py

FUNCTION SIGNATURE

`render_report_detail_page()`

PARAMETERS

None. The page reads what it needs from the service through **call_service**, and what the reader has already chosen from the browser session.

PURPOSE

Show everything established about one report, and let a person open its review.

REQUIREMENTS

- The report is chosen in queue order, labelled by its reference, defect types and street, opening on the one the queue page remembered.
- Everything shown comes from the report's detail and from the stored review of its current revision.
- A headline and counts show the assessment, status, queue position, district, channel, revision, the three requirement counts, the coordination triggers and the live and conflicting permits.
- They also show the likelihood with its band, and the expected against the planned crew hours, marked when an overrun is expected.
- A screening tab lists each standard's requirements with their results, sentences and any answers left blank, labelled by **config.FORM_ANSWER_LABELS**.
- It also lists the decisions open, the coordination triggers and the live permits, each conflicting one written as **config.CONFLICTING_PERMIT_TEMPLATE**.
- A statement evidence tab shows every sentence, document by document, with its type and the requirements it bears on; a report with no document says so.
- Each flagged sentence is marked with the figures the document states and the report's own.
- A report tab shows, once each and labelled by **config.FORM_ANSWER_LABELS** with what each is used for, the answers the works order details carry beyond the report's label (**config.REPORT_LABEL_COLUMNS**).
- It also shows the answers **config.REQUIREMENT_ANSWER_COLUMNS** gives the report's own requirements, the measurements **config.WORKS_ORDER_MEASUREMENT_COLUMNS** gives its defect types, and each trigger answer of **config.TRIGGER_FORM_ANSWERS** that applies to them.
- A review tab shows the stored review's recommendation and any decision, with who took it, their role and district and their reason.
- With no review, a Highways Inspector outside the report's district is told which district its street belongs to.
- Anyone else may start the review with what it should look at, of **config.COORDINATOR_REQUEST_MINIMUM_CHARACTERS** to **config.COORDINATOR_REQUEST_MAXIMUM_CHARACTERS** characters.
- Starting it is one call to the service, whose waiting message says **config.LLM_CALLS_BEFORE_DECISION** model calls are being made.

RETURNS

None. The page draws itself.

MODULE: dashboard/decision_queue.py

FUNCTION SIGNATURE

`render_decision_queue_page()`

PARAMETERS

None. The page reads what it needs from the service through **call_service**, and what the reader has already chosen from the browser session.

PURPOSE

Show the reviews of one district waiting at the gate, and let a Highways Inspector appointed for it decide one.

REQUIREMENTS

- The page names the signed-in Highways Inspector's district and its streets, and lists only reviews at **config.REVIEW_STATUS_AWAITING** on those streets, each by its report, defect types and street.
- With none it says so; a review that was open and has since been decided is reported.
- The chosen review shows its recommendation and opener, the likelihood with its band, the expected against the planned crew hours and the recommendation text.
- It also shows each standard's requirements with their results and any answers left blank, the coordination triggers, the live permits, each flagged sentence and, on request, what was asked and each agent's write-up.
- Where the signed-in person opened the review, the page says the other Highways Inspector appointed for the district must decide it, in place of the form.
- The form offers exactly the review's decisions open and requires a reason of **config.DECISION_REASON_MINIMUM_CHARACTERS** to **config.DECISION_REASON_MAXIMUM_CHARACTERS** characters; a decision sent but not recorded is filled in again.
- Recording calls the service once, with a waiting message, and reports the status reached, from **config.REPORT_STATUS_PHRASES**, with **config.DECISION_DESCRIPTIONS** for the decision.

RETURNS

None. The page draws itself.

MODULE: dashboard/audit_trail.py

FUNCTION SIGNATURE

```
render_audit_trail_page()
```

PARAMETERS

None. The page reads what it needs from the service through **call_service**, and what the reader has already chosen from the browser session.

PURPOSE

Show what has been reviewed and decided, the record behind each review, and every question the chat has replied to.

REQUIREMENTS

- Reviews are listed by the statuses the reader chooses from **config.REVIEW_STATUS_OPTIONS**, labelled by **config.REPORT_STATUS_LABELS**, with **config.DECIDED_REVIEW_STATUSES** chosen by default.
- Each is listed with its report, revision, defect type, linked defect type, street, status, who opened it, who decided it and the model calls.
- A selected review shows its assessment, recommended decision, the two figures, the decision reason with the decider's role and district, the recommendation and each agent's write-up.
- Where each exists, it also shows the works order or inspection request, titled by **config.DOCUMENT_KINDS**, and the resident update.
- A coordination brief is shown as a table of **config.COORDINATION_BRIEF_DETAIL_COLUMNS** beyond the report's label, labelled by **config.FORM_ANSWER_LABELS**, with the requirements, triggers, live permits and flagged statements.
- Its call budget shows the calls recorded against the calls expected, the total tokens, the total latency in seconds to **config.LATENCY_SECONDS_DECIMALS** place, and every call with what it cost.
- A second tab shows the latest **config.CHAT_AUDIT_MAX_ROWS** chat turns from everyone, newest first.
- Each turn shows who asked, their role, the type, the report, the street, the defect type, whether it was refused, the question, the answer and the sources.

RETURNS

None. The page draws itself.

MODULE: dashboard/street_care_chat.py

FUNCTION SIGNATURE

```
render_street_care_chat_page()
```

PARAMETERS

None. The page reads what it needs from the service through **call_service**, and what the reader has already chosen from the browser session.

PURPOSE

Let anyone signed in ask the street care assistant a question, and show the conversation.

REQUIREMENTS

- The conversation is the signed-in person's own turns, oldest at the top, at most **config.CHAT_PAGE_TURN_LIMIT** of them, each answer badged with its type and shown with its sources where it has any.
- A question shorter than **config.CHAT_QUESTION_MINIMUM_CHARACTERS** is not sent, and one longer than **config.CHAT_QUESTION_MAXIMUM_CHARACTERS** cannot be typed.
- Asking calls the service once, with a waiting message, and the conversation is shown again with the new turn.

RETURNS

None. The page draws itself.

========== THE SIX ENTRY POINTS ==========

Six files sit at the top of the project, and each is run from the **Project** folder. The first four build a stage of the system and are run once, in order; the last two start the service and the dashboard. Each prints its progress for the person running it; nothing in the marking suite reads console output.

MODULE: run_database_setup.py

Creates the database, its tables and the accounts people sign in with. It is safe to run again.

FUNCTION SIGNATURE

```
main()
```

PARAMETERS

None.

PURPOSE

Give the system a database to write to, and the staff the accounts to sign in with.

REQUIREMENTS

- **config.DATABASE_DIRECTORY** and every table exist when the run finishes, and a second run neither fails nor duplicates anything.
- A database holding a table of an earlier version, whose columns differ in name or in whether they may be empty, is refused, naming the table and saying that it and **config.CHECKPOINT_DATABASE_FILE** are to be deleted.

- Each account of **config.SEED_USER_ACCOUNTS** can sign in with the username, password, display name, role and district given there; an account already present is left exactly as it is.
- Each password is stored in the form **hash_password** gives, with its own salt of **config.PASSWORD_SALT_BYTES** random bytes written as hexadecimal text.
- The console names **config.STREETCARE_DATABASE_FILE**, the tables, each account as it is created, aligned by **config.USERNAME_WIDTH** and **config.DISPLAY_NAME_WIDTH**, and how many accounts were created and how many were already present.

RETURNS

None.

MAIN BLOCK

- Running the file performs the setup once; importing the module performs none of it.

MODULE: run_data_pipeline.py

Cleans the five raw extracts and writes the evidence that each one needed cleaning.

MODULE-LEVEL OBJECTS

- **DATASET_JOBS** — **list[tuple]**. The five cleaning jobs below, in this order, each a tuple of the raw file, the cleaned file, the cleaning function, the answer column and the detail profiler.

Raw file and cleaned file	Cleaning and detail profiler	Answer column
config.RAW_REPORT_HISTORY_TRAIN_FILE to **config.CLEAN_REPORT_HISTORY_TRAIN_FILE**	**clean_history_dataset**, **profile_history_details**	**config.HISTORY_TARGET_COLUMN**
config.RAW_REPORT_HISTORY_EVAL_FILE to **config.CLEAN_REPORT_HISTORY_EVAL_FILE**	**clean_history_dataset**, **profile_history_details**	**config.HISTORY_TARGET_COLUMN**
config.RAW_REPAIR_LOGS_TRAIN_FILE to **config.CLEAN_REPAIR_LOGS_TRAIN_FILE**	**clean_repair_log_dataset**, **profile_log_details**	**config.LOG_TARGET_COLUMN**
config.RAW_REPAIR_LOGS_EVAL_FILE to **config.CLEAN_REPAIR_LOGS_EVAL_FILE**	**clean_repair_log_dataset**, **profile_log_details**	**config.LOG_TARGET_COLUMN**
config.RAW_STATEMENT_SENTENCES_FILE to **config.CLEAN_STATEMENT_SENTENCES_FILE**	**clean_sentence_dataset**, **profile_sentence_details**	**config.SENTENCE_TARGET_COLUMN**

FUNCTION SIGNATURE

```
main()
```

PARAMETERS

None.

PURPOSE

Turn the five raw extracts into the cleaned tables the modelling stage reads, and produce the quality report.

REQUIREMENTS

- Each extract's profile is its general profile and its detail profile merged into one flat dictionary, filed under the raw file's name without its extension.
- Both profiles are worked out from the table **load_raw_dataset** reads and the table its cleaning returns.
- When any extract is refused, the run writes no cleaned file and no report.
- Each cleaned table is saved to its path, and the report to **config.DATA_QUALITY_REPORT_FILE**.
- One console line per extract, aligned by **config.DATASET_NAME_WIDTH** and **config.ROW_COUNT_WIDTH**, says how many rows were received, kept and removed, and a closing line names the report.

RETURNS

None.

MAIN BLOCK

- Running the file cleans all five extracts and writes the report; importing the module cleans nothing.

MODULE: run_model_training.py

Trains the three models in one run, and saves them only when every one reaches the minimum the configuration sets for it.

FUNCTION SIGNATURE

```
main()
```

PARAMETERS

None.

PURPOSE

Produce the three saved models, their schemas and their scores.

REQUIREMENTS

- The five standards are read with **read_defect_standard** from the documents of **config.DOCS_DIRECTORY**; no knowledge index is needed.
- The repeat model learns from **config.CLEAN_REPORT_HISTORY_TRAIN_FILE**, with each report's requirement counts under its own standards and its coordination trigger flags, and is scored on **config.CLEAN_REPORT_HISTORY_EVAL_FILE**

prepared the same way.
- Its target is encoded by **config.REPEAT_TARGET_ENCODING**; an answer the encoding does not know is refused, naming the column and the values.
- The crew hours model learns from **config.CLEAN_REPAIR_LOGS_TRAIN_FILE**, with each log's coordination trigger flags, and is scored on **config.CLEAN_REPAIR_LOGS_EVAL_FILE**; the inputs of both tabular models come from **build_feature_frame**.
- The sentences of **config.CLEAN_STATEMENT_SENTENCES_FILE** are split once, **config.SENTENCE_HELD_OUT_SHARE** held out, stratified by sentence type, with **config.RANDOM_SEED**.
- A model below its minimum, **config.MINIMUM_REPEAT_ROC_AUC**, **config.MINIMUM_CREW_HOURS_R2** or **config.MINIMUM_SENTENCE_MACRO_F1**, stops the run before anything is saved, naming every shortfall.
- Otherwise the three models and the two schemas are saved to their paths in config, each schema exactly the columns its model was fitted on, in order.
- **config.MODEL_METRICS_FILE** records under **repeat**, **crew_hours** and **sentence** the **algorithm**, **training_rows**, **held_out_rows**, **scores**, **minimum_required** and **scikit_learn_version**.
- **algorithm** is the class name of the **config.MODEL_STEP_NAME** step; for the sentence model it is every step's class name joined by a space, a plus sign and a space.
- The console gives each model's held-out score, training score and gap, and the crew hours model's mean absolute error; it names the metrics file and says every model reaches its minimum.

RETURNS

None.

MAIN BLOCK

- Running the file trains, scores and saves the three models; importing the module trains nothing.

MODULE: run_knowledge_indexing.py

Builds the knowledge base and reads it back.

FUNCTION SIGNATURE

```
main()
```

PARAMETERS

None.

PURPOSE

Turn the sixteen documents into the store the whole system reads from, and show that reading it works.

REQUIREMENTS

- The store at **config.VECTOR_STORE_DIRECTORY** is rebuilt from every document in **config.DOCS_DIRECTORY**.
- The console says how many documents were read, how many sections were found, how many passages were stored and where the store is.
- What an inspection records under each defect standard is read back from the reopened store with **read_inspection_records** and printed under its defect type and its document.
- A section that cannot be read stops the run with its refusal.

RETURNS

None.

MAIN BLOCK

- Running the file builds the store and reads it back; importing the module builds nothing.

MODULE: run_api_service.py

Starts the REST service, after checking that everything it needs is on disk.

FUNCTION SIGNATURE

```
main()
```

PARAMETERS

None.

PURPOSE

Start the service where the earlier stages have been run, and explain itself where they have not.

REQUIREMENTS

- Every path of **config.SERVICE_REQUIRED_FILES** is checked before anything is imported from the service.
- Those paths are the two cleaned history files, the three models, the two schemas, the three live files, the documents folder, the store's **config.VECTOR_STORE_FILE_NAME** and the database, each paired with what produces it.
- Where something is missing, the console says the service cannot start and names each missing path with what produces it.
- Otherwise the console says where the interactive documentation is, and the service starts on **config.API_HOST** and **config.API_PORT**, **8081**.

RETURNS

int. **0** once the service has run, **1** when it could not start.

MAIN BLOCK

- Running the file makes **main**'s result the process's exit status; importing the module starts nothing.

MODULE: run_dashboard.py
The dashboard's front door: the service check, signing in, and the navigation.
MODULE-LEVEL OBJECTS

- **PAGE_RENDERERS** — **dict**. Each page name in config paired with the function that draws it.

FUNCTION SIGNATURE
`main()`
PARAMETERS
None.
PURPOSE
Give the staff one screen to work from, with only the pages the signed-in person's role is offered.
REQUIREMENTS

- The page is titled **config.DASHBOARD_TITLE** with **config.DASHBOARD_ICON** and carries **config.DASHBOARD_PAGE_STYLE**.
- A service not answering at **config.API_BASE_URL** is reported, naming the address and how to start it, and nothing else is drawn.
- Until someone signs in, only the sign-in page is drawn, whichever page address was opened.
- Once signed in, the sidebar shows the person's name, role and any district, and a way to sign out that ends the session at the service.
- The pages offered are exactly those the service returned for the role, in that order, each with its renderer, its icon from **config.PAGE_ICONS** and its address path from **config.PAGE_URL_PATHS**.

RETURNS
None. The dashboard draws itself.
MAIN BLOCK

- Running the file through Streamlit draws the dashboard on port **8080**; importing the module draws nothing.

OUTPUT
Running the six entry points in order produces the following, all inside the **Project** folder.

FILES CREATED

Path	Created by	Contents
database/streetcare.db	**run_database_setup.py**	The six tables the system writes to, and the sixteen seed accounts
processed/report_history_train_clean.csv	**run_data_pipeline.py**	Cleaned repeat model training data
processed/report_history_eval_clean.csv	**run_data_pipeline.py**	Cleaned repeat model held-out data
processed/repair_logs_train_clean.csv	**run_data_pipeline.py**	Cleaned crew hours model training data
processed/repair_logs_eval_clean.csv	**run_data_pipeline.py**	Cleaned crew hours model held-out data
processed/statement_sentences_clean.csv	**run_data_pipeline.py**	Cleaned statement sentences
output/data_quality_report.json	**run_data_pipeline.py**	How dirty each raw extract was, and what cleaning did to it
output/models/repeat_model.pkl	**run_model_training.py**	The saved repeat model
output/models/crew_hours_model.pkl	**run_model_training.py**	The saved crew hours model
output/models/sentence_model.pkl	**run_model_training.py**	The saved sentence model, text handling included
output/models/repeat_features.json	**run_model_training.py**	The columns the repeat model expects, in order
output/models/crew_hours_features.json	**run_model_training.py**	The columns the crew hours model expects, in order
output/model_metrics.json	**run_model_training.py**	Every score of every saved model
database/chroma_store/	**run_knowledge_indexing.py**	The searchable knowledge base
database/checkpoint.db	**run_api_service.py**	Every run's workflow state, checkpointed after each node, so that a run paused at the Highways Inspector's gate can be resumed

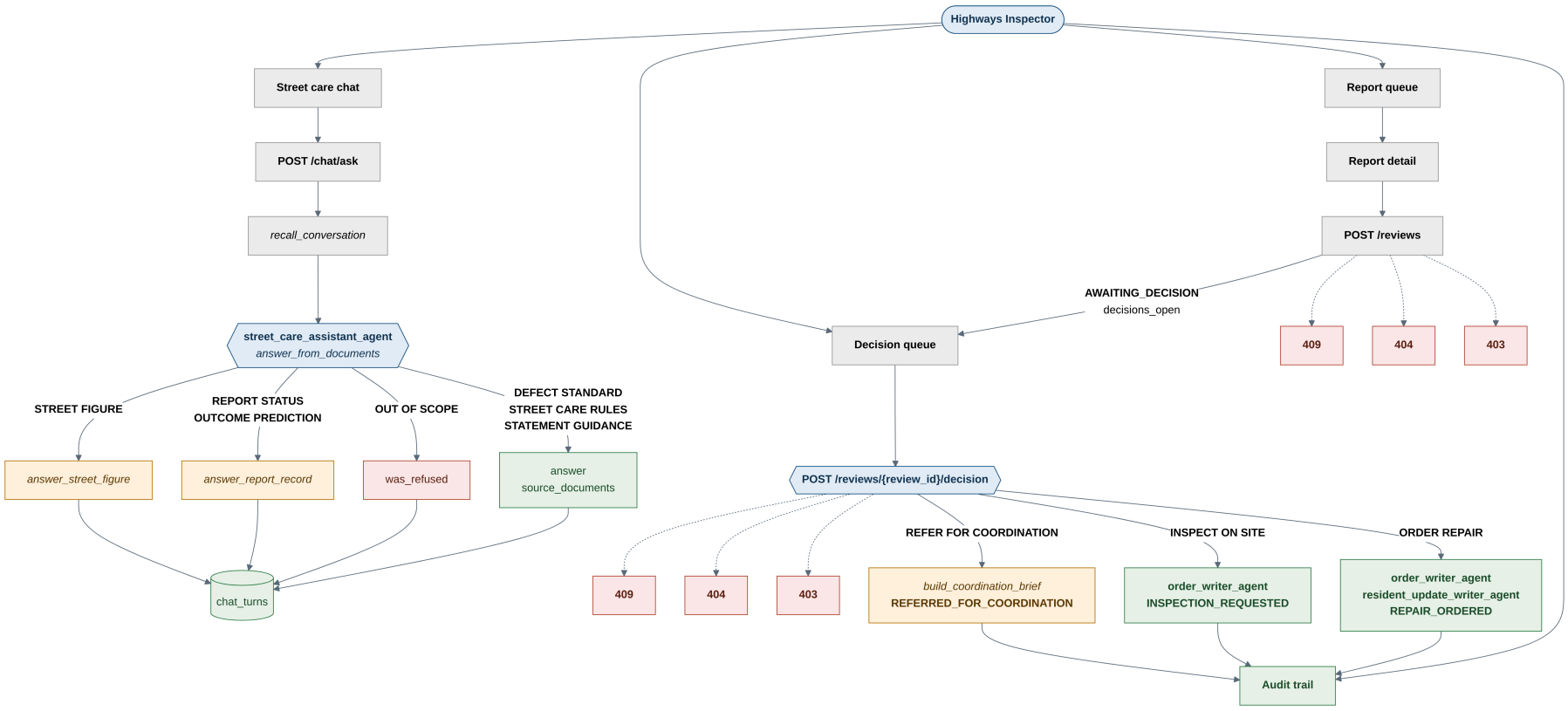
INFORMATION EACH COMMAND MUST REPORT

Command	What it must make clear
run_database_setup.py	Where the database is, which tables exist, and how many accounts were created and were already present
run_data_pipeline.py	For each of the five extracts, how many rows arrived and how many survived cleaning, and where the quality report was written
run_model_training.py	For each of the three models, the score on data it did not learn from, the score on data it did and the gap, and that every model reaches its minimum
run_knowledge_indexing.py	How many documents were read, how many sections were found and how many passages were stored, and what an inspection records under each defect standard, read back from the store
run_api_service.py	That the service is starting on port **8081**, and where its interactive documentation can be opened
run_dashboard.py	That the dashboard is listening on port **8080**, and the address at which it can be opened

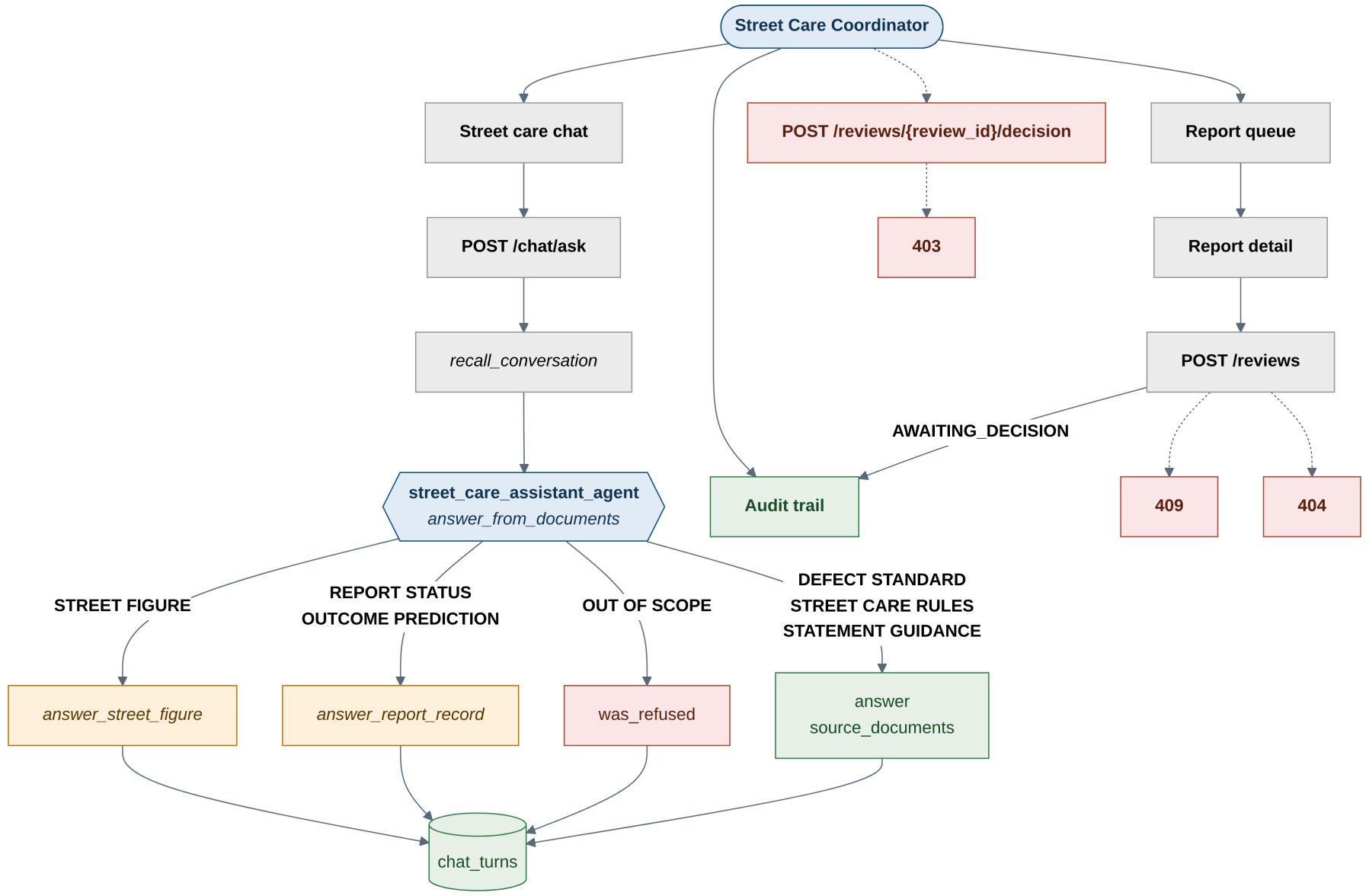
FLOW DIAGRAM:

Role Highways Inspector:

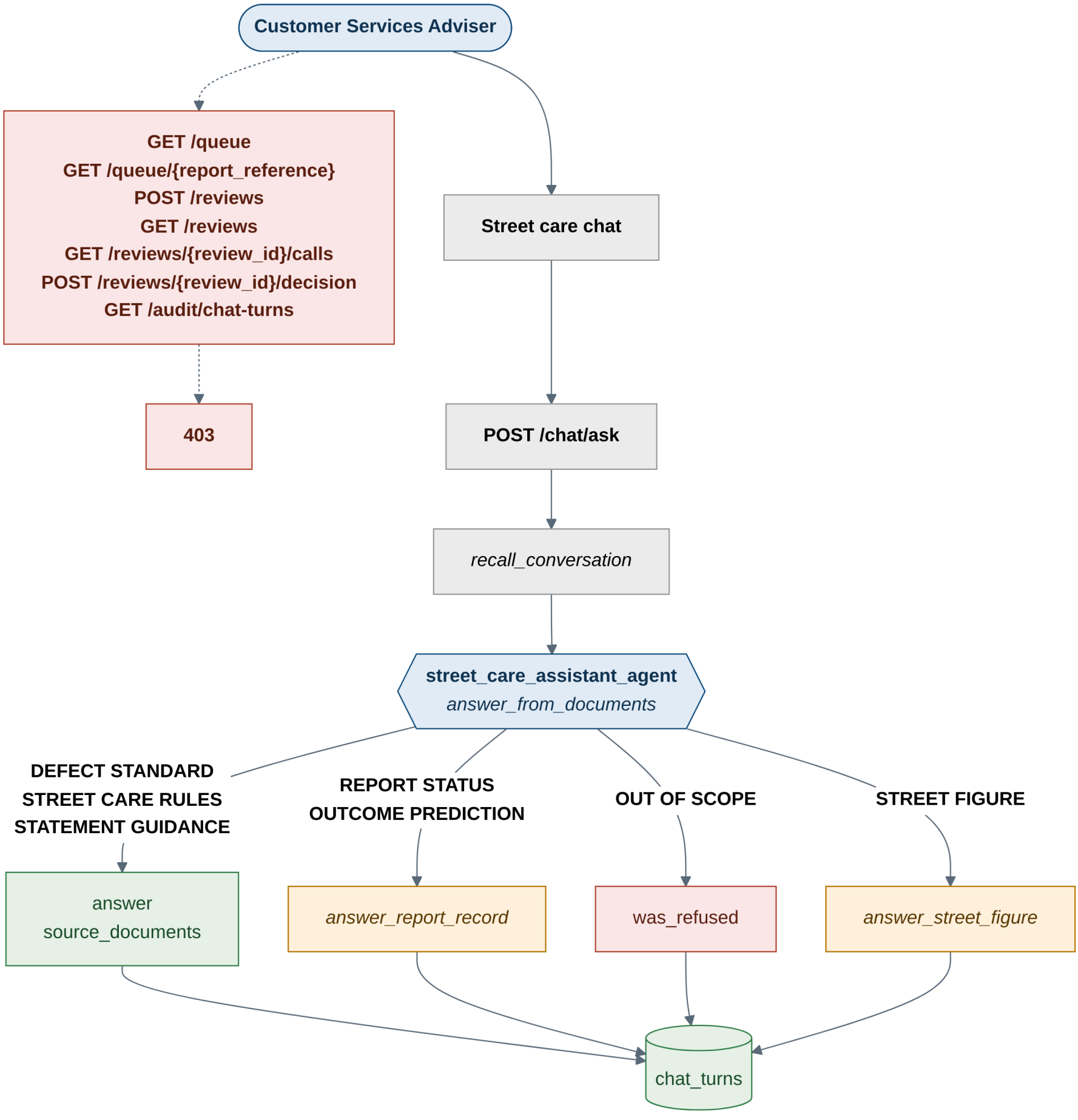
Role Street Care Coordinator:



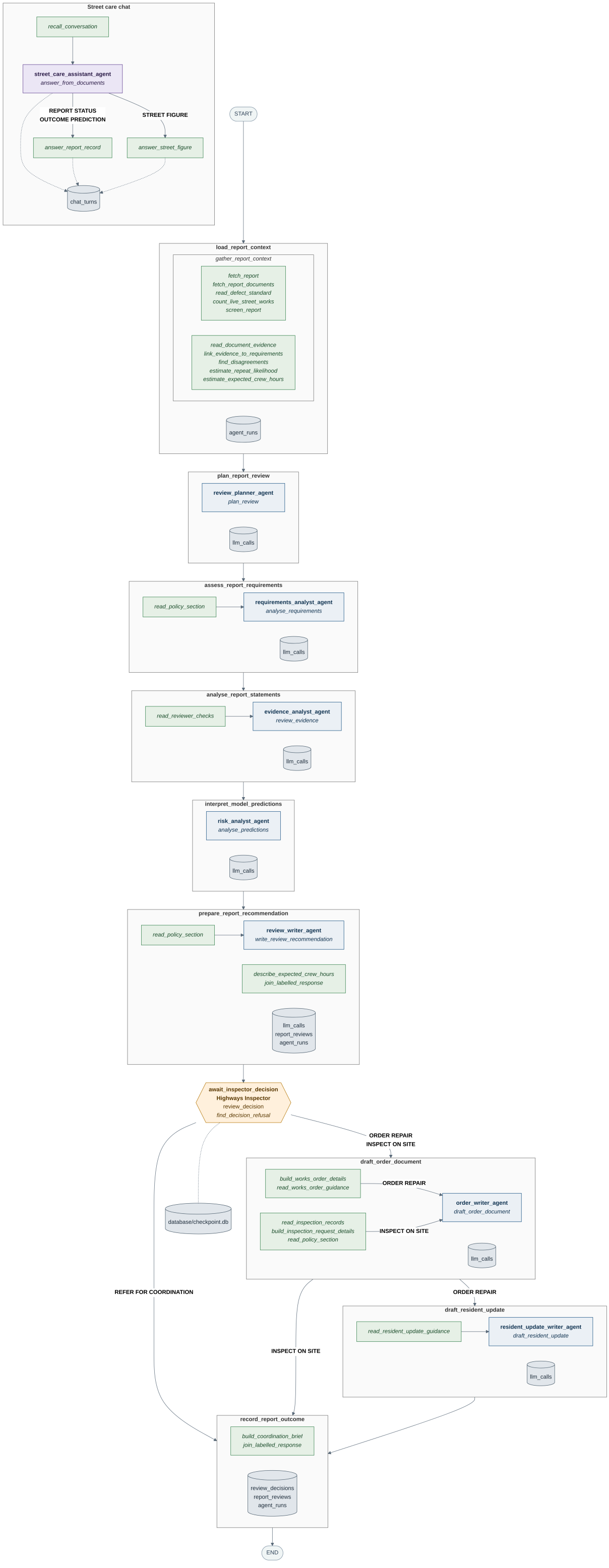
Role Street Care Coordinator:



Role Customer Services Adviser:



Agent to Agent Communication Flow:


Street care chat
recall_conversation
street_care_assistant_agent
answer_from_documents
REPORT STATUS
OUTCOME PREDICTION
STREET FIGURE
answer_report_record
answer_street_figure
chat_turns
START
load_report_context
gather_report_context
fetch_report
fetch_report_documents
read_defect_standard
count_live_street_works
screen_report
read_document_evidence
link_evidence_to_requirements
find_disagreements
estimate_repeat_likelihood
estimate_expected_crew_hours
agent_runs
plan_report_review
review_planner_agent
plan_review
llm_calls
assess_report_requirements
read_policy_section
requirements_analyst_agent
analyse_requirements
llm_calls
analyse_report_statements
read_reviewer_checks
evidence_analyst_agent
review_evidence
llm_calls
interpret_model_predictions
risk_analyst_agent
analyse_predictions
llm_calls
prepare_report_recommendation
read_policy_section
review_writer_agent
write_review_recommendation
describe_expected_crew_hours
join_labelled_response
llm_calls
report_reviews
agent_runs
await_inspector_decision
Highways Inspector
review_decision
find_decision_refusal
ORDER REPAIR
INSPECT ON SITE
draft_order_document
build_works_order_details
read_works_order_guidance
ORDER REPAIR
order_writer_agent
draft_order_document
read_inspection_records
build_inspection_request_details
read_policy_section
INSPECT ON SITE
llm_calls
database/checkpoint.db
REFER FOR COORDINATION
ORDER REPAIR
draft_resident_update
read_resident_update_guidance
resident_update_writer_agent
draft_resident_update
llm_calls
INSPECT ON SITE
record_report_outcome
build_coordination_brief
join_labelled_response
review_decisions
report_reviews
agent_runs
END

EXPECTED OUTPUT

The transcript below is one correct solution's output, shown so that the shape of each report is clear. The scores are shown as **N.NNNN** because they vary with how each team prepares the models' inputs, and the passage count follows from how the documents are cut. Each gap is the training score less the held-out score, printed with its sign, so it may be negative. The row counts, the document count and the section count should be these. The sections read back are shown in outline only: they come from the documents, and reading them is the work.

ENTRY POINT 1 — run_database_setup.py

```
database: database/streetcare.db
  tables: agent_runs, chat_turns, llm_calls, report_reviews, review_decisions, user_accounts
  account created  amelia   Amelia Hart      Highways Inspector
  account created  owen     Owen Parry       Highways Inspector
  ...
  account created  lewis    Lewis Grant      Customer Services Adviser
  accounts created: 16, already present: 0
```

Run a second time it creates nothing and reports **accounts created: 0, already present: 16**.

ENTRY POINT 2 — run_data_pipeline.py

```
report_history_train            32738 received ->  30569 kept  (2169 removed)
report_history_eval              8202 received ->   7648 kept  (554 removed)
repair_logs_train               26095 received ->  25537 kept  (558 removed)
repair_logs_eval                 6476 received ->   6337 kept  (139 removed)
statement_sentences             36661 received ->  35717 kept  (944 removed)

quality report written to output/data_quality_report.json
```

ENTRY POINT 3 — run_model_training.py

```
training three models
  repeat          held-out ROC AUC N.NNNN  (train N.NNNN, gap +N.NNNN)
  crew hours      held-out R2 N.NNNN  MAE N.NNNN hours  (train N.NNNN, gap +N.NNNN)
  sentence        held-out macro F1 N.NNNN  (train N.NNNN, gap +N.NNNN)

metrics written to output/model_metrics.json
every model reaches the minimum the configuration sets for it
```

A model below its minimum stops the run with a refusal naming it, and nothing is saved.

ENTRY POINT 4 — run_knowledge_indexing.py

```
16 documents, 136 sections, NNN passages indexed in database/chroma_store
what an inspection records under each defect standard, read back from the index
  Carriageway  Defect-Standard-Carriageway.pdf
      When a report is sent for inspection on site, the inspector who visits records ...
  Footway  Defect-Standard-Footway.pdf
      ...
  Drainage  Defect-Standard-Drainage.pdf
      ...
  Vegetation  Defect-Standard-Vegetation.pdf
      ...
  Street lighting  Defect-Standard-Street-Lighting.pdf
      ...
```

Each section is its own standard's section **config.INSPECTION_RECORDS_SECTION_NUMBER**, whole and without its heading. Two defect types read back with the same section, or a section from another document, is the failure this stage exists to catch.

ENTRY POINT 5 — run_api_service.py

```
Starting StreetCare AI API on port 8081. Interactive documentation: http://localhost:8081/docs
INFO:     Started server process [NNNN]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8081 (Press CTRL+C to quit)
```

Run with the saved repeat model missing, it names what is missing and stops:

```
The service cannot start.
  output/models/repeat_model.pkl is missing; it comes from run_model_training.py
```

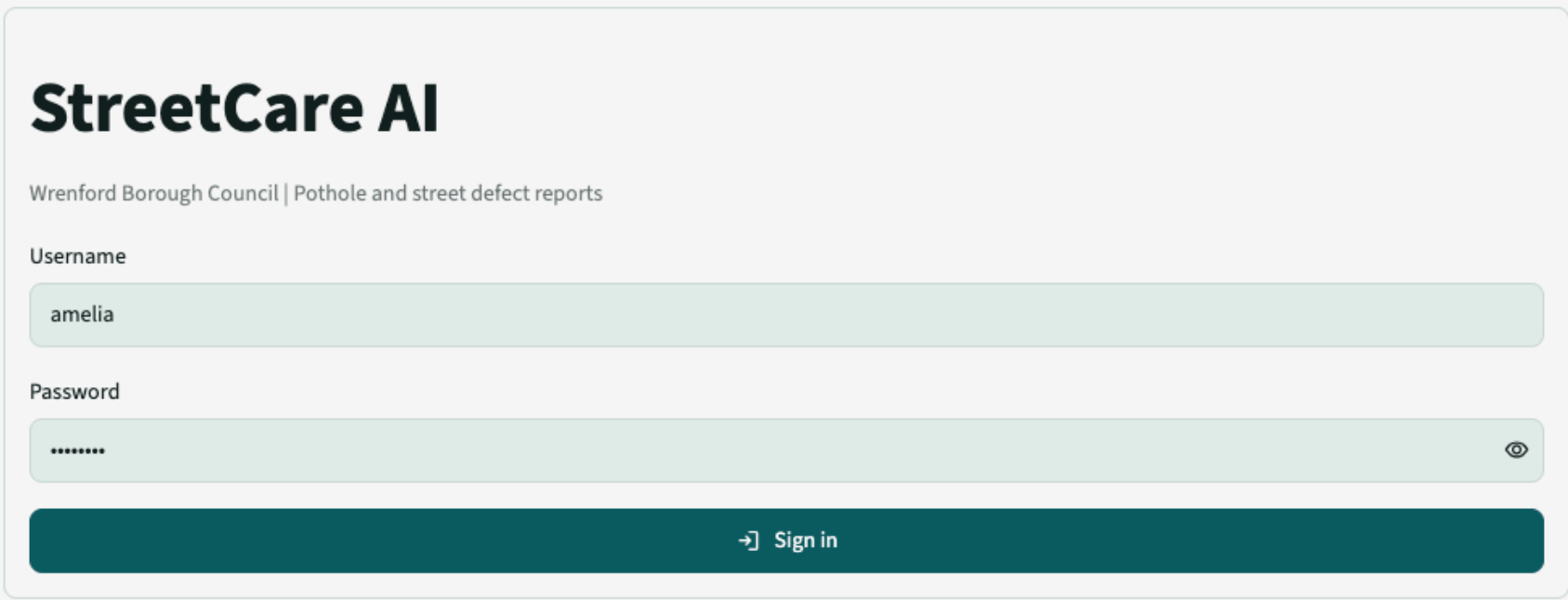
ENTRY POINT 6 — run_dashboard.py

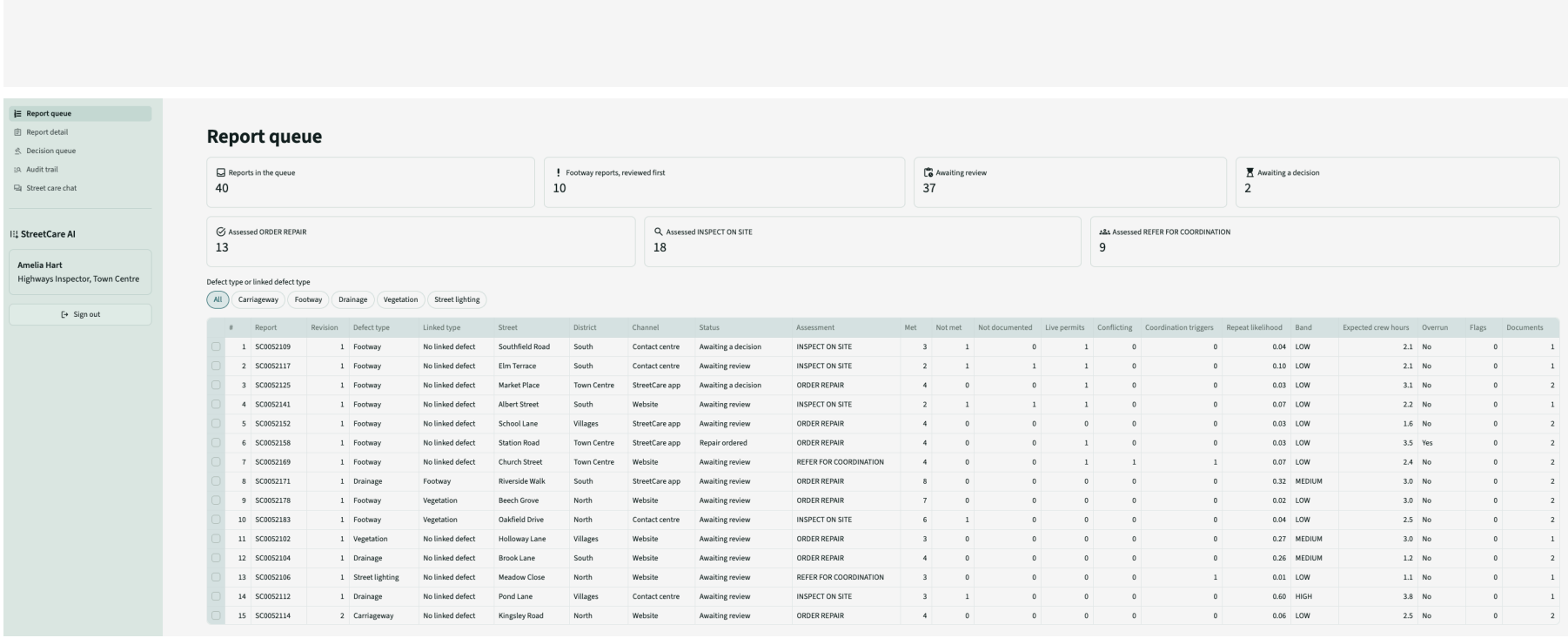
```
  You can now view your Streamlit app in your browser.
```

```
  Local URL: http://localhost:8080
  Network URL: http://N.N.N.N:8080
```

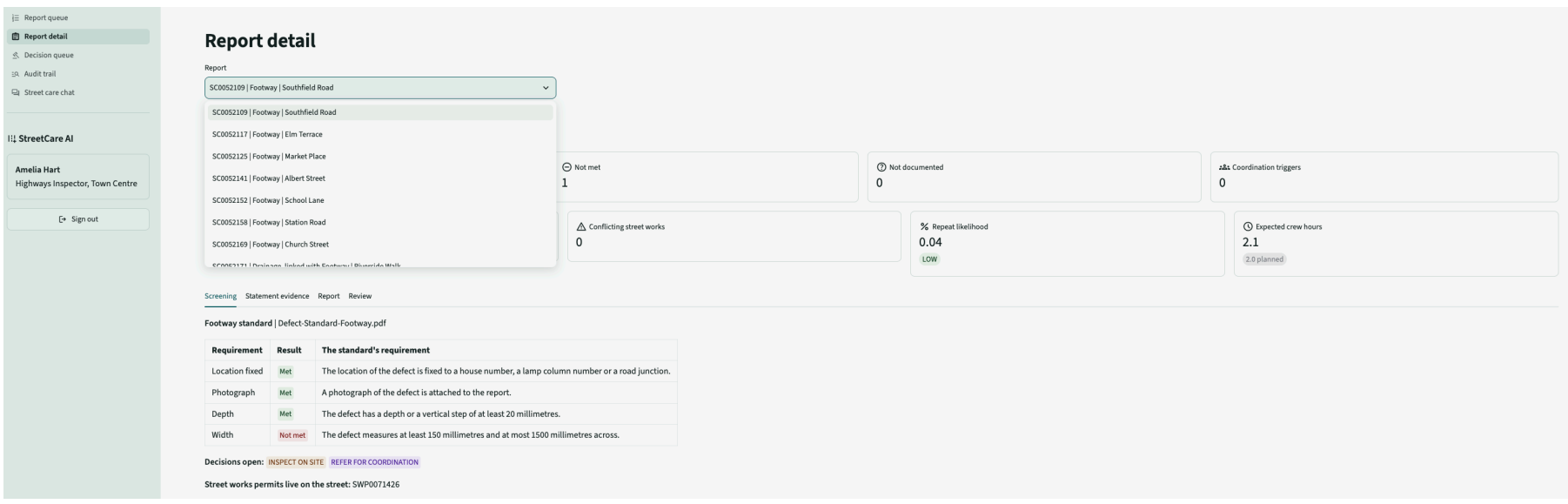
On some machines Streamlit also prints an External URL line.

DASHBOARD OUTPUT:

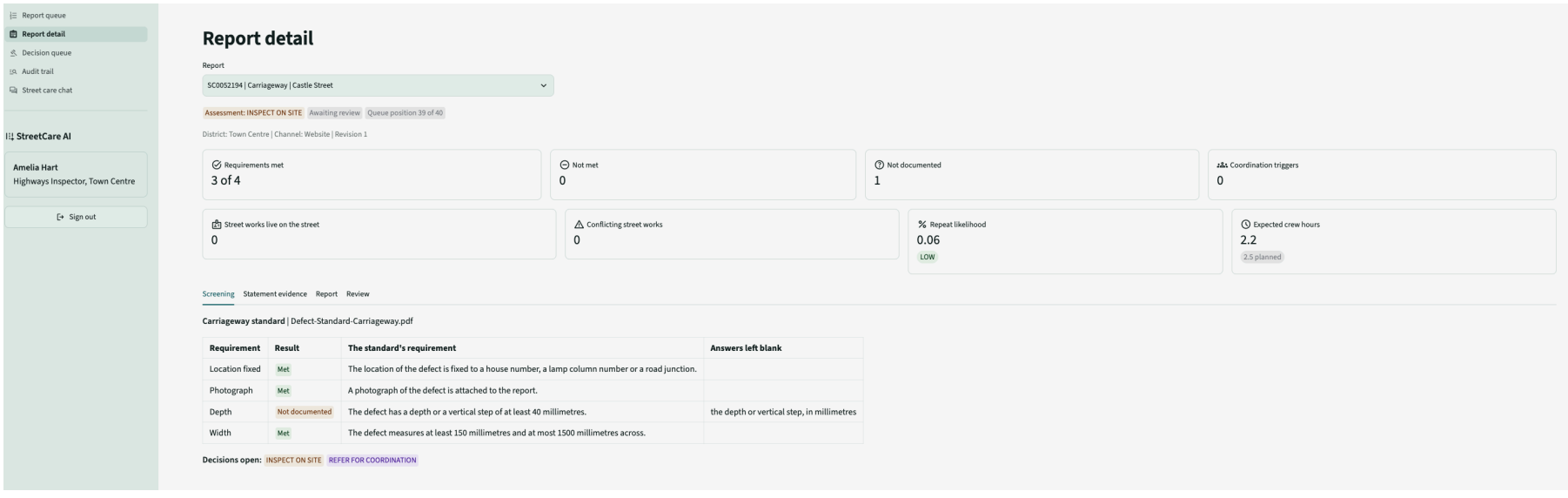




#	Report	Revision	Defect type	Linked type	Street	District	Channel	Status	Assessment	Met	Not met	Not documented	Live permits	Conflicting	Coordination triggers	Repeat likelihood	Band	Expected crew hours	Overrun	Flags	Documents
1	SC0052109	1	Footway	No linked defect	Southfield Road	South	Contact centre	Awaiting a decision	INSPECT ON SITE	3	1	0	1	0	0	0.04	LOW	2.1	No	0	1
2	SC0052117	1	Footway	No linked defect	Elm Terrace	South	Contact centre	Awaiting review	INSPECT ON SITE	2	1	1	1	0	0	0.10	LOW	2.1	No	0	1
3	SC0052125	1	Footway	No linked defect	Market Place	Town Centre	StreetCare app	Awaiting a decision	ORDER REPAIR	4	0	0	1	0	0	0.03	LOW	3.1	No	0	2
4	SC0052141	1	Footway	No linked defect	Albert Street	South	Website	Awaiting review	INSPECT ON SITE	2	1	1	1	0	0	0.07	LOW	2.2	No	0	1
5	SC0052152	1	Footway	No linked defect	School Lane	Villages	StreetCare app	Awaiting review	ORDER REPAIR	4	0	0	0	0	0	0.03	LOW	1.6	No	0	2
6	SC0052158	1	Footway	No linked defect	Station Road	Town Centre	StreetCare app	Repair ordered	ORDER REPAIR	4	0	0	1	0	0	0.03	LOW	3.5	Yes	0	2
7	SC0052169	1	Footway	No linked defect	Church Street	Town Centre	Website	Awaiting review	REFER FOR COORDINATION	4	0	0	1	1	1	0.07	LOW	2.4	No	0	2
8	SC0052171	1	Drainage	Footway	Riverside Walk	South	StreetCare app	Awaiting review	ORDER REPAIR	8	0	0	0	0	0	0.32	MEDIUM	3.0	No	0	2
9	SC0052178	1	Footway	Vegetation	Beech Grove	North	Website	Awaiting review	ORDER REPAIR	7	0	0	0	0	0	0.02	LOW	3.0	No	0	2
10	SC0052183	1	Footway	Vegetation	Oakfield Drive	North	Contact centre	Awaiting review	INSPECT ON SITE	6	1	0	0	0	0	0.04	LOW	2.5	No	0	2
11	SC0052102	1	Vegetation	No linked defect	Holloway Lane	Villages	Website	Awaiting review	ORDER REPAIR	3	0	0	0	0	0	0.27	MEDIUM	3.0	No	0	1
12	SC0052104	1	Drainage	No linked defect	Brook Lane	South	Website	Awaiting review	ORDER REPAIR	4	0	0	0	0	0	0.26	MEDIUM	1.2	No	0	2
13	SC0052106	1	Street lighting	No linked defect	Meadow Close	North	Website	Awaiting review	REFER FOR COORDINATION	3	0	0	0	0	1	0.01	LOW	1.1	No	0	1
14	SC0052112	1	Drainage	No linked defect	Pond Lane	Villages	Contact centre	Awaiting review	INSPECT ON SITE	3	1	0	0	0	0	0.60	HIGH	3.8	No	0	1
15	SC0052114	2	Carriageway	No linked defect	Kingsley Road	North	Website	Awaiting review	ORDER REPAIR	4	0	0	0	0	0	0.06	LOW	2.5	No	0	2



Requirement	Result	The standard's requirement
Location fixed	Met	The location of the defect is fixed to a house number, a lamp column number or a road junction.
Photograph	Met	A photograph of the defect is attached to the report.
Depth	Met	The defect has a depth or a vertical step of at least 20 millimetres.
Width	Not met	The defect measures at least 150 millimetres and at most 1500 millimetres across.



Requirement	Result	The standard's requirement	Answers left blank
Location fixed	Met	The location of the defect is fixed to a house number, a lamp column number or a road junction.	
Photograph	Met	A photograph of the defect is attached to the report.	
Depth	Not documented	The defect has a depth or a vertical step of at least 40 millimetres.	the depth or vertical step, in millimetres
Width	Met	The defect measures at least 150 millimetres and at most 1500 millimetres across.	

Report queue
Report detail
Decision queue
Audit trail
Street care chat

StreetCare AI

Amelia Hart
Highways Inspector, Town Centre

Sign out

Report detail

Report

SC0052194 | Carriageway | Castle Street

Assessment: INSPECT ON SITE · Awaiting review · Queue position 39 of 40

District: Town Centre | Channel: Website | Revision 1

Requirements met	Not met	Not documented	Coordination triggers
3 of 4	0	1	0

Street works live on the street	Conflicting street works	Repeat likelihood	Expected crew hours
0	0	0.06 LOW	2.2 2.5 planned

Screening · Statement evidence · Report · Review

Resident description | DOC-841226

Location Just to say, it is near number 157, by the driveway.
Bears on Carriageway: Location fixed

Measurement It is about 45 mm deep.
Bears on Carriageway: Depth, Width
DESCRIBES BLANK ANSWER: the depth or vertical step, in millimetres: the document states 45, the report gives not given.

Incident My husband tripped here and was hurt.

Measurement It is roughly 300 mm wide.
Bears on Carriageway: Depth, Width

Hazard It is right in the wheel track of the cars on Castle Street.

Traffic and use Castle Street is a cul-de-sac used mostly by residents.

Photo note | DOC-841227

Measurement The photo shows the hole is 300 mm wide against the tape.
Bears on Carriageway: Depth, Width

Photograph The photos show the hole in the road and the nearest house.
Bears on Carriageway: Photograph

Report queue
Report detail
Decision queue
Audit trail
Street care chat

StreetCare AI

Amelia Hart
Highways Inspector, Town Centre

Sign out

Report detail

Report

SC0052194 | Carriageway | Castle Street

Assessment: INSPECT ON SITE · Awaiting review · Queue position 39 of 40

District: Town Centre | Channel: Website | Revision 1

Requirements met	Not met	Not documented	Coordination triggers
3 of 4	0	1	0

Street works live on the street	Conflicting street works	Repeat likelihood	Expected crew hours
0	0	0.06 LOW	2.2 2.5 planned

Screening · Statement evidence · Report · Review

Question	Answer	Used for
Where on the street	outside no. 157	Works order
Whether the location is fixed to a house number, a lamp column number or a road junction	Yes	Location fixed (Carriageway)
Whether a photograph is attached	Yes	Photograph (Carriageway)
The depth or vertical step, in millimetres	not given	Depth (Carriageway), Works order
The width across, in millimetres	300	Width (Carriageway), Works order
The street category	Local road	Traffic-sensitive street trigger

Report queue
Report detail
Decision queue
Audit trail
Street care chat

StreetCare AI

Amelia Hart
Highways Inspector, Town Centre

Sign out

Report detail

Report

SC0052194 | Carriageway | Castle Street

Assessment: INSPECT ON SITE · Awaiting review · Queue position 39 of 40

District: Town Centre | Channel: Website | Revision 1

Requirements met	Not met	Not documented	Coordination triggers
3 of 4	0	1	0

Street works live on the street	Conflicting street works	Repeat likelihood	Expected crew hours
0	0	0.06 LOW	2.2 2.5 planned

Screening · Statement evidence · Report · Review

What should the review look at?

Need more info.

Start the review

The agents are reviewing the report: 5 model calls. (6.2 seconds)

Report queue
Report detail
Decision queue
Audit trail
Street care chat

StreetCare AI

Amelia Hart
Highways Inspector, Town Centre

Sign out

Report detail

Report

SC0052194 | Carriageway | Castle Street

Assessment: INSPECT ON SITE · Awaiting a decision · Queue position 39 of 40

District: Town Centre | Channel: Website | Revision 1

Requirements met	Not met	Not documented	Coordination triggers
3 of 4	0	1	0

Street works live on the street	Conflicting street works	Repeat likelihood	Expected crew hours
0	0	0.06 LOW	2.2 2.5 planned

Screening · Statement evidence · Report · Review

Recommended: INSPECT ON SITE REV-601209195759 | opened by amelia

Report SC0052194 meets three requirements for its carriageway defect type, featuring a fixed location outside number 157 on Castle Street, an attached photograph, and a width of 300 millimetres. However, the depth requirement is not documented because the depth or vertical step was left blank on the initial screening report. Supplementary evidence from the attached resident description and photo note (DOC-841226 and DOC-841227) clarifies that the defect has an unmeasured depth of 45 mm. No live conflicting street works permits or coordination triggers apply to Castle Street. Because a requirement was left not documented on the report, the screening assessment assigns an inspection. Therefore, INSPECT ON SITE fits and is the recommended decision so the inspector can verify the depth on site before any repair takes place.

Report queue
Report detail
Decision queue
Audit trail
Street care chat

StreetCare AI

Amelia Hart
Highways Inspector, Town Centre

Sign out

Report detail

Report

SC0052189 | Drainage | Willow Crescent

Assessment: INSPECT ON SITE · Awaiting review · Queue position 37 of 40

District: South | Channel: Contact centre | Revision 1

Requirements met	Not met	Not documented	Coordination triggers
2 of 4	0	2	0

Street works live on the street	Conflicting street works	Repeat likelihood	Expected crew hours
0	0	0.20 MEDIUM	1.8 1.5 planned

Screening · Statement evidence · Report · Review

Willow Crescent is in South, not in Town Centre: its review is opened by a Street Care Coordinator or by a Highways Inspector appointed for South.

Report queue
Report detail
Decision queue
Audit trail
Street care chat

StreetCare AI

Amelia Hart
Highways Inspector, Town Centre

Sign out

Decision queue

Appointed for Town Centre: High Street, Market Place, Church Street, Station Road, Bridge Street, King Street, Castle Street, Mill Lane, Chapel Row, Bank Parade, Sheep Street, Wool Lane

Review to decide (2 waiting)

SC0052125 | Footway | Market Place

SC0052125 | Footway | Market Place
SC0052194 | Carriageway | Castle Street

Repeat likelihood
0.03
LOW

Expected crew hours
3.1
2.5 planned

Recommendation

Street defect report SC0052125 for a footway defect opposite no. 50 Market Place, Town Centre successfully meets all four screening requirements for location, photograph, depth, and width. The report's own answers record a depth of 20 mm and a width of 400 mm, satisfying the footway standard criteria. Furthermore, the resident description (DOC-841233) and photo note (DOC-841234) confirm these measurements through visual scaling with a shoe, a ruler, and a tape measure. There are zero conflicting live street works permits and no applicable coordination triggers. Therefore, the policy assessment is ORDER REPAIR, and as all requirements are met with no coordination triggers, this decision fits best.

Footway standard

Requirement	Result
Location fixed	Met
Photograph	Met
Depth	Met
Width	Met

Coordination triggers: none
Street works permits live on the street: SWP0071384

› The review, agent by agent

Report queue
Report detail
Decision queue
Audit trail
Street care chat

StreetCare AI

Amelia Hart
Highways Inspector, Town Centre

Sign out

Decision queue

Appointed for Town Centre: High Street, Market Place, Church Street, Station Road, Bridge Street, King Street, Castle Street, Mill Lane, Chapel Row, Bank Parade, Sheep Street, Wool Lane

Review to decide (2 waiting)

SC0052125 | Footway | Market Place

Recommended: ORDER REPAIR

You opened this review, so the other Highways Inspector appointed for Town Centre must decide it.

Repeat likelihood
0.03
LOW

Expected crew hours
3.1
2.5 planned

Recommendation

Street defect report SC0052125 for a footway defect opposite no. 50 Market Place, Town Centre successfully meets all four screening requirements for location, photograph, depth, and width. The report's own answers record a depth of 20 mm and a width of 400 mm, satisfying the footway standard criteria. Furthermore, the resident description (DOC-841233) and photo note (DOC-841234) confirm these measurements through visual scaling with a shoe, a ruler, and a tape measure. There are zero conflicting live street works permits and no applicable coordination triggers. Therefore, the policy assessment is ORDER REPAIR, and as all requirements are met with no coordination triggers, this decision fits best.

Footway standard

Requirement	Result
Location fixed	Met
Photograph	Met
Depth	Met
Width	Met

Coordination triggers: none
Street works permits live on the street: SWP0071384

› The review, agent by agent

Report queue
Report detail
Decision queue
Audit trail
Street care chat

StreetCare AI

Amelia Hart
Highways Inspector, Town Centre

Sign out

Audit trail

Report reviews | Chat log

Status

Awaiting a decision | Repair ordered | Inspection requested | Referred for coordination

Report	Revision	Defect type	Linked type	Street	Status	Opened by	Decided by	Model calls
SC0052109	1	Footway	No linked defect	Southfield Road	Awaiting a decision	jonah		5
SC0052125	1	Footway	No linked defect	Market Place	Awaiting a decision	amelia		5
SC0052158	1	Footway	No linked defect	Station Road	Repair ordered	amelia	owen	7
SC0052194	1	Carriageway	No linked defect	Castle Street	Awaiting a decision	amelia		5

Report queue
Report detail
Decision queue
Audit trail
Street care chat

StreetCare AI

Jonah Ellis
Highways Inspector, South

Sign out

Report detail

Report

SC0052189 | Drainage | Willow Crescent

Assessment: INSPECT ON SITE | Awaiting review | Queue position 37 of 40

District: South | Channel: Contact centre | Revision 1

Requirements met
2 of 4

Not met
0

Not documented
2

Coordination triggers
0

Street works live on the street
0

Conflicting street works
0

Repeat likelihood
0.20
MEDIUM

Expected crew hours
1.8
1.5 planned

Screening | Statement evidence | Report | Review

Drainage standard | Defect-Standard-Drainage.pdf

Requirement	Result	The standard's requirement	Answers left blank
Location fixed	Met	The location of the defect is fixed to a house number, a lamp column number or a road junction.	
Photograph	Met	A photograph of the defect is attached to the report.	
Water depth	Not documented	The standing water is at least 50 millimetres deep.	the depth of standing water, in millimetres
Water reach	Not documented	Where the water stands on the carriageway, it reaches at least 1.0 metres out from the kerb.	where the water stands

Decisions open: INSPECT ON SITE | REFER FOR COORDINATION

Report queue
Report detail
Decision queue
Audit trail
Street care chat

StreetCare AI

Jonah Ellis
Highways Inspector, South

Sign out

Report detail

Report

SC0052189 | Drainage | Willow Crescent

Assessment: INSPECT ON SITE | Awaiting review | Queue position 37 of 40

District: South | Channel: Contact centre | Revision 1

Requirements met
2 of 4

Not met
0

Not documented
2

Coordination triggers
0

Street works live on the street
0

Conflicting street works
0

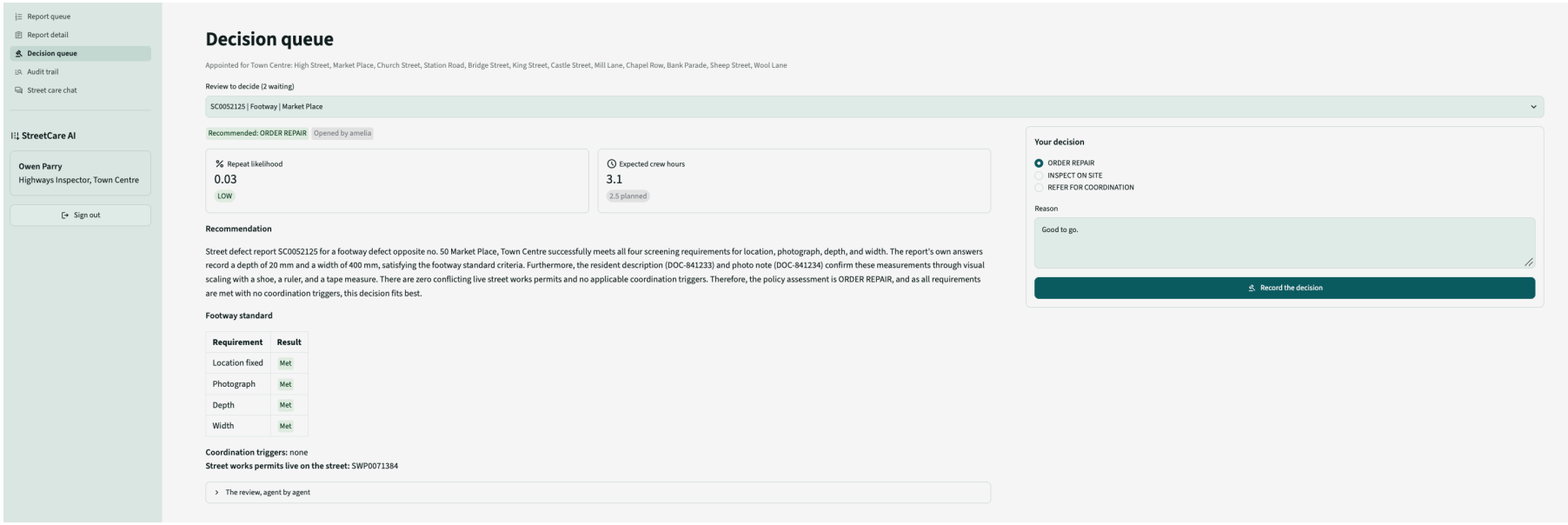
Repeat likelihood
0.20
MEDIUM

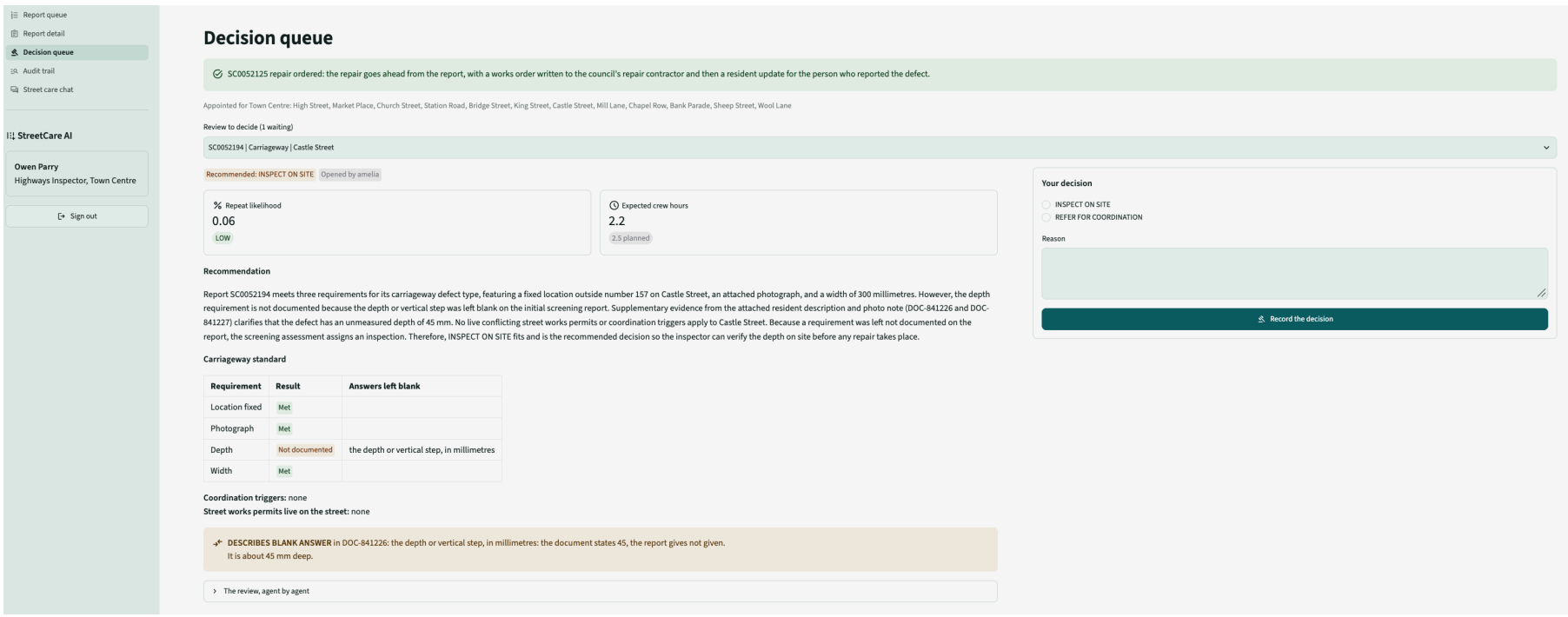
Expected crew hours
1.8
1.5 planned

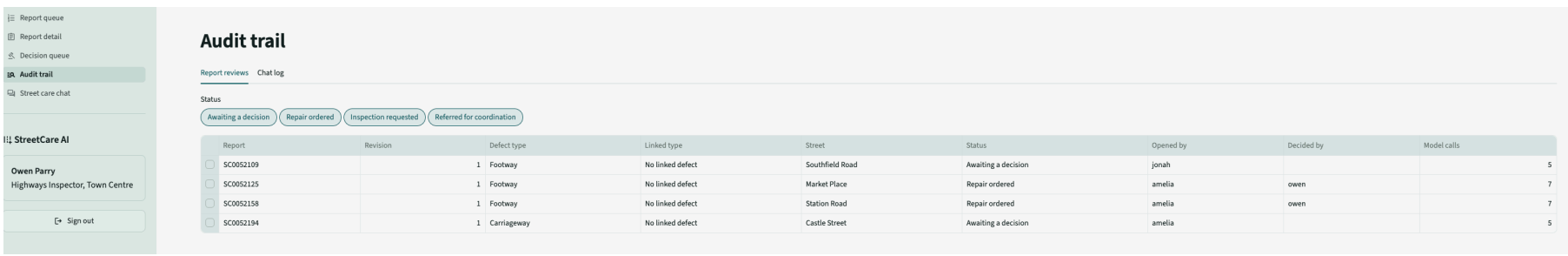
Screening | Statement evidence | Report | Review

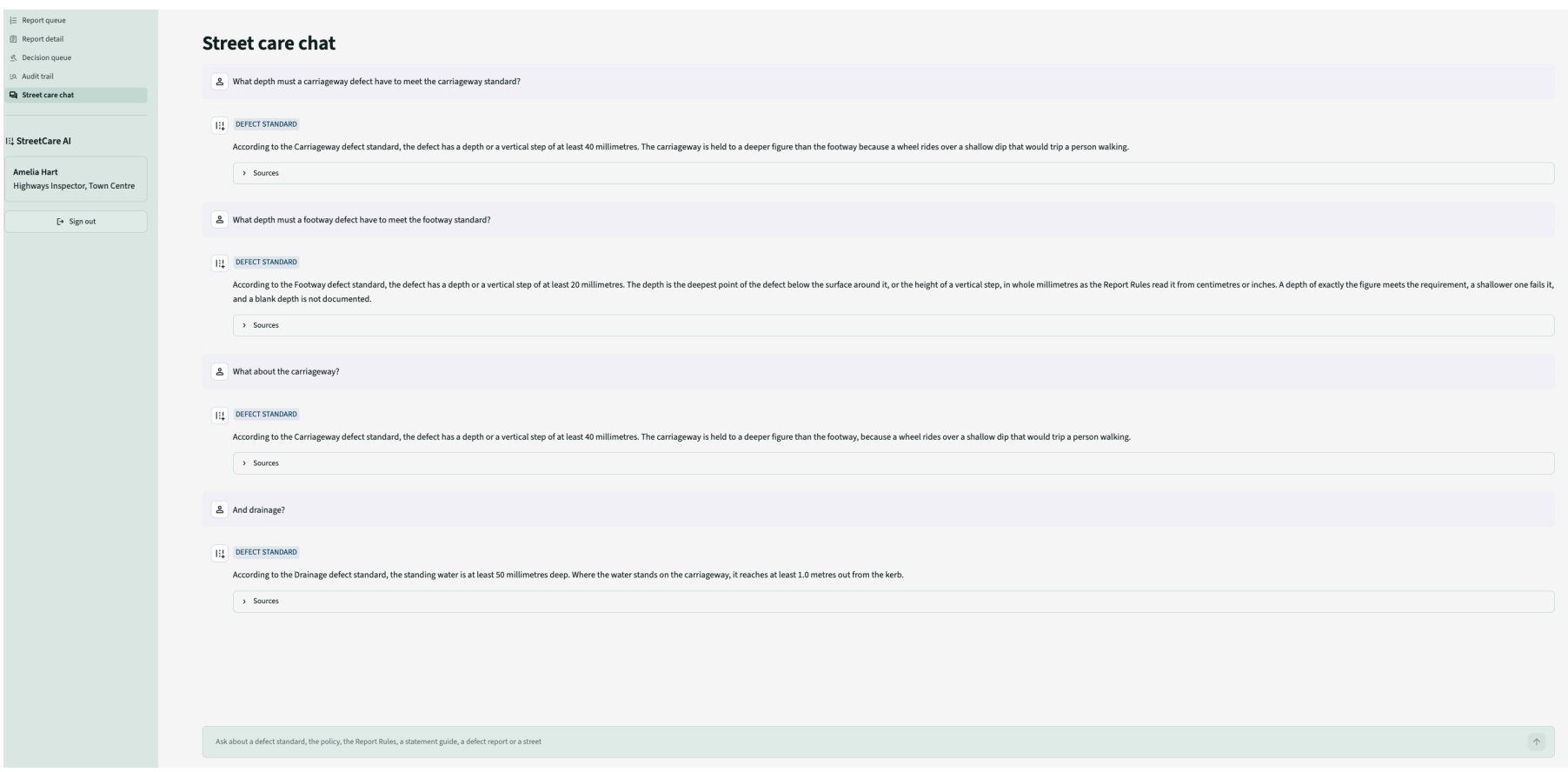
What should the review look at?

Start the review









IMPLEMENTATION STEPS

Every command is run from the Project folder.

STEP 0. INSTALL AND CONFIGURE

cd Project

pip install -r requirements.txt

STEP 1. CREATE THE DATABASE

python3 run_database_setup.py

STEP 2. CLEAN THE RAW DATA

python3 run_data_pipeline.py

STEP 3. TRAIN THE MODELS
python3 run_model_training.py

STEP 4. BUILD THE KNOWLEDGE BASE
python3 run_knowledge_indexing.py

STEP 5. START THE REST SERVICE
Leave this running and use a second terminal for anything else.
python3 run_api_service.py

STEP 6. START THE DASHBOARD
In a third terminal, with the service still running:
cd Project
python3 -m streamlit run run_dashboard.py